

Introduction to Algorithms

Notes from CLRS, 3rd Edition

Chapters 2–35

Sahil Raj

Codeforces: After_Dark

May 25, 2026

Contents

1	Getting Started	9
1.1	Insertion Sort	9
1.1.1	Complexity Analysis	9
1.2	Analyzing Algorithms	10
1.3	Merge Sort — Divide and Conquer	10
1.3.1	Recurrence and Complexity	11
2	Growth of Functions — Asymptotic Analysis	11
2.1	Why Asymptotic Analysis?	12
2.2	The Five Asymptotic Notations	12
2.3	Worked Examples	13
2.3.1	Example 1: Polynomial	13
2.3.2	Example 2: Logarithms	14
2.3.3	Example 3: Exponentials vs Polynomials	14
2.3.4	Example 4: Logs vs Powers	14
2.4	Common Growth Rates	14
2.5	Properties of Asymptotic Notation	15
2.6	Analyzing Code: From Loops to Big-O	15
2.6.1	Single Loop	15
2.6.2	Nested Loops	15
2.6.3	Dependent Inner Loop	16
2.6.4	Logarithmic Loop	16
2.6.5	Tricky: Harmonic Series	16
2.6.6	Amortized-Like Pattern	16
2.7	Standard Notations and Common Functions	17
2.8	Common Mistakes	17

3	Divide-and-Conquer	17
3.1	Recurrences	17
3.2	Solving Recurrences	17
3.2.1	Method 1: Substitution Method	17
3.2.2	Method 2: Recursion Tree	18
3.3	The Master Theorem	18
3.3.1	Examples	19
3.4	Strassen's Matrix Multiplication	19
4	Probabilistic Analysis and Randomized Algorithms	19
4.1	The Hiring Problem	19
4.2	Indicator Random Variables	20
4.3	Randomized Algorithms	20
4.4	Random Permutation (Fisher–Yates Shuffle)	21
5	Heapsort	21
5.1	Heaps	21
5.2	Core Heap Operations	22
5.2.1	Max-Heapify — $O(\lg n)$	22
5.2.2	Build-Max-Heap — $O(n)$	22
5.3	Heapsort Algorithm	23
5.4	Priority Queue	23
5.5	Correctness Proofs for Heap Operations	23
6	Quicksort	25
6.1	Algorithm	25
6.2	Complexity Analysis	25
6.3	Randomized Quicksort	26
7	Sorting in Linear Time	27
7.1	Lower Bound for Comparison Sorts	27
7.2	Counting Sort — $\Theta(n + k)$	27
7.3	Radix Sort — $\Theta(d(n + k))$	29
7.4	Bucket Sort — Expected $\Theta(n)$	29
8	Medians and Order Statistics	30
8.1	The Selection Problem	30
8.2	Randomized Select — Expected $\Theta(n)$	30
8.3	Worst-Case Linear Selection (Median of Medians)	31
8.3.1	Why Groups of 5?	31
8.4	Summary: Sorting & Selection	31
9	Elementary Data Structures	32
9.1	Stacks	32
9.1.1	Array-Based Implementation	32
9.1.2	C++ Implementation	33
9.2	Queues	34
9.2.1	Circular Array Implementation	34
9.3	Linked Lists	35

9.3.1	Operations and Complexity	36
9.3.2	Sentinels	37
9.4	Representing Rooted Trees	37
9.4.1	Binary Trees	37
9.4.2	Trees with Unbounded Branching	38
10	Hash Tables	38
10.1	Direct-Address Tables	39
10.2	Hash Tables with Chaining	39
10.3	Analysis: Simple Uniform Hashing	40
10.4	Hash Functions	40
10.4.1	The Division Method	40
10.4.2	The Multiplication Method	41
10.4.3	Universal Hashing	41
10.5	Open Addressing	41
10.5.1	Probing Strategies	42
10.6	Using <code>std::unordered_map</code>	43
11	Binary Search Trees	44
11.1	BST Property	44
11.2	Inorder Traversal	44
11.3	Querying a BST	45
11.3.1	Search	45
11.3.2	Minimum and Maximum	46
11.3.3	Successor and Predecessor	46
11.4	Insertion	47
11.5	Deletion	48
11.6	Randomly Built BSTs	49
11.7	Using STL Ordered Sets and Maps	49
12	AVL Trees	50
12.1	AVL Property	50
12.2	Rotations	51
12.3	Implementation	51
12.4	Correctness of AVL Balancing	54
12.5	AVL vs Red-Black vs Others	54
13	Counting and Structural Properties of Trees	55
13.1	Binary Tree Fundamentals	55
13.2	Number of Structurally Unique Binary Trees — Catalan Numbers	56
13.3	Number of Unique BSTs	57
13.4	AVL Tree Counting	58
13.5	B-Tree Counting	59
13.6	Red-Black Tree Counting	60
13.7	General Counting Formulas Summary	61

14 Red-Black Trees	61
14.1 Properties	62
14.2 Rotations	62
14.3 Insertion	63
14.4 Deletion	65
15 Augmenting Data Structures	66
15.1 The Methodology	66
15.2 Order-Statistic Trees	66
15.2.1 OS-Select: Find i -th Smallest	67
15.2.2 OS-Rank: Find Rank of Node	68
15.2.3 Maintaining Size During Rotations	68
15.3 Using Policy-Based Trees in C++ (Competitive Programming)	69
15.4 Interval Trees	70
15.4.1 Overlap Condition	70
15.5 Augmentation Theorem	71
15.6 Red-Black Trees vs AVL Trees	72
16 Dynamic Programming	73
16.1 The DP Paradigm	73
16.2 Rod Cutting	73
16.3 Matrix Chain Multiplication	74
16.4 Longest Common Subsequence (LCS)	74
16.5 0/1 Knapsack	75
16.6 Top-Down vs Bottom-Up	76
17 Greedy Algorithms	76
17.1 The Greedy Paradigm	76
17.2 Activity Selection	76
17.3 Huffman Codes	77
17.4 Fractional Knapsack	78
17.5 Greedy vs DP	79
17.6 Matroid Theory (Optional)	79
17.7 Correctness of Huffman's Algorithm	79
18 Amortized Analysis	81
18.1 Three Methods	81
18.2 Example: Dynamic Table (Aggregate Method)	81
18.3 Accounting Method	81
18.4 Potential Method	82
19 B-Trees	83
19.1 Motivation: Why Not Binary Trees?	83
19.2 B-Tree Properties	83
19.3 B-Tree Search	83
19.4 B-Tree Insertion	84
19.5 B-Tree Deletion	85

20 Fibonacci Heaps	85
20.1 Motivation	86
20.2 Structure	86
20.3 Key Operations	86
20.3.1 Insert — $O(1)$	86
20.3.2 Find-Min — $O(1)$	86
20.3.3 Union/Merge — $O(1)$	87
20.3.4 Extract-Min — $O(\lg n)$ amortized	87
20.3.5 Decrease-Key — $O(1)$ amortized	87
20.4 Why “Fibonacci”?	87
21 van Emde Boas Trees	88
21.1 Motivation	88
21.2 Key Idea: Recursive Universe Splitting	88
21.2.1 Index Decomposition	88
21.3 Operations	89
21.3.1 Successor — $O(\lg \lg u)$	89
21.3.2 Insert — $O(\lg \lg u)$	89
22 Data Structures for Disjoint Sets	89
22.1 The Disjoint-Set Problem	90
22.2 Linked-List Representation	90
22.3 Forest Representation (Optimal)	90
22.4 Complexity Analysis	91
22.5 Union by Size (Alternative)	92
22.6 Application: Kruskal’s MST	92
22.7 Common Competitive Programming Patterns	93
23 Elementary Graph Algorithms	94
23.1 Graph Representations	94
23.1.1 Adjacency List vs Adjacency Matrix	94
23.2 Breadth-First Search (BFS)	95
23.3 Depth-First Search (DFS)	96
23.3.1 Edge Classification	97
23.4 Topological Sort	98
23.5 Strongly Connected Components (SCCs)	99
23.6 Correctness of BFS	100
23.7 Correctness of DFS	101
24 Minimum Spanning Trees	102
24.1 The Cut Property	102
24.2 Kruskal’s Algorithm	102
24.3 Prim’s Algorithm	103
25 Single-Source Shortest Paths	105
25.1 Properties and Relaxation	105
25.2 Bellman-Ford Algorithm	105
25.3 Shortest Paths in DAGs	106
25.4 Dijkstra’s Algorithm	106

26 All-Pairs Shortest Paths	108
26.1 Floyd-Warshall Algorithm	108
26.2 Johnson's Algorithm	109
27 Maximum Flow	111
27.1 Flow Networks	111
27.2 Max-Flow Min-Cut Theorem	111
27.3 Ford-Fulkerson Method	111
27.4 Edmonds-Karp Algorithm	111
27.5 Dinic's Algorithm	113
28 Multithreaded Algorithms	114
28.1 Key Concepts	115
28.2 Parallel Merge Sort	115
28.3 Parallel Matrix Multiplication	115
29 Matrix Operations	116
29.1 Solving Systems of Linear Equations	116
29.2 Matrix Inversion	117
29.3 Least Squares Approximation	117
30 Linear Programming	117
30.1 Overview	117
30.2 The Simplex Algorithm	118
30.3 Duality	118
30.4 Applications	118
31 Polynomials and the FFT	119
31.1 Polynomial Representations	119
31.2 The Discrete Fourier Transform	119
31.3 Fast Fourier Transform (FFT)	119
32 Number-Theoretic Algorithms	122
32.1 GCD and Extended Euclidean	122
32.2 Modular Arithmetic	122
32.3 Modular Inverse	123
32.4 Chinese Remainder Theorem	123
32.5 Primality Testing	123
32.6 Sieve of Eratosthenes	124
33 String Matching	126
33.1 Naive Algorithm	126
33.2 Rabin-Karp (Rolling Hash)	127
33.3 KMP (Knuth-Morris-Pratt)	127
33.4 Z-Algorithm	128

34 Computational Geometry	130
34.1 Fundamental Primitives	130
34.2 Cross Product and Orientation	130
34.3 Convex Hull	131
34.4 Closest Pair of Points	132
35 NP-Completeness	133
35.1 Complexity Classes	133
35.2 Polynomial-Time Reductions	134
35.3 The First NP-Complete Problem: SAT	134
35.4 Classic NP-Complete Problems	135
35.5 Reduction Chain	135
35.6 Dealing with NP-Complete Problems	135
36 Approximation Algorithms	136
36.1 Approximation Ratio	136
36.2 Vertex Cover: 2-Approximation	137
36.3 TSP: 2-Approximation (Metric TSP)	137
36.4 Set Cover: $O(\lg n)$ -Approximation	138
36.5 Subset Sum: FPTAS	138
Appendix: LeetCode Practice Problems	140
Sorting	140
Heaps / Priority Queues	141
Stacks & Queues	143
Hash Tables	144
Binary Search Trees	145
Dynamic Programming	146
Greedy Algorithms	148
Union-Find	149
Graph Algorithms	150
Shortest Paths	152
String Matching	154
Backtracking	155
Appendix: Suggested Practice Problems	158
Arrays and Sorting	158
Divide and Conquer	158
Heaps and Priority Queues	158
Stacks and Queues	159
Hash Tables	159
BST and Balanced Trees	160
Dynamic Programming	160
Greedy Algorithms	160
Union-Find / Disjoint Sets	161
Graph Algorithms — BFS/DFS	161
Minimum Spanning Trees	162
Shortest Paths	162
Network Flow	162

Number Theory	163
String Matching	163
Backtracking / NP-Completeness	163

1 Getting Started

1.1 Insertion Sort

Insertion Sort

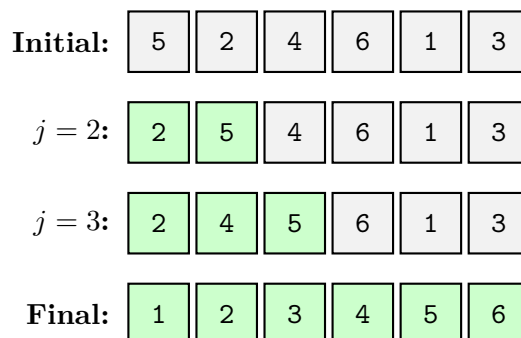
Sorts an array by repeatedly taking the next unsorted element and *inserting* it into its correct position among the already-sorted elements (like sorting playing cards in your hand).

Algorithm 1 Insertion-Sort(A, n)

```

1: for  $j = 2$  to  $n$  do
2:    $\text{key} = A[j]$ 
3:    $i = j - 1$ 
4:   while  $i > 0$  and  $A[i] > \text{key}$  do
5:      $A[i + 1] = A[i]$ 
6:      $i = i - 1$ 
7:   end while
8:    $A[i + 1] = \text{key}$ 
9: end for

```



Loop Invariant

At the start of each iteration of the **for** loop, the subarray $A[1..j-1]$ consists of the elements originally in $A[1..j-1]$, but in sorted order.

We prove correctness using three steps:

1. **Initialization:** Before the first iteration ($j = 2$), $A[1..1]$ is trivially sorted.
2. **Maintenance:** Each iteration inserts $A[j]$ into its correct position in $A[1..j-1]$, so $A[1..j]$ is sorted.
3. **Termination:** When $j = n + 1$, the invariant implies $A[1..n]$ is sorted.

1.1.1 Complexity Analysis

Case	Time	When
Best case	$\Theta(n)$	Already sorted
Worst case	$\Theta(n^2)$	Reverse sorted
Average case	$\Theta(n^2)$	Random order

Space: $\Theta(1)$ auxiliary (in-place). **Stable:** Yes.

1.2 Analyzing Algorithms

RAM Model

We analyze algorithms using the **Random Access Machine** model:

- Instructions execute sequentially (no parallelism).
- Each basic operation (add, compare, assign, memory access) takes constant time.
- Memory is unlimited and flat (any address in $O(1)$).

We focus on **worst-case** running time $T(n)$ because:

- It provides an upper bound guarantee.
- The worst case often occurs in practice.
- Average case is often as bad as worst case (e.g., insertion sort).

1.3 Merge Sort — Divide and Conquer

Divide-and-Conquer Paradigm

1. **Divide** the problem into smaller subproblems.
2. **Conquer** by solving subproblems recursively (base case: trivially small).
3. **Combine** subproblem solutions into the overall solution.

Merge Sort applies D&C:

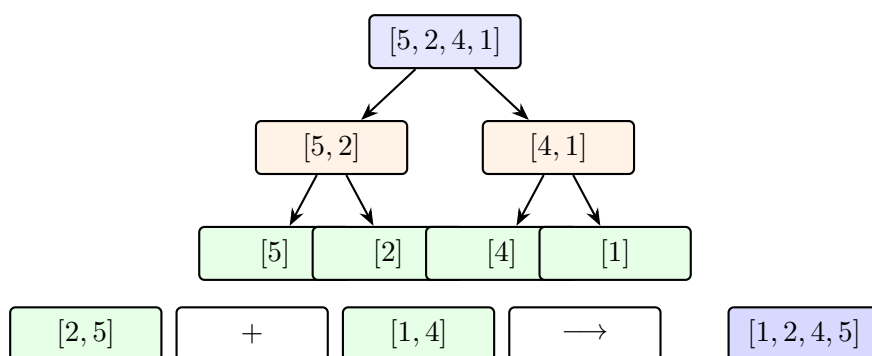
- **Divide:** Split array into two halves.
- **Conquer:** Recursively sort each half.
- **Combine:** Merge two sorted halves in $\Theta(n)$.

Algorithm 2 Merge-Sort(A, p, r)

```

1: if  $p < r$  then
2:    $q = \lfloor (p + r) / 2 \rfloor$ 
3:   Merge-Sort( $A, p, q$ )
4:   Merge-Sort( $A, q + 1, r$ )
5:   Merge( $A, p, q, r$ )
6: end if

```



1.3.1 Recurrence and Complexity

The recurrence for merge sort:

$$T(n) = \begin{cases} \Theta(1) & \text{if } n = 1 \\ 2T(n/2) + \Theta(n) & \text{if } n > 1 \end{cases}$$

Solution: $T(n) = \Theta(n \lg n)$.

The recursion tree has $\lg n$ levels, each doing $\Theta(n)$ total work $\Rightarrow \Theta(n \lg n)$.

Merge Sort vs Insertion Sort

	Insertion Sort	Merge Sort
Worst case	$\Theta(n^2)$	$\Theta(n \lg n)$
Best case	$\Theta(n)$	$\Theta(n \lg n)$
Space	$\Theta(1)$	$\Theta(n)$
Stable	Yes	Yes
In-place	Yes	No

Insertion sort wins for small n (low constant factors). Merge sort wins for large n .

Theorem 1.1 (Correctness of Merge Sort). Merge-Sort(A, p, r) correctly sorts $A[p..r]$ in non-decreasing order.

Proof by Strong Induction

Base ($n = 1$): A single element is trivially sorted. ✓

Inductive step: Assume Merge-Sort correctly sorts all subarrays of size $< n$.

- Merge-Sort(A, p, q) correctly sorts $A[p..q]$ (size $< n$). By IH. ✓
- Merge-Sort($A, q+1, r$) correctly sorts $A[q+1..r]$ (size $< n$). By IH. ✓
- It remains to show Merge correctly combines them.

Merge correctness (loop invariant): At the start of each iteration, $A[p..k-1]$ contains the $k - p$ smallest elements of the two sorted halves, in sorted order.

Init: $k = p$, subarray $A[p..p-1]$ is empty. ✓

Maintenance: The smaller of the two front elements is placed at $A[k]$, extending the sorted prefix. ✓

Termination: $k = r + 1$, so $A[p..r]$ contains all elements, sorted. □

2 Growth of Functions — Asymptotic Analysis

Asymptotic analysis describes how an algorithm's resource usage (time or space) scales as the input size n grows toward infinity. It lets us compare algorithms by ignoring constant factors and lower-order terms.

2.1 Why Asymptotic Analysis?

Motivation

- Exact running time depends on hardware, compiler, language — not useful for comparison.
- We care about *scalability*: how does performance change as n doubles?
- Constant factors matter in practice but are hard to predict theoretically.
- Asymptotic analysis gives a **hardware-independent** way to rank algorithms.

Example: Algorithm A takes $100n$ steps. Algorithm B takes $2n^2$ steps. For $n < 50$, B is faster. But for $n > 50$, A wins — and the gap grows forever. Asymptotically, $A = O(n)$ beats $B = O(n^2)$.

2.2 The Five Asymptotic Notations

Big-O: Upper Bound

$f(n) = O(g(n))$ if there exist constants $c > 0$ and $n_0 > 0$ such that:

$$0 \leq f(n) \leq c \cdot g(n) \quad \forall n \geq n_0$$

Meaning: f grows *no faster* than g (up to a constant factor).

Example: Show $3n^2 + 5n + 2 = O(n^2)$.

Choose $c = 10$, $n_0 = 1$: for $n \geq 1$, $3n^2 + 5n + 2 \leq 3n^2 + 5n^2 + 2n^2 = 10n^2$. $\square$

Big-Ω: Lower Bound

$f(n) = \Omega(g(n))$ if there exist constants $c > 0$ and $n_0 > 0$ such that:

$$0 \leq c \cdot g(n) \leq f(n) \quad \forall n \geq n_0$$

Meaning: f grows *at least as fast* as g .

Example: Show $3n^2 + 5n + 2 = \Omega(n^2)$.

Choose $c = 3$, $n_0 = 1$: for $n \geq 1$, $3n^2 + 5n + 2 \geq 3n^2$. $\square$

Big-Θ: Tight Bound

$f(n) = \Theta(g(n))$ if and only if $f(n) = O(g(n))$ **and** $f(n) = \Omega(g(n))$.

Equivalently: $\exists c_1, c_2 > 0$, n_0 such that:

$$c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n) \quad \forall n \geq n_0$$

Meaning: f and g grow at *the same rate*.

Example: $3n^2 + 5n + 2 = \Theta(n^2)$ (from the two proofs above with $c_1 = 3$, $c_2 = 10$).

Little- o and Little- ω : Strict Bounds

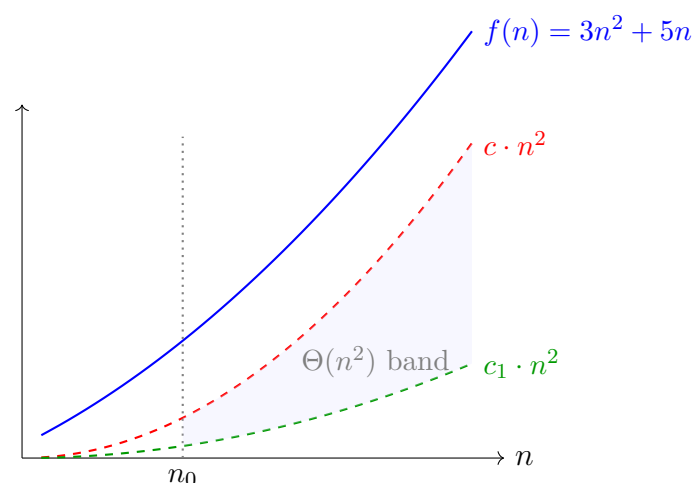
$$f(n) = o(g(n)) \iff \lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = 0 \quad (f \text{ grows strictly slower than } g)$$

$$f(n) = \omega(g(n)) \iff \lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = \infty \quad (f \text{ grows strictly faster than } g)$$

Example: $n = o(n^2)$ because $\lim_{n \rightarrow \infty} n/n^2 = \lim 1/n = 0$.

Example: $n^2 = \omega(n)$ because $\lim_{n \rightarrow \infty} n^2/n = \lim n = \infty$.

Notation	Analogy	Formal Condition	Limit Test
$f = O(g)$	$f \leq g$	$f(n) \leq cg(n)$ eventually	$\limsup \frac{f}{g} < \infty$
$f = \Omega(g)$	$f \geq g$	$f(n) \geq cg(n)$ eventually	$\liminf \frac{f}{g} > 0$
$f = \Theta(g)$	$f = g$	$c_1g \leq f \leq c_2g$	$0 < \lim \frac{f}{g} < \infty$
$f = o(g)$	$f < g$	$\forall c > 0: f < cg$	$\lim \frac{f}{g} = 0$
$f = \omega(g)$	$f > g$	$\forall c > 0: f > cg$	$\lim \frac{f}{g} = \infty$



2.3 Worked Examples

2.3.1 Example 1: Polynomial

Claim: $5n^3 - 2n^2 + 3n = \Theta(n^3)$.

Upper bound (O): For $n \geq 1$: $5n^3 - 2n^2 + 3n \leq 5n^3 + 3n^3 = 8n^3$. So $c_2 = 8$.

Lower bound (Ω): For $n \geq 1$: $5n^3 - 2n^2 + 3n \geq 5n^3 - 2n^3 = 3n^3$. So $c_1 = 3$.

Therefore $3n^3 \leq 5n^3 - 2n^2 + 3n \leq 8n^3$ for $n \geq 1$, proving $\Theta(n^3)$. $\square$

Shortcut for Polynomials

For any polynomial $p(n) = a_k n^k + a_{k-1} n^{k-1} + \dots + a_0$ with $a_k > 0$:

$$p(n) = \Theta(n^k)$$

The highest-degree term dominates. All lower-order terms and coefficients are irrelevant.

2.3.2 Example 2: Logarithms

Claim: $\lg(n!) = \Theta(n \lg n)$.

Upper: $n! \leq n^n$, so $\lg(n!) \leq n \lg n$. Thus $\lg(n!) = O(n \lg n)$.

Lower: $n! \geq (n/2)^{n/2}$ (the top half of the terms are each $\geq n/2$), so $\lg(n!) \geq (n/2) \lg(n/2) = \Theta(n \lg n)$.

Therefore $\lg(n!) = \Theta(n \lg n)$. $\square$

2.3.3 Example 3: Exponentials vs Polynomials

Claim: $n^{100} = o(2^n)$.

Using the limit test: $\lim_{n \rightarrow \infty} n^{100}/2^n = 0$ (exponentials always dominate polynomials). Apply L'Hôpital's rule 100 times, or note that $n^{100}/2^n = (n/2^{n/100})^{100}$, and $n/2^{n/100} \rightarrow 0$.

Key takeaway: Any exponential c^n (with $c > 1$) grows faster than any polynomial n^k .

2.3.4 Example 4: Logs vs Powers

Claim: $\lg n = o(n^\epsilon)$ for any constant $\epsilon > 0$.

$\lim_{n \rightarrow \infty} \frac{\lg n}{n^\epsilon} = 0$ (substitute $n = 2^m$: $\frac{m}{2^{m\epsilon}} \rightarrow 0$).

Key takeaway: Logarithms grow slower than any polynomial, even $n^{0.001}$.

2.4 Common Growth Rates

Name	Notation	$n = 10^6$	Example Algorithm
Constant	$O(1)$	1	Array access, hash lookup
Logarithmic	$O(\lg n)$	~ 20	Binary search
Square root	$O(\sqrt{n})$	1000	Trial division
Linear	$O(n)$	10^6	Linear scan, counting sort
Log-linear	$O(n \lg n)$	2×10^7	Merge sort, FFT
Quadratic	$O(n^2)$	10^{12}	Insertion sort, naive LCS
Cubic	$O(n^3)$	10^{18}	Floyd-Warshall, matrix mult
Exponential	$O(2^n)$	$\sim 10^{301029}$	Brute-force subsets
Factorial	$O(n!)$	astronomical	Brute-force permutations

$$1 \prec \lg n \prec \sqrt{n} \prec n \prec n \lg n \prec n^2 \prec n^3 \prec 2^n \prec n! \prec n^n$$

Competitive Programming Time Limits

Most judges allow $\sim 10^8$ operations per second. Rule of thumb:

n	Max affordable complexity
≤ 20	$O(2^n \cdot n)$ or $O(n!)$
≤ 25	$O(2^n)$
≤ 100	$O(n^4)$
≤ 500	$O(n^3)$
≤ 5000	$O(n^2)$
$\leq 10^6$	$O(n \lg n)$
$\leq 10^8$	$O(n)$

2.5 Properties of Asymptotic Notation

Algebraic Properties

- **Transitivity:** $f = O(g)$ and $g = O(h) \implies f = O(h)$. Same for all five notations.
- **Reflexivity:** $f = O(f)$, $f = \Omega(f)$, $f = \Theta(f)$.
- **Symmetry:** $f = \Theta(g) \iff g = \Theta(f)$.
- **Transpose symmetry:** $f = O(g) \iff g = \Omega(f)$. Also: $f = o(g) \iff g = \omega(f)$.

Arithmetic Rules

- **Sum rule:** $O(f + g) = O(\max(f, g))$. Example: $O(n^2 + n) = O(n^2)$.
- **Product rule:** $O(f) \cdot O(g) = O(f \cdot g)$. Example: $O(n) \cdot O(\lg n) = O(n \lg n)$.
- **Constant factor:** $O(c \cdot f) = O(f)$ for constant $c > 0$.
- **Log base doesn't matter:** $O(\log_a n) = O(\log_b n)$ because $\log_a n = \frac{\log_b n}{\log_b a}$ (constant factor).

2.6 Analyzing Code: From Loops to Big-O

2.6.1 Single Loop

```

1 for (int i = 0; i < n; i++) // n iterations
2     sum += a[i];           // O(1) per iteration
3 // Total: O(n)

```

Listing 1: $O(n)$

2.6.2 Nested Loops

```

1 for (int i = 0; i < n; i++) // n iterations
2     for (int j = 0; j < n; j++) // n iterations each
3         if (a[i] > a[j]) cnt++; // O(1)
4 // Total: O(n * n) = O(n^2)

```

Listing 2: $O(n^2)$

2.6.3 Dependent Inner Loop

```

1 for (int i = 0; i < n; i++)
2     for (int j = 0; j < i; j++) // j goes 0..i-1
3         process(i, j);
4 // Total: 0 + 1 + 2 + ... + (n-1) = n(n-1)/2 = O(n^2)

```

Listing 3: $O(n^2)$ — careful analysis

2.6.4 Logarithmic Loop

```

1 int i = n;
2 while (i > 0) {
3     process(i);
4     i /= 2;          // halving each time
5 }
6 // Iterations: lg(n), Total: O(lg n)

```

Listing 4: $O(\lg n)$

2.6.5 Tricky: Harmonic Series

```

1 for (int i = 1; i <= n; i++)
2     for (int j = i; j <= n; j += i) // n/i iterations
3         process(i, j);
4 // Total: n/1 + n/2 + n/3 + ... + n/n = n * H_n = O(n lg n)

```

Listing 5: $O(n \lg n)$ — harmonic sum

This pattern appears in the Sieve of Eratosthenes and divisor enumeration.

2.6.6 Amortized-Like Pattern

```

1 int j = 0;
2 for (int i = 0; i < n; i++) {
3     while (j < n && a[j] < a[i])
4         j++;          // j only increases
5 }
6 // j goes from 0 to n total across ALL iterations of i
7 // Total: O(n), not O(n^2)!

```

Listing 6: $O(n)$ total despite nested structure

This two-pointer pattern is $O(n)$ because j increments at most n times total.

2.7 Standard Notations and Common Functions

Important Identities and Bounds

- $\lg n = \log_2 n$ (CLRS convention), $\ln n = \log_e n$
- $\lg^k n = (\lg n)^k$ (polylogarithmic), $\lg \lg n = \lg(\lg n)$
- **Stirling:** $n! = \sqrt{2\pi n} \left(\frac{n}{e}\right)^n \left(1 + \Theta\left(\frac{1}{n}\right)\right)$
- $\lg(n!) = \Theta(n \lg n)$
- **Harmonic:** $H_n = \sum_{k=1}^n \frac{1}{k} = \ln n + \gamma + O(1/n)$, where $\gamma \approx 0.5772$ (Euler–Mascheroni constant)
- **Geometric:** $\sum_{k=0}^n x^k = \frac{x^{n+1}-1}{x-1}$. If $|x| < 1$: $\sum_{k=0}^{\infty} x^k = \frac{1}{1-x}$
- **Arithmetic:** $\sum_{k=1}^n k = \frac{n(n+1)}{2} = \Theta(n^2)$
- $\sum_{k=1}^n k^2 = \frac{n(n+1)(2n+1)}{6} = \Theta(n^3)$
- $\sum_{k=0}^{\infty} kx^k = \frac{x}{(1-x)^2}$ for $|x| < 1$
- **Iterated log:** $\lg^* n$ = number of times you take $\lg$ until ≤ 1 . $\lg^*(2^{65536}) = 5$.
Grows incredibly slowly.

2.8 Common Mistakes

Pitfalls to Avoid

1. **O is not Θ :** Saying “insertion sort is $O(n^2)$ ” is correct but weak. It is also $O(n^3)$ and $O(2^n)$. Say $\Theta(n^2)$ for tight bounds.
2. **Best case $\neq \Omega$:** $\Omega(g)$ is a *lower bound on the function*, not “best-case time.” Insertion sort is $\Omega(n)$ in all cases and $\Theta(n)$ in the best case.
3. **Drop all constants:** $O(2n) = O(n)$, $O(n/2) = O(n)$. Never write $O(2n)$ or $O(n/2)$.
4. **Drop lower-order terms:** $O(n^2 + n) = O(n^2)$. Never write $O(n^2 + n \lg n)$.
5. **Base of logarithm:** $O(\log_2 n) = O(\log_{10} n) = O(\ln n)$. They differ only by constant factors. Just write $O(\lg n)$.
6. $2^{n+1} = O(2^n)$ (just a constant factor of 2), but $2^{2n} \neq O(2^n)$ (that’s 4^n , exponentially larger).

3 Divide-and-Conquer

3.1 Recurrences

A D&C algorithm that divides a problem of size n into a subproblems, each of size n/b , with combine cost $f(n)$, has recurrence:

$$T(n) = aT(n/b) + f(n)$$

3.2 Solving Recurrences

3.2.1 Method 1: Substitution Method

1. **Guess** the form of the solution.

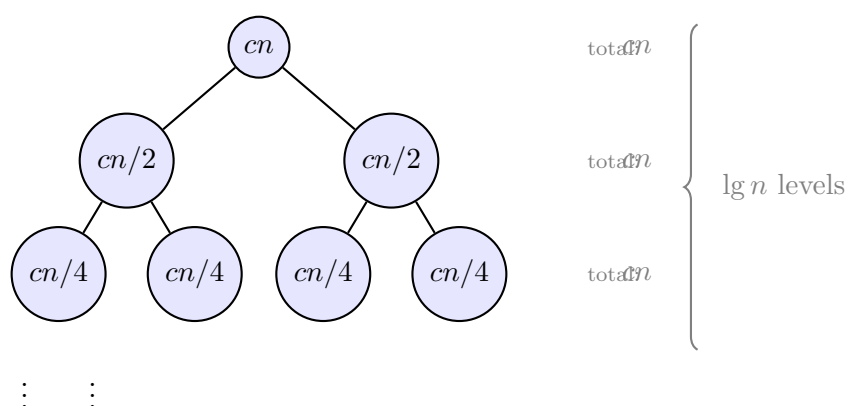
2. **Prove** by induction that the guess is correct.

Common Pitfall

When proving $T(n) = O(n \lg n)$ by induction, you must show $T(n) \leq cn \lg n$ for some specific c , not $T(n) \leq cn \lg n + n$. The latter proves $O(n \lg n + n)$, not $O(n \lg n)$. Subtract lower-order terms from your guess to make the proof work.

3.2.2 Method 2: Recursion Tree

Draw the tree of recursive calls. At each level, sum the non-recursive work. Total cost = sum across all levels.



Each level sums to cn , and there are $\lg n + 1$ levels, giving $\Theta(n \lg n)$.

3.3 The Master Theorem

Theorem 3.1 (Master Theorem). Let $T(n) = aT(n/b) + f(n)$ where $a \geq 1$, $b > 1$. Compare $f(n)$ with $n^{\log_b a}$:

Case	Condition	Result
1	$f(n) = O(n^{\log_b a - \epsilon})$ for some $\epsilon > 0$	$T(n) = \Theta(n^{\log_b a})$
2	$f(n) = \Theta(n^{\log_b a})$	$T(n) = \Theta(n^{\log_b a} \lg n)$
3	$f(n) = \Omega(n^{\log_b a + \epsilon})$ for some $\epsilon > 0$, and $af(n/b) \leq cf(n)$ for $c < 1$	$T(n) = \Theta(f(n))$

Intuition for the Master Theorem

$n^{\log_b a}$ is the cost of the *leaves* of the recursion tree.

- **Case 1:** Leaves dominate (tree-heavy). Work grows going down.
- **Case 2:** Every level does roughly equal work. Multiply by depth.
- **Case 3:** Root dominates. Work shrinks going down.

3.3.1 Examples

Recurrence	$n^{\log_b a}$	Case	Solution
$T(n) = 2T(n/2) + n$	n^1	2	$\Theta(n \lg n)$
$T(n) = 4T(n/2) + n$	n^2	1	$\Theta(n^2)$
$T(n) = T(n/2) + 1$	$n^0 = 1$	2	$\Theta(\lg n)$
$T(n) = 2T(n/2) + n^2$	n^1	3	$\Theta(n^2)$
$T(n) = 9T(n/3) + n$	n^2	1	$\Theta(n^2)$

When Master Theorem Fails

It does **not** apply when:

- $f(n)$ falls in a “gap” between cases (e.g., $f(n) = n^{\log_b a} / \lg n$).
- Subproblems are not equal size (e.g., $T(n) = T(n/3) + T(2n/3) + n$).
- The regularity condition in Case 3 is not met.

Use the recursion tree or Akra–Bazzi method instead.

3.4 Strassen’s Matrix Multiplication

Naive: $T(n) = 8T(n/2) + \Theta(n^2) \Rightarrow \Theta(n^3)$.

Strassen: Reduce to 7 recursive multiplications using clever addition/subtraction:

$$T(n) = 7T(n/2) + \Theta(n^2) \Rightarrow \Theta(n^{\log_2 7}) \approx \Theta(n^{2.807})$$

Key Idea

Strassen’s insight: trade one multiplication for several additions. Since addition is $\Theta(n^2)$ and multiplication recurses, reducing from 8 to 7 recursive calls drops the exponent from $\log_2 8 = 3$ to $\log_2 7 \approx 2.807$.

4 Probabilistic Analysis and Randomized Algorithms

4.1 The Hiring Problem

Hiring Problem

Interview n candidates sequentially. Hire a candidate if they are better than the current best. Hiring costs $c_h \gg c_i$ (interview cost). What is the expected hiring cost?

Worst case: Candidates arrive in increasing order $\Rightarrow$ hire all n , cost = $O(nc_h)$.

Average case (random order): Expected number of hires = $\ln n + O(1)$ (the n -th Harmonic number), so expected cost = $O(c_h \ln n)$.

Why $\ln n$?

Candidate i is hired iff they are the best of the first i candidates. Probability = $1/i$.
Expected hires:

$$E[\text{hires}] = \sum_{i=1}^n \frac{1}{i} = H_n = \ln n + O(1)$$

This uses **linearity of expectation**: $E[X_1 + \dots + X_n] = E[X_1] + \dots + E[X_n]$, even when variables are dependent.

4.2 Indicator Random Variables**Indicator Random Variable**

For event A :

$$I\{A\} = \begin{cases} 1 & \text{if } A \text{ occurs} \\ 0 & \text{otherwise} \end{cases}$$

Key property: $E[I\{A\}] = \Pr\{A\}$.

This technique simplifies expected value calculations enormously. Instead of computing a complex distribution, decompose into simple binary indicators and use linearity of expectation.

4.3 Randomized Algorithms**Randomized vs Deterministic**

- **Deterministic:** Behavior determined entirely by input. Average-case analysis assumes input distribution.
- **Randomized:** Algorithm makes random choices. Expected running time is over the algorithm's randomness, not the input. *No input is "worst case."*

Randomized hiring: Randomly permute the candidate list before interviewing. This ensures the input is in random order regardless of adversarial input.

Algorithm 3 Randomized-Hire(A, n)

```

1: Randomly permute  $A$ 
2:  $\text{best} = 0$ 
3: for  $i = 1$  to  $n$  do
4:   if  $A[i]$  is better than  $\text{best}$  then
5:      $\text{best} = A[i]$ ; hire  $A[i]$ 
6:   end if
7: end for

```

4.4 Random Permutation (Fisher–Yates Shuffle)

Algorithm 4 Randomize-In-Place(A, n)

```

1: for  $i = 1$  to  $n$  do
2:   swap  $A[i]$  with  $A[\text{Random}(i, n)]$ 
3: end for
  
```

Theorem 4.1. Randomize-In-Place produces a **uniform random permutation**: each of the $n!$ permutations is equally likely.

The Birthday Paradox

In a room of k people, the probability that two share a birthday ($n = 365$) exceeds $1/2$ when $k \geq 23$. More generally, with n possible values, collisions are expected after $\Theta(\sqrt{n})$ samples. This is fundamental to hashing analysis and cryptography.

5 Heapsort

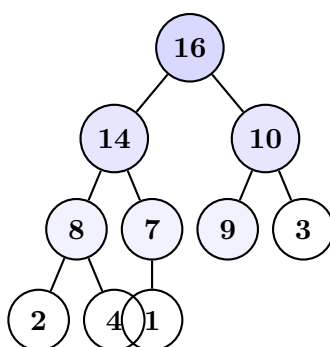
5.1 Heaps

Binary Heap

A **(max-)heap** is a nearly complete binary tree stored in an array where every node satisfies the **heap property**: $A[\text{Parent}(i)] \geq A[i]$.

For 1-indexed array:

- $\text{Parent}(i) = \lfloor i/2 \rfloor$
- $\text{Left}(i) = 2i$
- $\text{Right}(i) = 2i + 1$



Key property: Height of a heap with n elements = $\lfloor \lg n \rfloor$.

5.2 Core Heap Operations

5.2.1 Max-Heapify — $O(\lg n)$

Fixes a single violation: assumes $\text{Left}(i)$ and $\text{Right}(i)$ are valid max-heaps, but $A[i]$ may be smaller than its children. “Float down” the offending node.

Algorithm 5 Max-Heapify(A, i)

```

1:  $l = \text{Left}(i)$ ,  $r = \text{Right}(i)$ 
2:  $\text{largest} = i$ 
3: if  $l \leq \text{heap-size}$  and  $A[l] > A[\text{largest}]$  then  $\text{largest} = l$ 
4: end if
5: if  $r \leq \text{heap-size}$  and  $A[r] > A[\text{largest}]$  then  $\text{largest} = r$ 
6: end if
7: if  $\text{largest} \neq i$  then
8:   swap  $A[i] \leftrightarrow A[\text{largest}]$ 
9:   Max-Heapify( $A, \text{largest}$ )
10: end if
```

5.2.2 Build-Max-Heap — $O(n)$

Convert an unsorted array into a max-heap by calling Max-Heapify bottom-up:

Algorithm 6 Build-Max-Heap(A)

```

1:  $\text{heap-size} = n$ 
2: for  $i = \lfloor n/2 \rfloor$  downto 1 do
3:   Max-Heapify( $A, i$ )
4: end for
```

Why Build-Max-Heap is $O(n)$ and not $O(n \lg n)$

Naive analysis: $n/2$ calls $\times O(\lg n)$ each = $O(n \lg n)$.

Tighter analysis: nodes at height h number $\leq \lceil n/2^{h+1} \rceil$, and Max-Heapify at height h costs $O(h)$. Total:

$$\sum_{h=0}^{\lfloor \lg n \rfloor} \left\lceil \frac{n}{2^{h+1}} \right\rceil \cdot O(h) = O\left(n \sum_{h=0}^{\infty} \frac{h}{2^h}\right) = O(n)$$

since $\sum_{h=0}^{\infty} h/2^h = 2$.

5.3 Heapsort Algorithm

Algorithm 7 Heapsort(A)

```

1: Build-Max-Heap( $A$ )
2: for  $i = n$  downto 2 do
3:   swap  $A[1] \leftrightarrow A[i]$  ▷ move max to end
4:   heap-size = heap-size - 1
5:   Max-Heapify( $A, 1$ )
6: end for
  
```

	Best	Average	Worst	Space	Stable?
Heapsort	$\Theta(n \lg n)$	$\Theta(n \lg n)$	$\Theta(n \lg n)$	$O(1)$	No

5.4 Priority Queue

A heap naturally implements a **max-priority queue** with operations:

Operation	Time
Insert(S, x)	$O(\lg n)$
Maximum(S)	$O(1)$
Extract-Max(S)	$O(\lg n)$
Increase-Key(S, x, k)	$O(\lg n)$

5.5 Correctness Proofs for Heap Operations

Theorem 5.1 (Correctness of Max-Heapify). If the subtrees rooted at $\text{left}(i)$ and $\text{right}(i)$ are max-heaps, then after calling $\text{MAX-HEAPIFY}(A, i)$, the subtree rooted at i is a max-heap.

Proof by Induction on Height

Base case ($h = 0$): Node i is a leaf. It trivially satisfies the max-heap property (no children). Max-Heapify returns immediately. ✓

Inductive step: Assume Max-Heapify is correct for all nodes of height $< h$. Consider a node i of height h .

Max-Heapify compares $A[i]$ with $A[\text{left}(i)]$ and $A[\text{right}(i)]$, finding the largest among the three. Two cases:

Case 1: $A[i]$ is the largest. The heap property holds at i , and both subtrees are max-heaps by assumption. Done.

Case 2: A child (say left) has the largest value. We swap $A[i] \leftrightarrow A[\text{left}(i)]$. Now:

- The right subtree is unaffected and remains a max-heap.
- The left subtree may have its root violated, but its children are still max-heaps. The left child has height $h - 1 < h$.
- We recursively call Max-Heapify on the left child, which by the inductive hypothesis restores it to a max-heap.

After the recursive call, both subtrees are max-heaps and $A[i] \geq A[\text{left}(i)]$, $A[i] \geq A[\text{right}(i)]$, so the subtree rooted at i is a max-heap. □

Theorem 5.2 (Correctness of Build-Max-Heap). $\text{BUILD-MAX-HEAP}(A)$ produces a max-heap from an arbitrary array.

Proof: Loop Invariant

Loop invariant: At the start of each iteration with index i , every node $i+1, i+2, \dots, n$ is the root of a max-heap.

Initialization ($i = \lfloor n/2 \rfloor$): Nodes $\lfloor n/2 \rfloor + 1$ through n are all leaves, hence trivially max-heaps. ✓

Maintenance: When we call $\text{Max-Heapify}(A, i)$, nodes $i+1, \dots, n$ are roots of max-heaps. In particular, the children of i (at indices $2i$ and $2i+1$, both $> i$) are roots of max-heaps. By the correctness of Max-Heapify , after the call, i is also the root of a max-heap. Decrementing i preserves the invariant. ✓

Termination ($i = 0$): Every node $1, 2, \dots, n$ is the root of a max-heap. In particular, node 1 (the root) is a max-heap, so the entire array is a valid max-heap. □

Theorem 5.3 (Correctness of Heapsort). $\text{HEAPSORT}(A)$ sorts array A in non-decreasing order.

Proof: Loop Invariant

Loop invariant: At the start of each iteration i :

1. $A[1..i]$ is a max-heap containing the i smallest elements.
2. $A[i+1..n]$ contains the $n - i$ largest elements in sorted order.

Initialization ($i = n$): After Build-Max-Heap , $A[1..n]$ is a max-heap. The sorted portion $A[n+1..n]$ is empty. ✓

Maintenance: $A[1]$ is the maximum of $A[1..i]$. We swap it to position i , where it joins the sorted suffix (it's $\leq$ everything in $A[i+1..n]$ since those are the largest elements). We reduce heap size to $i-1$ and call Max-Heapify to restore the heap on $A[1..i-1]$. ✓

Termination ($i = 1$): $A[2..n]$ is sorted and $A[1]$ is the minimum. The entire array is sorted. □

Theorem 5.4 ($O(n)$ Build-Max-Heap). Build-Max-Heap runs in $O(n)$, not $O(n \lg n)$.

Proof

A heap of n elements has at most $\lceil n/2^{h+1} \rceil$ nodes at height h . Max-Heapify at height h costs $O(h)$. Total work:

$$\sum_{h=0}^{\lfloor \lg n \rfloor} \left\lceil \frac{n}{2^{h+1}} \right\rceil \cdot O(h) = O\left(n \sum_{h=0}^{\infty} \frac{h}{2^h}\right) = O(n \cdot 2) = O(n)$$

using the identity $\sum_{h=0}^{\infty} hx^h = x/(1-x)^2$ with $x = 1/2$. □

6 Quicksort

6.1 Algorithm

Quicksort

A divide-and-conquer sorting algorithm:

1. **Divide:** Partition $A[p..r]$ around a *pivot* element $A[r]$ into two subarrays: elements $\leq$ pivot and elements $>$ pivot.
2. **Conquer:** Recursively sort the two subarrays.
3. **Combine:** Nothing — the array is sorted in place.

Algorithm 8 Quicksort(A, p, r)

```

1: if  $p < r$  then
2:    $q = \text{Partition}(A, p, r)$ 
3:   Quicksort( $A, p, q - 1$ )
4:   Quicksort( $A, q + 1, r$ )
5: end if

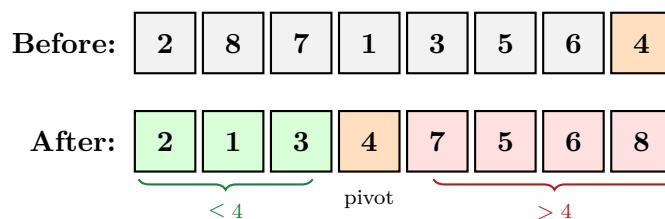
```

Algorithm 9 Partition(A, p, r) — Lomuto scheme

```

1:  $x = A[r]$  ▷ pivot
2:  $i = p - 1$ 
3: for  $j = p$  to  $r - 1$  do
4:   if  $A[j] \leq x$  then
5:      $i = i + 1$ 
6:     swap  $A[i] \leftrightarrow A[j]$ 
7:   end if
8: end for
9: swap  $A[i + 1] \leftrightarrow A[r]$ 
10: return  $i + 1$ 

```



6.2 Complexity Analysis

Case	Recurrence	Time
Worst case	$T(n) = T(n - 1) + T(0) + \Theta(n)$	$\Theta(n^2)$
Best case	$T(n) = 2T(n/2) + \Theta(n)$	$\Theta(n \lg n)$
Average case	—	$\Theta(n \lg n)$

- **Worst case** occurs when pivot is always the min or max (e.g., already sorted input with last-element pivot).

- **Best case** occurs when pivot always splits the array in half.
- Even a 9:1 split gives $\Theta(n \lg n)$ — only perfectly unbalanced splits cause n^2 .

Why Quicksort is Fast in Practice

- Average case is $\Theta(n \lg n)$ with small constant factors.
- Excellent cache locality (sequential memory access).
- In-place: only $O(\lg n)$ stack space.
- “Bad” and “good” splits alternate and cancel out on random data.

6.3 Randomized Quicksort

Choose pivot *uniformly at random* instead of always using $A[r]$:

Algorithm 10 Randomized-Partition(A, p, r)

- 1: $i = \text{Random}(p, r)$
 - 2: swap $A[i] \leftrightarrow A[r]$
 - 3: **return** Partition(A, p, r)
-

Theorem 6.1. The expected running time of Randomized Quicksort is $O(n \lg n)$ on *any* input. Expected comparisons = $2n \ln n + O(n)$.

Proof Sketch

For elements $z_i < z_j$, let $X_{ij} = I\{z_i \text{ is compared with } z_j\}$. They are compared iff one of them is chosen as pivot before any element between them. $\Pr[X_{ij} = 1] = \frac{2}{j-i+1}$.
Total comparisons:

$$E\left[\sum_{i < j} X_{ij}\right] = \sum_{i=1}^{n-1} \sum_{j=i+1}^n \frac{2}{j-i+1} = O(n \lg n)$$

Theorem 6.2 (Correctness of Partition). After Partition(A, p, r), the pivot x is at position $q = i + 1$ and: $A[k] \leq x$ for $p \leq k \leq q - 1$ and $A[k] > x$ for $q + 1 \leq k \leq r$.

Proof: Loop Invariant

Invariant: At the start of each iteration of the for loop (j):

1. $A[k] \leq x$ for all $p \leq k \leq i$.
2. $A[k] > x$ for all $i + 1 \leq k \leq j - 1$.
3. $A[r] = x$ (pivot).

Init ($j = p$): Regions 1 and 2 are empty. ✓

Maintenance: If $A[j] > x$: j increments, extending region 2. ✓

If $A[j] \leq x$: swap $A[i+1]$ with $A[j]$, increment i and j . The swapped element ($\leq x$) joins region 1; region 2 shifts right. ✓

Termination ($j = r$): After the final swap of $A[i+1]$ with $A[r]$, the pivot is at position $i + 1$ with all smaller elements to its left and all larger elements to its right.

□

Theorem 6.3 (Correctness of Quicksort). Quicksort(A, p, r) correctly sorts $A[p..r]$.

Proof by Induction

Base ($n \leq 1$): $p \geq r$, do nothing. Trivially sorted. ✓

Inductive step: Partition places pivot $A[q]$ in its final sorted position. By Partition correctness: all elements left of q are $\leq A[q]$, all right are $> A[q]$. By IH: Quicksort($A, p, q-1$) sorts the left, Quicksort($A, q+1, r$) sorts the right. Combined with the pivot in place, $A[p..r]$ is sorted. □

7 Sorting in Linear Time

7.1 Lower Bound for Comparison Sorts

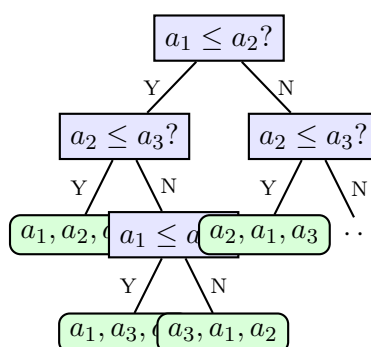
Theorem 7.1 (Comparison Sort Lower Bound). Any comparison-based sorting algorithm requires $\Omega(n \lg n)$ comparisons in the worst case.

Proof via Decision Tree

A comparison sort can be modeled as a binary *decision tree*:

- Each internal node is a comparison $A[i] \leq A[j]$.
- Each leaf is a permutation (output).
- There are $n!$ possible outputs, so at least $n!$ leaves.
- A binary tree with ℓ leaves has height $\geq \lg \ell$.
- Height $\geq \lg(n!) = \Theta(n \lg n)$ (by Stirling).

Therefore: merge sort and heapsort are **asymptotically optimal** among comparison sorts.



To beat $\Omega(n \lg n)$, we must use **non-comparison** information about the input.

7.2 Counting Sort — $\Theta(n + k)$

Counting Sort

Sorts n integers, each in range $[0, k]$, by counting occurrences. No comparisons are made. Requires $k = O(n)$ for linear time.

Algorithm 11 Counting-Sort(A, B, k)

```

1: Let  $C[0..k]$  be a new array, initialized to 0
2: for  $j = 1$  to  $n$  do
3:    $C[A[j]] = C[A[j]] + 1$  ▷ count occurrences
4: end for
5: for  $i = 1$  to  $k$  do
6:    $C[i] = C[i] + C[i - 1]$  ▷ prefix sums
7: end for
8: for  $j = n$  downto 1 do
9:    $B[C[A[j]]] = A[j]$ 
10:   $C[A[j]] = C[A[j]] - 1$ 
11: end for

```

Time: $\Theta(n + k)$. **Space:** $\Theta(n + k)$. **Stable:** Yes (the *downto* loop preserves order).

Not In-Place

Counting sort needs auxiliary arrays B and C . It is *not* in-place. But stability is crucial—it's what makes Radix Sort work correctly.

Theorem 7.2 (Correctness of Counting Sort). Counting-Sort places each element $A[j]$ in its correct final position in B .

Proof

After the prefix-sum computation, $C[v]$ = number of elements $\leq v$. Thus $C[v]$ is exactly the position where the last occurrence of value v should go.

The **downto** loop processes elements from right to left. For element $A[j]$ with value v :

- Place $A[j]$ at $B[C[v]]$ — the correct position.
- Decrement $C[v]$ so the next occurrence of v goes one position earlier.

Stability: Elements with equal values appear in B in the same relative order as in A , because the *downto* loop processes them right-to-left and places them right-to-left (decrementing $C[v]$ each time). $\square$

Theorem 7.3 (Correctness of Radix Sort). Sorting by LSD-first with a stable subroutine correctly sorts d -digit numbers.

Proof by Induction on digit position k

Base ($k = 1$): After sorting by digit 1 (LSD), elements are correctly ordered with respect to the last digit. $\checkmark$

Inductive step: Assume elements are correctly sorted by digits $1, \dots, k-1$. We now sort by digit k using a stable sort.

For two elements a, b : if their k -th digits differ, the sort by digit k puts them in correct order. If their k -th digits are the same, the *stability* of the sort preserves their relative order from the previous pass, which is correct by the inductive hypothesis.

□

7.3 Radix Sort — $\Theta(d(n + k))$

Radix Sort

Sort n numbers with d digits, each digit in range $[0, k-1]$. Sort by *least significant digit first*, using a **stable** sort (counting sort) as the subroutine.

Input	Sort by 1s	Sort by 10s	Sort by 100s
	→	→	→
329	720	720	329
457	355	329	355
657	436	436	436
839	457	839	457
436	657	355	657
720	329	457	720
355	839	657	839

Why Least Significant Digit First?

If we sorted most-significant first, we'd need to recursively sort within each “bucket.” Sorting LSD-first with a stable sort automatically preserves the order of higher-significance digits sorted in earlier passes.

Time: $\Theta(d(n + k))$. When d is constant and $k = O(n)$, this is $\Theta(n)$.

Choosing the Radix

Given n numbers, each b bits. Choose radix r , giving $d = \lceil b/\lg r \rceil$ digits in range $[0, r-1]$. Time = $\Theta\left(\frac{b}{\lg r}(n + r)\right)$. Optimal $r = \Theta(n)$ gives time $\Theta(bn/\lg n)$.

7.4 Bucket Sort — Expected $\Theta(n)$

Bucket Sort

Assumes input is uniformly distributed over $[0, 1)$. Distribute n elements into n equal-width buckets, sort each bucket (insertion sort), then concatenate.

Expected time: $\Theta(n)$ when input is uniform.

Worst case: $\Theta(n^2)$ (all elements in one bucket).

Algorithm	Time	Space	Stable	In-place	Comparison?
Counting Sort	$\Theta(n + k)$	$\Theta(n + k)$	Yes	No	No
Radix Sort	$\Theta(d(n + k))$	$\Theta(n + k)$	Yes	No	No
Bucket Sort	$\Theta(n)$ avg	$\Theta(n)$	Yes	No	No

8 Medians and Order Statistics

8.1 The Selection Problem

Selection Problem

Given an unsorted array A of n elements, find the i -th smallest element (the i -th **order statistic**).

Special cases: $i = 1$ (minimum), $i = n$ (maximum), $i = \lfloor (n + 1)/2 \rfloor$ (median).

Minimum/Maximum: Trivially $\Theta(n)$ with a single scan ($n - 1$ comparisons).

Simultaneous min and max: Can be done in $\lceil 3n/2 \rceil - 2$ comparisons by processing elements in pairs.

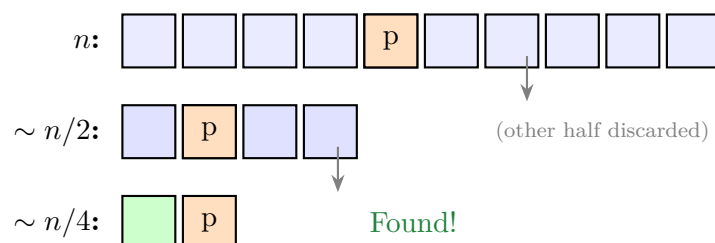
8.2 Randomized Select — Expected $\Theta(n)$

Use the Quicksort partition, but recurse into *only one* side:

Algorithm 12 Randomized-Select(A, p, r, i)

```

1: if  $p = r$  then return  $A[p]$ 
2: end if
3:  $q = \text{Randomized-Partition}(A, p, r)$ 
4:  $k = q - p + 1$  ▷ rank of pivot
5: if  $i = k$  then return  $A[q]$  ▷ pivot is the answer
6: else if  $i < k$  then return Randomized-Select( $A, p, q - 1, i$ )
7: else return Randomized-Select( $A, q + 1, r, i - k$ )
8: end if
```



Case	Time
Expected	$\Theta(n)$
Worst case	$\Theta(n^2)$

Why Expected $\Theta(n)$

Each partition reduces the problem by a constant fraction (in expectation). Expected work: $n + n/2 + n/4 + \dots = 2n = \Theta(n)$.

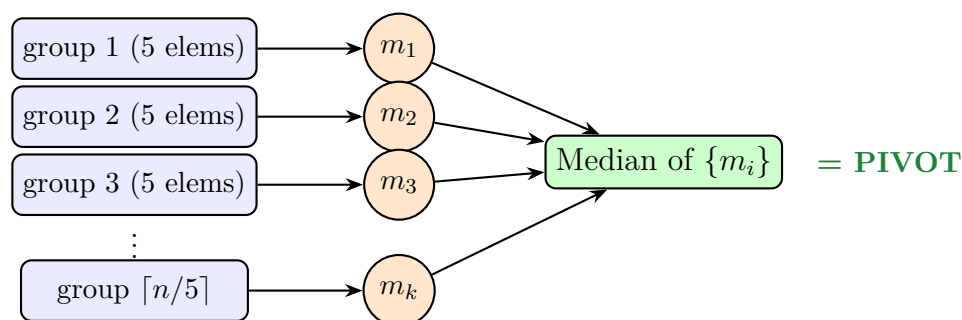
More precisely: $E[T(n)] \leq 2 \cdot \sum_{k=\lceil n/2 \rceil}^{n-1} \frac{T(k)}{n} + O(n)$, which solves to $E[T(n)] = O(n)$.

8.3 Worst-Case Linear Selection (Median of Medians)

SELECT — Worst-Case $\Theta(n)$

The **Median of Medians** algorithm guarantees a good pivot:

1. Divide n elements into groups of 5.
2. Find the median of each group (by brute force).
3. Recursively find the median of these $\lceil n/5 \rceil$ medians. Use this as pivot.
4. Partition around this pivot and recurse into the relevant side.



8.3.1 Why Groups of 5?

The median of medians guarantees pivot rank between $3n/10$ and $7n/10$. This gives the recurrence:

$$T(n) \leq T(\lceil n/5 \rceil) + T(7n/10) + O(n)$$

Since $n/5 + 7n/10 = 9n/10 < n$, this solves to $T(n) = \Theta(n)$.

Why Not Groups of 3?

With groups of 3, the pivot is between $n/3$ and $2n/3$: $T(n) \leq T(n/3) + T(2n/3) + O(n)$. Since $1/3 + 2/3 = 1$, this gives $T(n) = O(n \lg n)$, not linear. Groups of 5 is the smallest odd group size that works.

8.4 Summary: Sorting & Selection

Algorithm	Best	Worst	Notes
Insertion Sort	$\Theta(n)$	$\Theta(n^2)$	In-place, stable
Merge Sort	$\Theta(n \lg n)$	$\Theta(n \lg n)$	Stable, $O(n)$ space
Heapsort	$\Theta(n \lg n)$	$\Theta(n \lg n)$	In-place, not stable
Quicksort	$\Theta(n \lg n)$	$\Theta(n^2)$	In-place, fast in practice
Counting Sort	$\Theta(n + k)$	$\Theta(n + k)$	Integers in $[0, k]$
Radix Sort	$\Theta(dn)$	$\Theta(dn)$	d -digit numbers
Bucket Sort	$\Theta(n)$	$\Theta(n^2)$	Uniform distribution
Randomized Select	$\Theta(n)$	$\Theta(n^2)$	Expected linear
Median of Medians	$\Theta(n)$	$\Theta(n)$	Guaranteed linear

9 Elementary Data Structures

This chapter covers the fundamental building blocks: stacks, queues, linked lists, and representations of rooted trees. These structures underpin nearly every advanced algorithm.

9.1 Stacks

Stack

A **stack** is a **LIFO** (Last-In, First-Out) data structure. The element deleted is the one most recently inserted. Think of a stack of plates.

Operations:

- **Push**(S, x): insert x on top.
- **Pop**(S): remove and return the top element.
- **Top**(S) / **Peek**(S): return the top element without removing.
- **Empty**(S): return whether the stack is empty.

All operations are $O(1)$.

9.1.1 Array-Based Implementation

We maintain an attribute $S.top$ that indexes the most recently inserted element. The stack consists of elements $S[1..S.top]$.

Algorithm 13 Stack Operations (Array-Based)

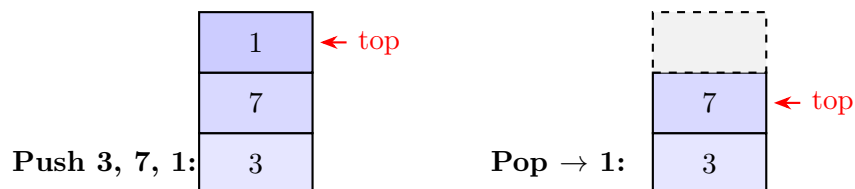
```

1: function STACK-EMPTY( $S$ )
2:   if  $S.top = 0$  then return TRUE
3:   else return FALSE
4:   end if
5: end function

6: function PUSH( $S, x$ )
7:    $S.top = S.top + 1$ 
8:    $S[S.top] = x$ 
9: end function

10: function POP( $S$ )
11:   if Stack-Empty( $S$ ) then
12:     error "underflow"
13:   else
14:      $S.top = S.top - 1$ 
15:     return  $S[S.top + 1]$ 
16:   end if
17: end function

```



9.1.2 C++ Implementation

```

1 struct Stack {
2     int data[MAX_SIZE];
3     int top = -1; // -1 means empty (0-indexed)
4
5     bool empty() { return top == -1; }
6
7     void push(int x) {
8         if (top == MAX_SIZE - 1) throw runtime_error("overflow");
9         data[++top] = x;
10    }
11
12    int pop() {
13        if (empty()) throw runtime_error("underflow");
14        return data[top--];
15    }
16
17    int peek() {
18        if (empty()) throw runtime_error("empty");
19        return data[top];
20    }
21 };

```

Listing 7: Stack using array

```

1 #include <stack>
2 stack<int> s;
3 s.push(3);
4 s.push(7);
5 s.push(1);
6 cout << s.top(); // 1
7 s.pop(); // removes 1
8 cout << s.top(); // 7
9 cout << s.size(); // 2
10 cout << s.empty(); // false

```

Listing 8: Using STL stack

9.2 Queues

Queue

A **queue** is a **FIFO** (First-In, First-Out) data structure. The element deleted is the one that has been in the set the longest. Think of a line of people.

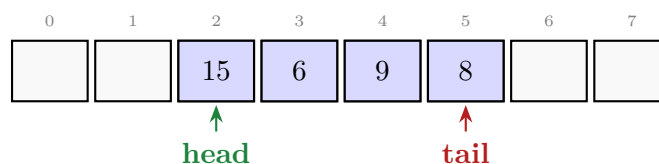
Operations:

- **Enqueue**(Q, x): insert x at the tail (back).
- **Dequeue**(Q): remove and return the element at the head (front).

All operations are $O(1)$.

9.2.1 Circular Array Implementation

Use a *circular* array with **head** and **tail** pointers. When either pointer reaches the end, it wraps around to position 1.



Algorithm 14 Queue Operations (Circular Array)

```

1: function ENQUEUE( $Q, x$ )
2:    $Q[Q.\text{tail}] = x$ 
3:   if  $Q.\text{tail} = Q.\text{length}$  then  $Q.\text{tail} = 1$ 
4:   else  $Q.\text{tail} = Q.\text{tail} + 1$ 
5:   end if
6: end function

7: function DEQUEUE( $Q$ )
8:    $x = Q[Q.\text{head}]$ 
9:   if  $Q.\text{head} = Q.\text{length}$  then  $Q.\text{head} = 1$ 
10:  else  $Q.\text{head} = Q.\text{head} + 1$ 
11:  end if
12:  return  $x$ 
13: end function

```

```

1 struct Queue {
2     int data[MAX_SIZE];
3     int head = 0, tail = 0, cnt = 0;
4
5     bool empty() { return cnt == 0; }
6     bool full() { return cnt == MAX_SIZE; }
7
8     void enqueue(int x) {
9         if (full()) throw runtime_error("overflow");
10        data[tail] = x;

```

```

11     tail = (tail + 1) % MAX_SIZE;
12     cnt++;
13 }
14
15 int dequeue() {
16     if (empty()) throw runtime_error("underflow");
17     int x = data[head];
18     head = (head + 1) % MAX_SIZE;
19     cnt--;
20     return x;
21 }
22
23 int front() { return data[head]; }
24 };

```

Listing 9: Queue using circular array

```

1 #include <queue>
2 queue<int> q;
3 q.push(3);    // enqueue
4 q.push(7);
5 cout << q.front(); // 3
6 q.pop();      // dequeue (removes 3)
7 cout << q.front(); // 7
8
9 // Deque: double-ended queue
10 #include <deque>
11 deque<int> dq;
12 dq.push_back(1); dq.push_front(2);
13 dq.pop_back();   dq.pop_front();
14 // Also supports random access: dq[i]

```

Listing 10: Using STL queue and deque

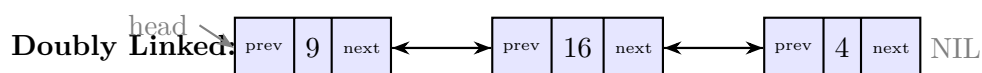
9.3 Linked Lists

Linked List

A **linked list** is a data structure where elements (nodes) are connected by pointers. Each node stores a **key** and pointer(s) to adjacent nodes.

Variants:

- **Singly linked**: each node has a **next** pointer.
- **Doubly linked**: each node has **next** and **prev** pointers.
- **Circular**: the last node points back to the first.
- **With sentinel**: a dummy node simplifies boundary conditions.



9.3.1 Operations and Complexity

Operation	Singly	Doubly	Notes
Search	$O(n)$	$O(n)$	Linear scan
Insert at head	$O(1)$	$O(1)$	Just pointer updates
Insert after node	$O(1)$	$O(1)$	Given pointer to node
Delete (given pointer)	$O(n)$	$O(1)$	Singly needs predecessor

```

1 struct Node {
2     int key;
3     Node* prev;
4     Node* next;
5     Node(int k) : key(k), prev(nullptr), next(nullptr) {}
6 };
7
8 struct DoublyLinkedList {
9     Node* head = nullptr;
10
11     // Search: O(n)
12     Node* search(int k) {
13         Node* x = head;
14         while (x != nullptr && x->key != k)
15             x = x->next;
16         return x;
17     }
18
19     // Insert at head: O(1)
20     void insert(int k) {
21         Node* x = new Node(k);
22         x->next = head;
23         if (head != nullptr)
24             head->prev = x;
25         head = x;
26     }
27
28     // Delete node (given pointer): O(1)
29     void remove(Node* x) {
30         if (x->prev != nullptr)
31             x->prev->next = x->next;
32         else
33             head = x->next; // deleting head
34         if (x->next != nullptr)
35             x->next->prev = x->prev;
36         delete x;
37     }
38 };

```

Listing 11: Doubly linked list implementation

9.3.2 Sentinels

A **sentinel** (dummy node) $L.nil$ sits between the head and tail, turning the list into a circular doubly linked list. This eliminates all boundary checks (if $x \rightarrow prev \neq nullptr$) and simplifies code:

```

1 struct SentinelList {
2     Node* nil; // sentinel node
3
4     SentinelList() {
5         nil = new Node(0);
6         nil->next = nil; // empty list: sentinel points to itself
7         nil->prev = nil;
8     }
9
10    void insert(int k) {
11        Node* x = new Node(k);
12        x->next = nil->next;
13        nil->next->prev = x;
14        nil->next = x;
15        x->prev = nil;
16    }
17
18    void remove(Node* x) {
19        x->prev->next = x->next; // no boundary check needed!
20        x->next->prev = x->prev;
21        delete x;
22    }
23
24    Node* search(int k) {
25        Node* x = nil->next;
26        while (x != nil && x->key != k)
27            x = x->next;
28        return (x != nil) ? x : nullptr;
29    }
30 };

```

Listing 12: Doubly linked list with sentinel

When to Use Sentinels

Sentinels reduce code complexity by eliminating edge cases at the cost of a small amount of extra memory. Use them when list manipulations are frequent and clarity matters. Avoid when memory is extremely tight or the list has very few elements (the sentinel overhead is proportionally larger).

9.4 Representing Rooted Trees

9.4.1 Binary Trees

Each node stores: p (parent), left (left child), right (right child).

```

1 struct TreeNode {
2     int key;
3     TreeNode* parent;
4     TreeNode* left;
5     TreeNode* right;
6     TreeNode(int k) : key(k), parent(nullptr),
7                     left(nullptr), right(nullptr) {}
8 };

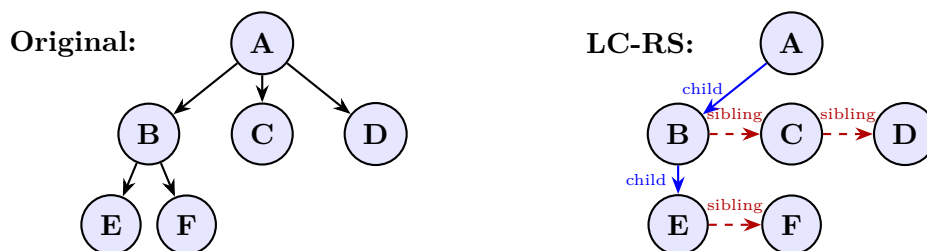
```

Listing 13: Binary tree node

9.4.2 Trees with Unbounded Branching

For trees where the number of children varies, use the **left-child, right-sibling** representation:

- $x.\text{left_child}$: pointer to the *leftmost* child of x .
- $x.\text{right_sibling}$: pointer to the sibling immediately to x 's right.
- $x.\text{parent}$: pointer to parent.



```

1 struct LCRSNode {
2     int key;
3     LCRSNode* parent;
4     LCRSNode* left_child; // leftmost child
5     LCRSNode* right_sibling; // next sibling
6     LCRSNode(int k) : key(k), parent(nullptr),
7                     left_child(nullptr), right_sibling(nullptr) {}
8 };

```

Listing 14: Left-child right-sibling node

Comparison of Tree Representations

Representation	Pointers per Node	Best For
Binary tree (parent + L + R)	3	Binary trees
Left-child right-sibling	3	Unbounded branching
Array of children	varies	Small/fixed branching

10 Hash Tables

Hash tables provide expected $O(1)$ lookup, insertion, and deletion — making them one of the most practically important data structures.

10.1 Direct-Address Tables

Direct-Address Table

If keys are drawn from a small universe $U = \{0, 1, \dots, m-1\}$, store the element with key k directly in slot $T[k]$. All operations are $O(1)$, but space is $\Theta(|U|)$ — wasteful if $|U|$ is huge.

```

1 // Only practical when universe is small
2 struct DirectAddress {
3     int table[MAX_KEY + 1];
4     bool occupied[MAX_KEY + 1];
5
6     DirectAddress() { memset(occupied, false, sizeof(occupied)); }
7     void insert(int k, int v) { table[k] = v; occupied[k] = true; }
8     int search(int k) { return occupied[k] ? table[k] : -1; }
9     void remove(int k) { occupied[k] = false; }
10 };

```

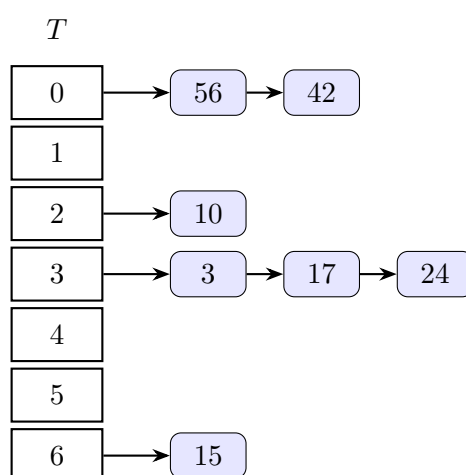
Listing 15: Direct-address table

10.2 Hash Tables with Chaining

Hash Function

A **hash function** $h : U \rightarrow \{0, 1, \dots, m-1\}$ maps keys from a (large) universe U to slots in a table of size m . Since $|U| > m$, **collisions** are inevitable (by the pigeonhole principle).

Chaining resolves collisions by storing all elements that hash to the same slot in a *linked list*:



```

1 struct HashChain {
2     static const int M = 1000003; // prime table size
3     vector<list<pair<int,int>>> table;
4 }

```

```

5  HashChain() : table(M) {}
6
7  int hash(int key) {
8      return ((long long)key % M + M) % M;
9  }
10
11 void insert(int key, int val) {
12     int h = hash(key);
13     for (auto& [k, v] : table[h])
14         if (k == key) { v = val; return; } // update
15     table[h].push_back({key, val});
16 }
17
18 int search(int key) {
19     int h = hash(key);
20     for (auto& [k, v] : table[h])
21         if (k == key) return v;
22     return -1; // not found
23 }
24
25 void remove(int key) {
26     int h = hash(key);
27     table[h].remove_if([&](auto& p) { return p.first == key; });
28 }
29 };

```

Listing 16: Hash table with chaining

10.3 Analysis: Simple Uniform Hashing

Load Factor

The **load factor** $\alpha = n/m$, where n = number of stored elements, m = number of slots. Under chaining, α can exceed 1.

Theorem 10.1 (Expected Search Time under Simple Uniform Hashing). Under the assumption that each key is equally likely to hash to any slot:

- **Unsuccessful search:** $\Theta(1 + \alpha)$
- **Successful search:** $\Theta(1 + \alpha)$

If $n = O(m)$, i.e., $\alpha = O(1)$, then all operations are $O(1)$ on average.

10.4 Hash Functions

10.4.1 The Division Method

$$h(k) = k \bmod m$$

Simple and fast. **Choose** m to be a prime not close to a power of 2.

10.4.2 The Multiplication Method

$$h(k) = \lfloor m \cdot (kA \bmod 1) \rfloor$$

where $0 < A < 1$. Knuth suggests $A \approx (\sqrt{5} - 1)/2 \approx 0.6180339\dots$

Advantage: the value of m is not critical. Works well with $m = 2^p$.

10.4.3 Universal Hashing

Universal Hash Family

A family $\mathcal{H}$ of hash functions is **universal** if for any two distinct keys $k_1 \neq k_2$:

$$\Pr_{h \in \mathcal{H}}[h(k_1) = h(k_2)] \leq \frac{1}{m}$$

By choosing h randomly from $\mathcal{H}$ at runtime, no adversary can force worst-case behavior.

A common universal family: choose prime $p > |U|$ and random $a \in \{1, \dots, p-1\}$, $b \in \{0, \dots, p-1\}$:

$$h_{a,b}(k) = ((ak + b) \bmod p) \bmod m$$

```

1 struct UniversalHash {
2     long long a, b, p, m;
3
4     UniversalHash(int table_size) : m(table_size) {
5         p = 1000000007LL; // large prime
6         mt19937 rng(chrono::steady_clock::now().time_since_epoch().count());
7         a = uniform_int_distribution<long long>(1, p - 1)(rng);
8         b = uniform_int_distribution<long long>(0, p - 1)(rng);
9     }
10
11     int hash(long long key) {
12         return ((a * key + b) % p % m + m) % m;
13     }
14 };

```

Listing 17: Universal hash function

10.5 Open Addressing

Open Addressing

All elements are stored *directly* in the table (no linked lists). On collision, **probe** for the next empty slot. Load factor must satisfy $\alpha < 1$ (table can't be more than full).

The hash function becomes $h(k, i)$ where $i = 0, 1, 2, \dots$ is the probe number.

10.5.1 Probing Strategies

Strategy	Probe Sequence	Issue
Linear	$h(k, i) = (h'(k) + i) \bmod m$	Primary clustering
Quadratic	$h(k, i) = (h'(k) + c_1 i + c_2 i^2) \bmod m$	Secondary clustering
Double hashing	$h(k, i) = (h_1(k) + i \cdot h_2(k)) \bmod m$	Best distribution

Primary Clustering

With linear probing, occupied slots tend to form long contiguous runs (“clusters”). New insertions are more likely to extend a large cluster, causing it to grow even further. This degrades performance to $O(n)$ in pathological cases.

```

1 struct OpenAddressHash {
2     static const int M = 1000003;
3     int table[M];
4     bool occupied[M];
5
6     OpenAddressHash() { memset(occupied, false, sizeof(occupied)); }
7
8     int h1(int k) { return ((long long)k % M + M) % M; }
9     int h2(int k) { return 1 + ((long long)k % (M - 1) + M - 1) % (M - 1); }
10
11    int probe(int k, int i) { return (h1(k) + (long long)i * h2(k)) % M; }
12
13    void insert(int k) {
14        for (int i = 0; i < M; i++) {
15            int j = probe(k, i);
16            if (!occupied[j]) {
17                table[j] = k;
18                occupied[j] = true;
19                return;
20            }
21        }
22        throw runtime_error("hash table overflow");
23    }
24
25    bool search(int k) {
26        for (int i = 0; i < M; i++) {
27            int j = probe(k, i);
28            if (!occupied[j]) return false;
29            if (table[j] == k) return true;
30        }
31        return false;
32    }
33 };

```

Listing 18: Open addressing with double hashing

Theorem 10.2 (Expected Probes for Open Addressing). Under uniform hashing with load factor $\alpha = n/m < 1$:

- **Unsuccessful search:** expected probes $\leq \frac{1}{1-\alpha}$
- **Successful search:** expected probes $\leq \frac{1}{\alpha} \ln \frac{1}{1-\alpha}$

α	Unsuccessful	Successful
0.50	2.0	1.4
0.75	4.0	1.8
0.90	10.0	2.6
0.95	20.0	3.2

Chaining vs Open Addressing

	Chaining	Open Addressing
Collision resolution	Linked lists	Probing
Load factor α	Can be > 1	Must be < 1
Cache performance	Poor (pointer chasing)	Good (array locality)
Deletion	Easy	Tricky (need tombstones)
Implementation	Simpler	More complex

10.6 Using `std::unordered_map`

```

1 #include <unordered_map>
2
3 unordered_map<string, int> mp;
4 mp["alice"] = 100;
5 mp["bob"] = 200;
6 mp.insert({"charlie", 300});
7
8 // Lookup: O(1) average
9 if (mp.count("alice"))
10     cout << mp["alice"]; // 100
11
12 // Iterate
13 for (auto& [key, val] : mp)
14     cout << key << ": " << val << "\n";
15
16 mp.erase("bob");
17 cout << mp.size(); // 2
18
19 // Custom hash for competitive programming (anti-hack)
20 struct custom_hash {
21     size_t operator()(long long x) const {
22         x = (x ^ (x >> 30)) * 0xbf58476d1ce4e5b9LL;
23         x = (x ^ (x >> 27)) * 0x94d049bb133111ebLL;

```

```

24     return x ^ (x >> 31);
25 }
26 };
27 unordered_map<long long, int, custom_hash> safe_map;

```

Listing 19: STL hash map usage

Competitive Programming Warning

The default `std::unordered_map` hash can be attacked with adversarial inputs (e.g., on Codeforces). Always use a **custom hash** or use `std::map` ($O(\log n)$ but safe) when you suspect hack attempts.

11 Binary Search Trees

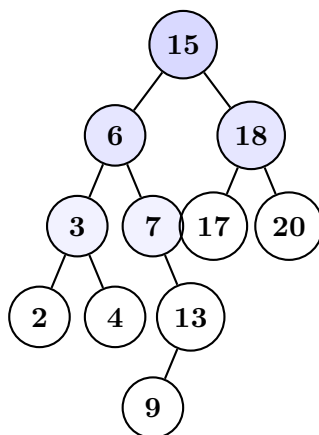
11.1 BST Property

Binary Search Tree Property

For every node x in a BST:

- All keys in the **left** subtree of x are $\leq x.\text{key}$.
- All keys in the **right** subtree of x are $\geq x.\text{key}$.

This property enables efficient search by eliminating half the tree at each step.



11.2 Inorder Traversal

An **inorder traversal** of a BST visits nodes in *sorted order*: visit left subtree, then root, then right subtree.

Algorithm 15 Inorder-Tree-Walk(x)

```

1: if  $x \neq \text{NIL}$  then
2:   Inorder-Tree-Walk( $x.\text{left}$ )
3:   print  $x.\text{key}$ 
4:   Inorder-Tree-Walk( $x.\text{right}$ )
5: end if

```

Time: $\Theta(n)$ — visits every node exactly once.

```

1 struct Node {
2     int key;
3     Node *left, *right, *parent;
4     Node(int k) : key(k), left(nullptr), right(nullptr), parent(nullptr) {}
5 };
6
7 // Inorder: sorted output for BST
8 void inorder(Node* x) {
9     if (!x) return;
10    inorder(x->left);
11    cout << x->key << " ";
12    inorder(x->right);
13 }
14
15 // Preorder: root, left, right (useful for serialization)
16 void preorder(Node* x) {
17     if (!x) return;
18     cout << x->key << " ";
19     preorder(x->left);
20     preorder(x->right);
21 }
22
23 // Postorder: left, right, root (useful for deletion)
24 void postorder(Node* x) {
25     if (!x) return;
26     postorder(x->left);
27     postorder(x->right);
28     cout << x->key << " ";
29 }

```

Listing 20: All three traversals in C++

11.3 Querying a BST

11.3.1 Search

Follow the BST property: go left if key is smaller, right if larger.

Algorithm 16 Tree-Search(x, k)

```

1: if  $x = \text{NIL}$  or  $k = x.\text{key}$  then
2:     return  $x$ 
3: end if
4: if  $k < x.\text{key}$  then
5:     return Tree-Search( $x.\text{left}, k$ )
6: else
7:     return Tree-Search( $x.\text{right}, k$ )
8: end if

```

```

1 Node* search(Node* root, int k) {
2     Node* x = root;
3     while (x != nullptr && x->key != k) {
4         if (k < x->key) x = x->left;
5         else          x = x->right;
6     }
7     return x;
8 }

```

Listing 21: BST search — iterative version (preferred)

11.3.2 Minimum and Maximum

```

1 Node* minimum(Node* x) {
2     while (x->left != nullptr) // go all the way left
3         x = x->left;
4     return x;
5 }
6
7 Node* maximum(Node* x) {
8     while (x->right != nullptr) // go all the way right
9         x = x->right;
10    return x;
11 }

```

Listing 22: BST minimum and maximum

11.3.3 Successor and Predecessor

The **successor** of node x is the node with the smallest key greater than $x.key$.

Algorithm 17 Tree-Successor(x)

```

1: if  $x.right \neq \text{NIL}$  then
2:     return Tree-Minimum( $x.right$ )           ▷ leftmost in right subtree
3: end if
4:  $y = x.parent$ 
5: while  $y \neq \text{NIL}$  and  $x = y.right$  do
6:      $x = y$ 
7:      $y = y.parent$ 
8: end while
9: return  $y$                                    ▷ first ancestor where  $x$  is in the left subtree

```

```

1 Node* successor(Node* x) {
2     if (x->right) // Case 1: has right subtree
3         return minimum(x->right);
4     // Case 2: go up until we find an ancestor where x is in left subtree
5     Node* y = x->parent;
6     while (y && x == y->right) {

```

```

7     x = y;
8     y = y->parent;
9 }
10 return y;
11 }
12
13 Node* predecessor(Node* x) {
14     if (x->left)
15         return maximum(x->left);
16     Node* y = x->parent;
17     while (y && x == y->left) {
18         x = y;
19         y = y->parent;
20     }
21     return y;
22 }

```

Listing 23: Successor and predecessor

All Query Operations: $O(h)$

Search, minimum, maximum, successor, and predecessor all run in $O(h)$ time, where h is the height of the tree. For a balanced BST, $h = O(\lg n)$. For a skewed BST (worst case), $h = O(n)$.

11.4 Insertion

Traverse the tree as in search. When we reach a NIL, insert the new node there.

Algorithm 18 Tree-Insert(T, z)

```

1:  $y = \text{NIL}, x = T.\text{root}$ 
2: while  $x \neq \text{NIL}$  do
3:    $y = x$ 
4:   if  $z.\text{key} < x.\text{key}$  then  $x = x.\text{left}$ 
5:   else  $x = x.\text{right}$ 
6:   end if
7: end while
8:  $z.\text{parent} = y$ 
9: if  $y = \text{NIL}$  then  $T.\text{root} = z$  ▷ tree was empty
10: else if  $z.\text{key} < y.\text{key}$  then  $y.\text{left} = z$ 
11: else  $y.\text{right} = z$ 
12: end if

```

```

1 Node* insert(Node* root, int k) {
2     Node* z = new Node(k);
3     Node* y = nullptr;
4     Node* x = root;
5
6     while (x != nullptr) {

```

```

7     y = x;
8     if (k < x->key) x = x->left;
9     else           x = x->right;
10  }
11  z->parent = y;
12  if (y == nullptr) return z; // tree was empty
13  if (k < y->key) y->left = z;
14  else           y->right = z;
15  return root;
16 }

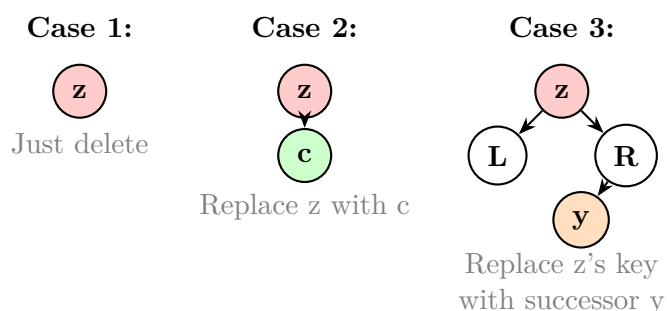
```

Listing 24: BST insertion

11.5 Deletion

Deletion is the most complex BST operation. There are three cases:

1. **z has no children:** simply remove z .
2. **z has one child:** replace z with its child.
3. **z has two children:** replace z with its **successor** y (the minimum of z 's right subtree), then remove y from its original position.



The helper TRANSPLANT replaces subtree rooted at u with subtree rooted at v :

```

1  // Replace subtree rooted at u with subtree rooted at v
2  void transplant(Node*& root, Node* u, Node* v) {
3      if (u->parent == nullptr)
4          root = v;
5      else if (u == u->parent->left)
6          u->parent->left = v;
7      else
8          u->parent->right = v;
9      if (v != nullptr)
10         v->parent = u->parent;
11  }
12
13  void deleteNode(Node*& root, Node* z) {
14      if (z->left == nullptr) {
15          transplant(root, z, z->right); // Case 1 or 2a
16      } else if (z->right == nullptr) {

```



```

17     transplant(root, z, z->left);    // Case 2b
18 } else {
19     // Case 3: two children
20     Node* y = minimum(z->right);    // successor
21     if (y->parent != z) {
22         transplant(root, y, y->right); // detach y
23         y->right = z->right;
24         y->right->parent = y;
25     }
26     transplant(root, z, y);        // replace z with y
27     y->left = z->left;
28     y->left->parent = y;
29 }
30 delete z;
31 }

```

Listing 25: BST deletion with transplant

11.6 Randomly Built BSTs

Theorem 11.1. The expected height of a randomly built BST on n distinct keys (inserted in random order) is $O(\lg n)$.

Worst Case is Still $O(n)$

If keys are inserted in sorted order, the BST degenerates into a linked list with height $n - 1$. This is why we need **balanced BSTs** (Chapter 13).

11.7 Using STL Ordered Sets and Maps

```

1 #include <set>
2 #include <map>
3
4 // std::set: ordered, unique keys (Red-Black Tree internally)
5 set<int> s;
6 s.insert(5); s.insert(3); s.insert(8); s.insert(3); // {3, 5, 8}
7 s.erase(5); // {3, 8}
8 auto it = s.find(3); // O(log n)
9 auto it2 = s.lower_bound(4); // first >= 4, i.e., points to 8
10 cout << *s.begin(); // minimum: 3
11 cout << *s.rbegin(); // maximum: 8
12
13 // std::map: ordered key-value pairs
14 map<string, int> m;
15 m["apple"] = 5;
16 m["banana"] = 3;
17 for (auto& [key, val] : m) // sorted by key
18     cout << key << ": " << val << "\n";
19

```

```

20 // Useful: ordered statistics with policy-based tree
21 // #include <ext/pb_ds/assoc_container.hpp>
22 // #include <ext/pb_ds/tree_policy.hpp>
23 // Provides find_by_order(k) and order_of_key(k)

```

Listing 26: std::set and std::map (balanced BST underneath)

12 AVL Trees

AVL trees (Adelson-Velsky and Landis, 1962) are the *first* self-balancing BST ever invented. They enforce a stricter balance condition than Red-Black trees, resulting in slightly faster lookups but slower insertions/deletions.

12.1 AVL Property

AVL Tree

An **AVL tree** is a BST where, for every node x :

$$|\text{height}(x.\text{left}) - \text{height}(x.\text{right})| \leq 1$$

The **balance factor** $\text{bf}(x) = \text{height}(x.\text{left}) - \text{height}(x.\text{right}) \in \{-1, 0, +1\}$.

Theorem 12.1 (AVL Height Bound). An AVL tree with n nodes has height $h \leq 1.44 \lg(n+2) - 0.328$. In other words, $h = O(\lg n)$.

Proof of Height Bound

Let $N(h)$ be the *minimum* number of nodes in an AVL tree of height h . The worst case (tallest for fewest nodes) is when one subtree has height $h-1$ and the other has height $h-2$ (maximum allowed imbalance):

$$N(h) = N(h-1) + N(h-2) + 1$$

with $N(0) = 1$, $N(1) = 2$. This is like the Fibonacci recurrence!

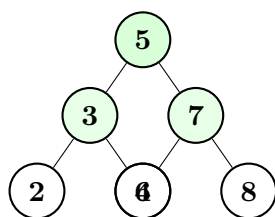
We can show $N(h) \geq \phi^h - 1$ where $\phi = (1 + \sqrt{5})/2 \approx 1.618$. Therefore:

$$n \geq N(h) \geq \phi^h - 1 \implies h \leq \log_{\phi}(n+1) = \frac{\ln(n+1)}{\ln \phi} \approx 1.44 \lg(n+1)$$

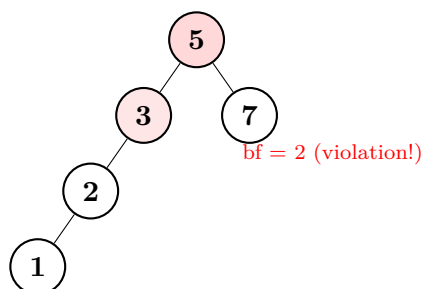
Since $\log_{\phi} 2 \approx 1.44$, AVL trees are at most 44% taller than perfectly balanced trees.

□

AVL (balanced):



NOT AVL:



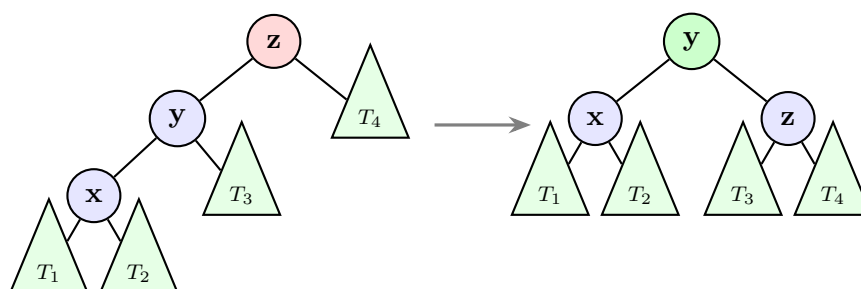
12.2 Rotations

AVL trees use the same rotations as Red-Black trees (left and right rotations), but the triggering conditions and cases differ.

When an insertion or deletion causes $|\text{bf}(x)| = 2$, we fix it with one of **four rotation cases**:

Case	Condition	Fix	Rotations
LL	$\text{bf}(x) = +2, \text{bf}(x.\text{left}) \geq 0$	Right rotate x	1
RR	$\text{bf}(x) = -2, \text{bf}(x.\text{right}) \leq 0$	Left rotate x	1
LR	$\text{bf}(x) = +2, \text{bf}(x.\text{left}) < 0$	Left rotate $x.\text{left}$, then right rotate x	2
RL	$\text{bf}(x) = -2, \text{bf}(x.\text{right}) > 0$	Right rotate $x.\text{right}$, then left rotate x	2

LL Case: Right Rotate



12.3 Implementation

```

1 struct AVLNode {
2     int key, height;
3     AVLNode *left, *right;
4     AVLNode(int k) : key(k), height(1), left(nullptr), right(nullptr) {}
5 };
6
7 int height(AVLNode* n) { return n ? n->height : 0; }
8
9 int balanceFactor(AVLNode* n) {

```

```

10     return n ? height(n->left) - height(n->right) : 0;
11 }
12
13 void updateHeight(AVLNode* n) {
14     if (n) n->height = 1 + max(height(n->left), height(n->right));
15 }
16
17 // Right rotation (LL case)
18 AVLNode* rightRotate(AVLNode* y) {
19     AVLNode* x = y->left;
20     AVLNode* T2 = x->right;
21     x->right = y;
22     y->left = T2;
23     updateHeight(y);
24     updateHeight(x);
25     return x; // new root
26 }
27
28 // Left rotation (RR case)
29 AVLNode* leftRotate(AVLNode* x) {
30     AVLNode* y = x->right;
31     AVLNode* T2 = y->left;
32     y->left = x;
33     x->right = T2;
34     updateHeight(x);
35     updateHeight(y);
36     return y; // new root
37 }
38
39 // Rebalance node after insertion/deletion
40 AVLNode* balance(AVLNode* node) {
41     updateHeight(node);
42     int bf = balanceFactor(node);
43
44     // LL case
45     if (bf > 1 && balanceFactor(node->left) >= 0)
46         return rightRotate(node);
47
48     // RR case
49     if (bf < -1 && balanceFactor(node->right) <= 0)
50         return leftRotate(node);
51
52     // LR case
53     if (bf > 1 && balanceFactor(node->left) < 0) {
54         node->left = leftRotate(node->left);
55         return rightRotate(node);
56     }
57
58     // RL case
59     if (bf < -1 && balanceFactor(node->right) > 0) {
60         node->right = rightRotate(node->right);

```

```

61     return leftRotate(node);
62 }
63
64 return node; // already balanced
65 }

```

Listing 27: AVL tree — complete implementation

```

1  AVLNode* insert(AVLNode* node, int key) {
2      // Standard BST insert
3      if (!node) return new AVLNode(key);
4      if (key < node->key)
5          node->left = insert(node->left, key);
6      else if (key > node->key)
7          node->right = insert(node->right, key);
8      else
9          return node; // no duplicates
10
11     return balance(node); // rebalance on the way up
12 }
13
14 AVLNode* minNode(AVLNode* n) {
15     while (n->left) n = n->left;
16     return n;
17 }
18
19 AVLNode* remove(AVLNode* node, int key) {
20     if (!node) return nullptr;
21
22     if (key < node->key)
23         node->left = remove(node->left, key);
24     else if (key > node->key)
25         node->right = remove(node->right, key);
26     else {
27         // Node found
28         if (!node->left || !node->right) {
29             AVLNode* child = node->left ? node->left : node->right;
30             delete node;
31             return child; // might be nullptr
32         }
33         // Two children: replace with inorder successor
34         AVLNode* succ = minNode(node->right);
35         node->key = succ->key;
36         node->right = remove(node->right, succ->key);
37     }
38
39     return balance(node); // rebalance on the way up
40 }
41
42 // Search: identical to BST search, O(lg n) guaranteed
43 AVLNode* search(AVLNode* node, int key) {
44     if (!node || node->key == key) return node;

```

```

45  if (key < node->key) return search(node->left, key);
46  return search(node->right, key);
47  }

```

Listing 28: AVL insertion and deletion

12.4 Correctness of AVL Balancing

Theorem 12.2 (AVL Insert Maintains Balance). After inserting a node into a valid AVL tree, applying at most one single or double rotation on the lowest unbalanced ancestor restores the AVL property for the entire tree.

Proof

Claim: Only one rotation (single or double) is needed after insertion.

After inserting a leaf, only ancestors on the insertion path can become unbalanced. Let z be the *lowest* ancestor with $|\text{bf}(z)| = 2$.

Case 1 (LL/RR): A single rotation at z restores the subtree height to its pre-insertion value, so no ancestor of z is affected. $\square$

Case 2 (LR/RL): A double rotation at z similarly restores the subtree height. $\square$

For deletion: A rotation may reduce the subtree height, potentially unbalancing an ancestor. So deletion may require $O(\lg n)$ rotations (one per level), unlike insertion which needs at most 2. $\square$

12.5 AVL vs Red-Black vs Others

Tree	Search	Insert	Delete	Height	Rotations (Insert)
AVL	$O(\lg n)$	$O(\lg n)$	$O(\lg n)$	$\leq 1.44 \lg n$	≤ 2
Red-Black	$O(\lg n)$	$O(\lg n)$	$O(\lg n)$	$\leq 2 \lg n$	≤ 2
Splay (amort.)	$O(\lg n)$	$O(\lg n)$	$O(\lg n)$	$O(n)$ worst	$O(\lg n)$
Treap (exp.)	$O(\lg n)$	$O(\lg n)$	$O(\lg n)$	$O(\lg n)$	$O(\lg n)$

When to Use Which

- **AVL:** Best for *lookup-heavy* workloads. Stricter balance $\Rightarrow$ shorter height $\Rightarrow$ faster search.
- **Red-Black:** Best for *insert/delete-heavy* workloads. Used in C++ STL (`std::set/map`).
- **Splay:** Best when access patterns have locality (recently accessed items are fast).
- **Treap:** Simplest to implement for competitive programming. Randomized balance.

13 Counting and Structural Properties of Trees

This section collects the combinatorial results about binary trees, BSTs, AVL trees, and B-trees that appear in DSA courses and exams.

13.1 Binary Tree Fundamentals

Node Classification

- **Leaf (external node):** a node with no children.
- **Internal node:** a node with at least one child.
- **Full binary tree:** every node has 0 or 2 children.
- **Complete binary tree:** all levels full except possibly the last, which is filled left-to-right.
- **Perfect binary tree:** all internal nodes have 2 children and all leaves are at the same depth.

Theorem 13.1 (Leaves vs Internal Nodes in a Full Binary Tree). In a full binary tree with n internal nodes, the number of leaves is:

$$L = n + 1$$

Proof by Induction

Base case ($n = 0$): A single node (no internal nodes) has $L = 1 = 0 + 1$. ✓

Inductive step: Assume true for all full binary trees with $< n$ internal nodes. Take a full binary tree T with n internal nodes. Pick any internal node v whose two children are both leaves. Remove v 's two children, making v a leaf.

The resulting tree T' is a full binary tree with $n - 1$ internal nodes. By inductive hypothesis, T' has $(n - 1) + 1 = n$ leaves.

Going back to T : we removed 2 leaves and turned v from internal to leaf, so T has $n - 2 + 1 + 2 = n + 1$ leaves.

Wait — more carefully: T' has n leaves. v is one of those leaves. To get T , we give v two children, so v becomes internal (lose 1 leaf) and gain 2 new leaves. Total: $n - 1 + 2 = n + 1$ leaves. □

Theorem 13.2 (General Binary Tree Edge/Node Counts). For any binary tree with n nodes:

- Number of edges: $n - 1$ (every node except root has exactly one parent edge).
- If the tree has n_0 leaves, n_1 nodes with 1 child, and n_2 nodes with 2 children, then $n = n_0 + n_1 + n_2$ and:

$$n_0 = n_2 + 1$$

Proof (Edge Counting)

Count edges from the child side: every non-root node contributes 1 edge, giving $n - 1$ edges.

Count edges from the parent side: a node with k children contributes k edges:

$$\text{edges} = 0 \cdot n_0 + 1 \cdot n_1 + 2 \cdot n_2$$

Setting equal: $n_1 + 2n_2 = n - 1 = n_0 + n_1 + n_2 - 1$.

Simplifying: $2n_2 = n_0 + n_2 - 1 \implies n_0 = n_2 + 1$. $\square$

Theorem 13.3 (Perfect Binary Tree Properties). A perfect binary tree of height h (root at height 0) has:

- $2^{h+1} - 1$ total nodes.
- 2^h leaves.
- $2^h - 1$ internal nodes.
- Height $h = \lfloor \lg n \rfloor$ for n nodes.

Proof

Level k has 2^k nodes ($k = 0, 1, \dots, h$). Total: $\sum_{k=0}^h 2^k = 2^{h+1} - 1$ (geometric sum).

Leaves are at level h : 2^h . Internal nodes: $2^{h+1} - 1 - 2^h = 2^h - 1$. $\square$

13.2 Number of Structurally Unique Binary Trees — Catalan Numbers

Theorem 13.4 (Catalan Numbers). The number of structurally distinct binary trees with n nodes is the n -th **Catalan number**:

$$C_n = \frac{1}{n+1} \binom{2n}{n} = \frac{(2n)!}{(n+1)!n!}$$

First values: $C_0 = 1$, $C_1 = 1$, $C_2 = 2$, $C_3 = 5$, $C_4 = 14$, $C_5 = 42$.

Derivation via Recurrence

Let C_n = number of distinct binary trees with n nodes. The root uses 1 node. The left subtree gets k nodes and the right gets $n - 1 - k$, for $k = 0, 1, \dots, n - 1$:

$$C_n = \sum_{k=0}^{n-1} C_k \cdot C_{n-1-k}, \quad C_0 = 1$$

This is the **Catalan recurrence**. To solve it, use generating functions. Let $f(x) = \sum_{n=0}^{\infty} C_n x^n$. The recurrence gives:

$$f(x) = 1 + x \cdot f(x)^2$$

(The x accounts for the root; $f(x)^2$ is the convolution of left and right subtrees.)
Solving the quadratic: $xf^2 - f + 1 = 0$:

$$f(x) = \frac{1 - \sqrt{1 - 4x}}{2x}$$

(choosing the “ $-$ ” root so $f(0) = 1$). Expanding via the generalized binomial theo-

rem:

$$\sqrt{1-4x} = \sum_{n=0}^{\infty} \binom{1/2}{n} (-4x)^n$$

After simplification, the coefficient of x^n gives:

$$C_n = \frac{1}{n+1} \binom{2n}{n} \quad \square$$

Catalan Number Applications

C_n counts *many* structures:

1. Structurally distinct binary trees with n nodes.
2. Distinct BSTs storing keys $\{1, 2, \dots, n\}$ (LC 96!).
3. Valid parenthesizations with n pairs: e.g., $((()()))$.
4. Monotonic lattice paths from $(0, 0)$ to (n, n) staying below diagonal.
5. Triangulations of a convex $(n+2)$ -gon.
6. Full binary trees with $n+1$ leaves.

```

1 // Method 1: DP (the recurrence)
2 long long catalanDP(int n) {
3     vector<long long> C(n + 1, 0);
4     C[0] = C[1] = 1;
5     for (int i = 2; i <= n; i++)
6         for (int j = 0; j < i; j++)
7             C[i] += C[j] * C[i-1-j];
8     return C[n];
9 }
10
11 // Method 2: Direct formula (for small n, or use modular arithmetic)
12 long long catalanFormula(int n) {
13     long long res = 1;
14     for (int i = 0; i < n; i++) {
15         res *= (2 * n - i);
16         res /= (i + 1);
17     }
18     return res / (n + 1);
19 }

```

Listing 29: Compute n -th Catalan number

Asymptotic growth: $C_n \sim \frac{4^n}{n^{3/2}\sqrt{\pi}}$.

13.3 Number of Unique BSTs

Theorem 13.5 (Unique BSTs). The number of structurally distinct BSTs storing n distinct keys is C_n (the n -th Catalan number).

Proof

Fix the keys as $1, 2, \dots, n$. Choose key k as the root. Then keys $1, \dots, k-1$ go in the left subtree and keys $k+1, \dots, n$ go in the right. The number of BSTs with root k is:

$$\text{BST}(k) = C_{k-1} \cdot C_{n-k}$$

Summing over all choices of root:

$$\text{BST}(n) = \sum_{k=1}^n C_{k-1} \cdot C_{n-k} = C_n$$

by the Catalan recurrence. $\square$

```

1 int numTrees(int n) {
2     vector<int> dp(n + 1, 0);
3     dp[0] = dp[1] = 1;
4     for (int i = 2; i <= n; i++)
5         for (int root = 1; root <= i; root++)
6             dp[i] += dp[root-1] * dp[i-root]; // left * right
7     return dp[n];
8 }
```

Listing 30: LC 96: Unique BSTs — $O(n^2)$ DP

13.4 AVL Tree Counting

Theorem 13.6 (Minimum Nodes in AVL Tree of Height h). Let $N(h)$ be the minimum number of nodes in an AVL tree of height h . Then:

$$N(h) = N(h-1) + N(h-2) + 1, \quad N(0) = 1, N(1) = 2$$

This gives $N(h) = F_{h+3} - 1$, where F_k is the k -th Fibonacci number.

Proof

The minimum-node AVL tree of height h (called a **Fibonacci tree**) has:

- One subtree of height $h-1$ (to achieve height h).
- The other subtree of height $h-2$ (minimum allowed by AVL balance).
- Both subtrees are themselves minimum-node AVL trees.

Hence: $N(h) = 1 + N(h-1) + N(h-2)$.

Let $M(h) = N(h) + 1$. Then $M(h) = M(h-1) + M(h-2)$ with $M(0) = 2 = F_4$, $M(1) = 3 = F_5$. So $M(h) = F_{h+3}$, giving $N(h) = F_{h+3} - 1$. $\square$

h	0	1	2	3	4	5	6	7
$N(h)$	1	2	4	7	12	20	33	54
F_{h+3}	2	3	5	8	13	21	34	55

Theorem 13.7 (Maximum Height of AVL Tree with n Nodes).

$$h \leq 1.4404 \log_2(n + 2) - 0.3277$$

Equivalently: $n \geq N(h) = F_{h+3} - 1 \geq \phi^{h+1} - 1$, so $h \leq \log_\phi(n + 1) - 1 \approx 1.44 \lg(n + 1)$.

Theorem 13.8 (Maximum Nodes in AVL Tree of Height h). Same as any binary tree: $2^{h+1} - 1$ (a perfect binary tree is a valid AVL tree).

Theorem 13.9 (Leaves in AVL Trees). In an AVL tree with n nodes:

- **Minimum leaves:** $\lceil n/2 \rceil$ (when the tree is as “stringy” as AVL allows).
- **Maximum leaves:** $\lceil (n + 1)/2 \rceil$ (when the tree is nearly complete).

For a minimum-node AVL tree of height h : leaves = F_{h+1} (Fibonacci!).

Proof of Fibonacci Tree Leaf Count

Let $L(h)$ = leaves in a Fibonacci tree (min-node AVL) of height h .

- $L(0) = 1 = F_1$, $L(1) = 1 = F_2$.
- $L(h) = L(h-1) + L(h-2)$ (leaves come from both subtrees, which are Fibonacci trees of height $h-1$ and $h-2$).

This is exactly the Fibonacci recurrence, so $L(h) = F_{h+1}$. $\square$

13.5 B-Tree Counting

B-Tree of Order m (Minimum Degree t)

A B-tree of minimum degree t (order $m = 2t$) satisfies:

1. Every node has at most $m - 1 = 2t - 1$ keys.
2. Every non-root internal node has at least $t - 1$ keys.
3. The root has at least 1 key (if non-empty).
4. All leaves are at the same depth.
5. An internal node with k keys has $k + 1$ children.

Theorem 13.10 (B-Tree Height Bound). A B-tree of minimum degree t with $n \geq 1$ keys has height:

$$h \leq \log_t \frac{n + 1}{2}$$

Proof

At minimum fill, the root has 1 key (2 children), and every other internal node has $t - 1$ keys (t children). At depth d :

- Depth 0 (root): 1 node, ≥ 1 key.
- Depth 1: ≥ 2 nodes, each $\geq t - 1$ keys.
- Depth d : $\geq 2t^{d-1}$ nodes, each $\geq t - 1$ keys.

Total keys n satisfies:

$$n \geq 1 + (t - 1) \sum_{d=1}^h 2t^{d-1} = 1 + 2(t - 1) \cdot \frac{t^h - 1}{t - 1} = 2t^h - 1$$

So $t^h \leq (n+1)/2$, giving $h \leq \log_t \frac{n+1}{2}$. $\square$

Theorem 13.11 (B-Tree Node and Key Counts). A B-tree of min degree t and height h :

	Minimum	Maximum
Keys per node (non-root)	$t - 1$	$2t - 1$
Keys per root	1	$2t - 1$
Children per internal node	t	$2t$
Total nodes	$1 + 2^{\frac{t^h-1}{t-1}}$	$\frac{(2t)^{h+1}-1}{2t-1}$
Total keys (n)	$2t^h - 1$	$(2t)^{h+1} - 1$
Number of leaves	$2^{t^{h-1}}$ (min)	$(2t)^h$ (max)

Derivation of Max Keys

At maximum fill, every node has $2t - 1$ keys and $2t$ children. Depth d has $(2t)^d$ nodes. Total keys:

$$\sum_{d=0}^h (2t)^d \cdot (2t - 1) = (2t - 1) \cdot \frac{(2t)^{h+1} - 1}{2t - 1} = (2t)^{h+1} - 1 \quad \square$$

B-Tree Practical Example

- Each node has 1–3 keys and 2–4 children.
- With $h = 3$: min keys = $2 \cdot 2^3 - 1 = 15$, max keys = $4^4 - 1 = 255$.
- A B-tree with $t = 1000$ and $n = 10^9$: $h \leq \log_{1000}(5 \times 10^8) \approx 3$. Only 3 disk reads!

13.6 Red-Black Tree Counting

Theorem 13.12 (Red-Black Tree Height). A red-black tree with n internal nodes has height at most $2 \lg(n+1)$.

Proof

Claim 1: Any subtree rooted at x has $\geq 2^{bh(x)} - 1$ internal nodes, where $bh(x)$ is the black-height (number of black nodes on any root-to-leaf path, not counting x).

Proof by induction on height of x :

- Base: x is a leaf (NIL): $2^0 - 1 = 0$ internal nodes. $\checkmark$
- Inductive step: x has children with black-height $\geq bh(x) - 1$ (exactly $bh(x) - 1$ if child is black). By IH each child subtree has $\geq 2^{bh(x)-1} - 1$ nodes. Total: $n \geq 2(2^{bh(x)-1} - 1) + 1 = 2^{bh(x)} - 1$. $\checkmark$

Claim 2: $bh(\text{root}) \geq h/2$.

Since no two consecutive red nodes exist (property 4), at least half the nodes on any

root-to-leaf path are black. So $bh \geq h/2$.
Combining: $n \geq 2^{h/2} - 1 \implies h \leq 2 \lg(n + 1)$. $\square$

13.7 General Counting Formulas Summary

Quantity	Formula	Reference
Unique binary trees, n nodes	$C_n = \frac{1}{n+1} \binom{2n}{n}$	Catalan
Unique BSTs, n keys	C_n	Same as above
Unique full binary trees, n internal	C_n	Catalan
Leaves in full binary tree	$n_0 = n_2 + 1$	Edge counting
Nodes in perfect tree, height h	$2^{h+1} - 1$	Geometric sum
Min nodes in AVL, height h	$F_{h+3} - 1$	Fibonacci
Leaves in Fibonacci tree, height h	F_{h+1}	Fibonacci
Max AVL height, n nodes	$\leq 1.44 \lg(n + 1)$	Fibonacci inversion
Max RB-tree height, n nodes	$\leq 2 \lg(n + 1)$	Black-height
Max B-tree height, n keys, degree t	$\leq \log_t \frac{n+1}{2}$	Min-fill argument
Min B-tree keys, height h , degree t	$2t^h - 1$	Geometric sum
Max B-tree keys, height h , degree t	$(2t)^{h+1} - 1$	Geometric sum

Common Exam Mistakes

1. The number of unique BSTs with n keys is C_n , **not** $n!$. Different permutations can produce the same BST structure.
2. $n_0 = n_2 + 1$ holds for *any* binary tree, not just full trees. (In a full tree, $n_1 = 0$, but the identity $n_0 = n_2 + 1$ is general.)
3. AVL minimum-node trees follow Fibonacci, not powers of 2.
4. B-tree height is $O(\log_t n)$, not $O(\log_2 n)$. The base matters!
5. “Order” of a B-tree is ambiguous: some texts define it as max children (m), others as min degree (t). CLRS uses t , where $m = 2t$.

14 Red-Black Trees

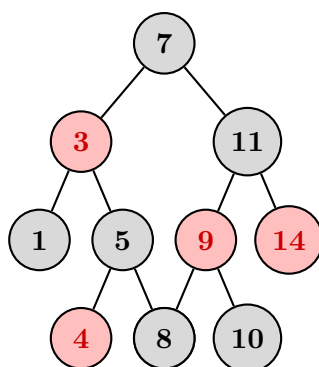
Red-Black trees are self-balancing BSTs that guarantee $O(\lg n)$ height, ensuring all BST operations run in $O(\lg n)$ worst case.

14.1 Properties

Red-Black Tree Properties

A Red-Black tree is a BST where every node has a **color** (red or black), and the following five properties hold:

1. Every node is either **red** or **black**.
2. The **root** is black.
3. Every **leaf (NIL)** is black.
4. If a node is **red**, both its children are **black** (no two red nodes in a row).
5. For each node, all simple paths from that node to descendant leaves contain the **same number of black nodes** (the *black-height*).



Theorem 14.1 (Height Bound). A Red-Black tree with n internal nodes has height at most $2 \lg(n + 1)$.

Proof Sketch

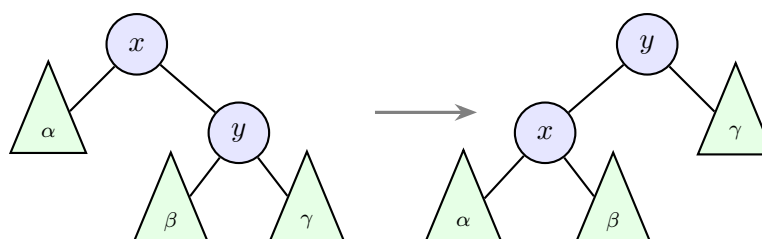
Let $bh(x)$ be the black-height of node x . The subtree rooted at x has $\geq 2^{bh(x)} - 1$ internal nodes (by induction). Since $bh(\text{root}) \geq h/2$ (property 4: at most half the nodes on any path are red):

$$n \geq 2^{h/2} - 1 \implies h \leq 2 \lg(n + 1)$$

14.2 Rotations

Rotations are the fundamental restructuring operation. They change the local structure of the tree while preserving the BST property.

Left-Rotate(x)



```

1 struct RBNode {
2     int key;
3     bool color; // true = RED, false = BLACK
4     RBNode *left, *right, *parent;
5     RBNode(int k) : key(k), color(true),
6         left(nullptr), right(nullptr), parent(nullptr) {}
7 };
8 const bool RED = true, BLACK = false;
9
10 void leftRotate(RBNode*& root, RBNode* x) {
11     RBNode* y = x->right; // y is x's right child
12     x->right = y->left; // turn y's left subtree into x's right
13     if (y->left)
14         y->left->parent = x;
15     y->parent = x->parent; // link x's parent to y
16     if (!x->parent) root = y;
17     else if (x == x->parent->left) x->parent->left = y;
18     else x->parent->right = y;
19     y->left = x; // put x on y's left
20     x->parent = y;
21 }
22
23 void rightRotate(RBNode*& root, RBNode* y) {
24     RBNode* x = y->left;
25     y->left = x->right;
26     if (x->right)
27         x->right->parent = y;
28     x->parent = y->parent;
29     if (!y->parent) root = x;
30     else if (y == y->parent->left) y->parent->left = x;
31     else y->parent->right = x;
32     x->right = y;
33     y->parent = x;
34 }

```

Listing 31: Left and right rotations

Rotation Properties

- Rotations run in $O(1)$ — only pointer changes.
- The BST property is preserved.
- The inorder traversal order is unchanged: $\alpha \prec x \prec \beta \prec y \prec \gamma$.

14.3 Insertion

Step 1: Insert as in a normal BST. Color the new node **red**.

Step 2: Fix violations via RB-INSERT-FIXUP. The only possible violation is property 4 (red parent with red child).

There are **three cases** (and their symmetric mirrors):

Case	Uncle Color	Action	Result
1	Uncle is red	Recolor parent, uncle, grandparent	Move problem up
2	Uncle is black, z is inner child	Rotate z 's parent	Convert to Case 3
3	Uncle is black, z is outer child	Rotate grandparent + recolor	Fixed!

```

1 void rbInsertFixup(RBNode*& root, RBNode* z) {
2     while (z->parent && z->parent->color == RED) {
3         if (z->parent == z->parent->parent->left) {
4             RBNode* uncle = z->parent->parent->right;
5
6             if (uncle && uncle->color == RED) {
7                 // Case 1: uncle is red -> recolor
8                 z->parent->color = BLACK;
9                 uncle->color = BLACK;
10                z->parent->parent->color = RED;
11                z = z->parent->parent; // move up
12            } else {
13                if (z == z->parent->right) {
14                    // Case 2: z is right child -> left rotate
15                    z = z->parent;
16                    leftRotate(root, z);
17                }
18                // Case 3: z is left child -> right rotate
19                z->parent->color = BLACK;
20                z->parent->parent->color = RED;
21                rightRotate(root, z->parent->parent);
22            }
23        } else {
24            // Symmetric: parent is right child of grandparent
25            RBNode* uncle = z->parent->parent->left;
26            if (uncle && uncle->color == RED) {
27                z->parent->color = BLACK;
28                uncle->color = BLACK;
29                z->parent->parent->color = RED;
30                z = z->parent->parent;
31            } else {
32                if (z == z->parent->left) {
33                    z = z->parent;
34                    rightRotate(root, z);
35                }
36                z->parent->color = BLACK;
37                z->parent->parent->color = RED;
38                leftRotate(root, z->parent->parent);
39            }
40        }
41    }
42    root->color = BLACK; // ensure root is black
43 }

```



```

44
45 void rbInsert(RBNode*& root, int key) {
46     RBNode* z = new RBNode(key);
47     // Standard BST insert
48     RBNode* y = nullptr;
49     RBNode* x = root;
50     while (x) {
51         y = x;
52         if (z->key < x->key) x = x->left;
53         else x = x->right;
54     }
55     z->parent = y;
56     if (!y) root = z;
57     else if (z->key < y->key) y->left = z;
58     else y->right = z;
59
60     z->left = nullptr;
61     z->right = nullptr;
62     z->color = RED;
63     rbInsertFixup(root, z);
64 }

```

Listing 32: RB-Tree Insertion with fixup

14.4 Deletion

RB-Tree deletion is the most complex operation. The key insight:

1. Perform standard BST deletion.
2. If the deleted/moved node was **black**, we may have violated properties 2, 4, or 5.
3. RB-DELETE-FIXUP restores properties using at most 3 rotations and $O(\lg n)$ recolorings.

The fixup has **four cases** (and symmetric mirrors), involving the **sibling** w of the “double-black” node x :

Case	Condition	Action
1	Sibling w is red	Recolor w and parent; rotate parent. Converts to case 2/3/4.
2	w is black with two black children	Recolor w red; move x up.
3	w is black; w 's near child is red	Recolor and rotate w . Converts to case 4.
4	w is black; w 's far child is red	Rotate parent; recolor. Done.

Insertion and Deletion Complexity

Operation	Rotations	Total Time
Insert	≤ 2	$O(\lg n)$
Delete	≤ 3	$O(\lg n)$
Search	0	$O(\lg n)$

At most a constant number of rotations per operation. The $O(\lg n)$ comes from traversing up the tree to fix violations.

Red-Black Trees in Practice

- C++ `std::set`, `std::map`, `std::multiset`, `std::multimap` are all implemented as Red-Black Trees.
- Java's `TreeMap` and `TreeSet` also use RB Trees.
- Linux kernel's `rbtree.h` is used for process scheduling, memory management, etc.
- In competitive programming, you rarely implement RB Trees from scratch — use STL containers or simpler balanced BSTs (treaps, AVL).

15 Augmenting Data Structures

This chapter shows how to extend existing data structures with additional information to support new operations efficiently.

15.1 The Methodology

Four Steps to Augment a Data Structure

1. **Choose an underlying data structure** (e.g., Red-Black Tree).
2. **Determine additional information** to store in each node.
3. **Verify** that the additional information can be maintained during normal operations (insert, delete, rotate) without changing their asymptotic cost.
4. **Develop new operations** that use the augmented information.

15.2 Order-Statistic Trees

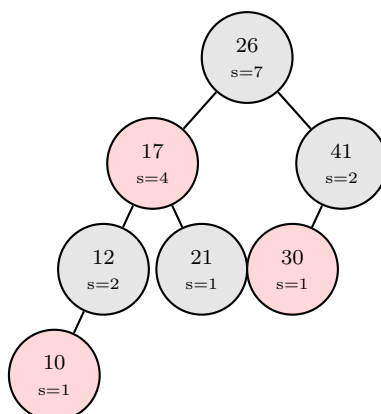
Order-Statistic Tree

An **order-statistic tree** is a Red-Black Tree where each node x stores an additional field $x.size = \text{number of nodes in the subtree rooted at } x \text{ (including } x \text{ itself)}$.

This supports two operations in $O(\lg n)$:

- **OS-Select**(x, i): find the i -th smallest element.

- $\text{OS-Rank}(T, x)$: find the rank (position in sorted order) of x .



15.2.1 OS-Select: Find i -th Smallest

Algorithm 19 OS-Select(x, i)

- 1: $r = x.\text{left.size} + 1$ $\triangleright$ rank of x within its subtree
 - 2: **if** $i = r$ **then return** x
 - 3: **else if** $i < r$ **then return** OS-Select($x.\text{left}, i$)
 - 4: **else return** OS-Select($x.\text{right}, i - r$)
 - 5: **end if**
-

```

1 struct OSNode {
2     int key, size;
3     bool color;
4     OSNode *left, *right, *parent;
5     OSNode(int k) : key(k), size(1), color(true),
6         left(nullptr), right(nullptr), parent(nullptr) {}
7 };
8
9 int getSize(OSNode* x) { return x ? x->size : 0; }
10
11 void updateSize(OSNode* x) {
12     if (x) x->size = 1 + getSize(x->left) + getSize(x->right);
13 }
14
15 // Returns node with i-th smallest key (1-indexed)
16 OSNode* osSelect(OSNode* x, int i) {
17     int r = getSize(x->left) + 1; // rank of x in subtree
18     if (i == r) return x;
19     if (i < r) return osSelect(x->left, i);
20     else return osSelect(x->right, i - r);
21 }
  
```

Listing 33: OS-Select: find i -th smallest element

15.2.2 OS-Rank: Find Rank of Node

Algorithm 20 OS-Rank(T, x)

```

1:  $r = x.\text{left.size} + 1$ 
2:  $y = x$ 
3: while  $y \neq T.\text{root}$  do
4:   if  $y = y.\text{parent.right}$  then
5:      $r = r + y.\text{parent.left.size} + 1$ 
6:   end if
7:    $y = y.\text{parent}$ 
8: end while
9: return  $r$ 

```

```

1 int osRank(OSNode* root, OSNode* x) {
2     int r = getSize(x->left) + 1;
3     OSNode* y = x;
4     while (y != root) {
5         if (y == y->parent->right)
6             r += getSize(y->parent->left) + 1;
7         y = y->parent;
8     }
9     return r;
10 }

```

Listing 34: OS-Rank: find rank of node x

15.2.3 Maintaining Size During Rotations

```

1 void leftRotateOS(OSNode*& root, OSNode* x) {
2     OSNode* y = x->right;
3     x->right = y->left;
4     if (y->left) y->left->parent = x;
5     y->parent = x->parent;
6     if (!x->parent) root = y;
7     else if (x == x->parent->left) x->parent->left = y;
8     else x->parent->right = y;
9     y->left = x;
10    x->parent = y;
11
12    // Update sizes (order matters: x first, then y)
13    updateSize(x);
14    updateSize(y);
15 }

```

Listing 35: Rotation with size update

Why Size is Easy to Maintain

After any rotation, only the two rotated nodes change their subtree composition. Update their sizes in $O(1)$. During insertion/deletion, update sizes along the path from the modified node to the root in $O(\lg n)$.

15.3 Using Policy-Based Trees in C++ (Competitive Programming)

The GNU C++ library provides an order-statistic tree out of the box:

```

1 #include <ext/pb_ds/assoc_container.hpp>
2 #include <ext/pb_ds/tree_policy.hpp>
3 using namespace __gnu_pbds;
4
5 typedef tree<int, null_type, less<int>, rb_tree_tag,
6             tree_order_statistics_node_update> ordered_set;
7
8 ordered_set os;
9 os.insert(10);
10 os.insert(20);
11 os.insert(30);
12 os.insert(15);
13
14 // find_by_order(k): returns iterator to k-th element (0-indexed)
15 cout << *os.find_by_order(0); // 10 (smallest)
16 cout << *os.find_by_order(2); // 20
17
18 // order_of_key(k): returns number of elements strictly less than k
19 cout << os.order_of_key(15); // 1 (only 10 is < 15)
20 cout << os.order_of_key(25); // 3 (10, 15, 20 are < 25)
21
22 os.erase(20);
23 cout << os.order_of_key(30); // 2 (10, 15 are < 30)

```

Listing 36: GNU policy-based order-statistic tree

Duplicate Keys

The `ordered_set` above does **not** support duplicate keys (it's a set, not a multiset). For duplicates, use `less_equal<int>` as the comparator, but beware: `erase()` and `find()` break.

Alternative: store pairs `pair<int, int>` where the second element is a unique ID (e.g., insertion order) to make all elements distinct.

15.4 Interval Trees

Interval Tree

An **interval tree** is a Red-Black Tree augmented to efficiently support operations on *intervals* $[low, high]$.

Each node stores an interval, and each node is augmented with:

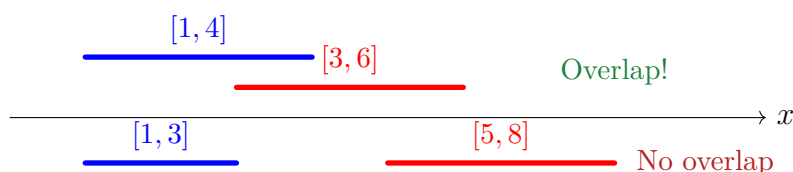
$$x.max = \max(\text{endpoints in subtree rooted at } x)$$

This supports **Interval-Search** in $O(\lg n)$: find any interval that overlaps a given query interval.

15.4.1 Overlap Condition

Two intervals $[a, b]$ and $[c, d]$ overlap iff $a \leq d$ **and** $c \leq b$.

Equivalently, they do *not* overlap iff $b < c$ or $d < a$.



Algorithm 21 Interval-Search(T, i)

```

1:  $x = T.root$ 
2: while  $x \neq NIL$  and  $i$  does not overlap  $x.int$  do
3:   if  $x.left \neq NIL$  and  $x.left.max \geq i.low$  then
4:      $x = x.left$ 
5:   else
6:      $x = x.right$ 
7:   end if
8: end while
9: return  $x$ 
```

```

1 struct Interval { int low, high; };
2
3 struct ITNode {
4     Interval intv;
5     int max_val; // max endpoint in subtree
6     bool color;
7     ITNode *left, *right, *parent;
8
9     ITNode(int lo, int hi) : intv{lo, hi}, max_val(hi), color(true),
10        left(nullptr), right(nullptr), parent(nullptr) {}
11 };
12
13 int getMax(ITNode* x) { return x ? x->max_val : INT_MIN; }
14
```

```

15 void updateMax(ITNode* x) {
16     if (x) x->max_val = max({x->intv.high,
17                             getMax(x->left),
18                             getMax(x->right)});
19 }
20
21 bool overlaps(Interval a, Interval b) {
22     return a.low <= b.high && b.low <= a.high;
23 }
24
25 // Find any interval overlapping query [lo, hi]
26 ITNode* intervalSearch(ITNode* root, int lo, int hi) {
27     Interval query = {lo, hi};
28     ITNode* x = root;
29     while (x != nullptr && !overlaps(x->intv, query)) {
30         if (x->left && x->left->max_val >= lo)
31             x = x->left;
32         else
33             x = x->right;
34     }
35     return x; // nullptr if no overlap found
36 }

```

Listing 37: Interval tree node and search

Why Interval Search Works

- If we go **left**: $x.\text{left}.\text{max} \geq i.\text{low}$, so there *might* be an overlapping interval in the left subtree. If not, then no interval in the right subtree can overlap either (by BST property on low endpoints).
- If we go **right**: no interval in the left subtree overlaps i (since all left subtree highs $< i.\text{low}$).

Either way, we make the correct choice, and the search takes $O(\lg n)$.

15.5 Augmentation Theorem

Theorem 15.1 (Augmentation of Red-Black Trees). Let f be an attribute of each node that depends only on the node itself and its two children (i.e., $x.f = g(x, x.\text{left}.f, x.\text{right}.f)$). Then f can be maintained during insertions and deletions *without* affecting the $O(\lg n)$ asymptotic performance.

This is why both **size** (order-statistic trees) and **max** (interval trees) work: they can be computed from children in $O(1)$, and updated after rotations in $O(1)$.

Summary of Augmented Trees

Tree Type	Augmented Field	New Operations	Time
Order-Statistic	size	Select, Rank	$O(\lg n)$
Interval Tree	max endpoint	Interval-Search	$O(\lg n)$

Any attribute computable from children in $O(1)$ can be augmented.

15.6 Red-Black Trees vs AVL Trees

Property	AVL Tree	Red-Black Tree
Balance condition	$ h_L - h_R \leq 1$	Color properties (5 rules)
Max height	$1.44 \lg(n + 1)$	$2 \lg(n + 1)$
Lookup	$O(\lg n)$ — faster	$O(\lg n)$ — slightly slower
Insertion	$O(\lg n)$, ≤ 2 rotations	$O(\lg n)$, ≤ 2 rotations
Deletion	$O(\lg n)$, $\leq O(\lg n)$ rotations	$O(\lg n)$, ≤ 3 rotations
Space overhead	1 extra int (height or BF)	1 extra bit (color)

When to Use Which

- **AVL trees** are *more strictly balanced* (height $\leq 1.44 \lg n$ vs $2 \lg n$). Use when **lookups dominate** (e.g., databases, dictionaries).
- **Red-Black trees** are *cheaper to maintain* during modifications — deletion needs at most 3 rotations vs $O(\lg n)$ for AVL. Use when **insertions/deletions are frequent** (e.g., `std::map`, Linux kernel).
- **C++ STL** (`std::set`, `std::map`) uses Red-Black trees.
- **Java TreeMap/TreeSet** also uses Red-Black trees.
- In **competitive programming**, you rarely implement either — use `std::set` or policy-based trees (`__gnu_pbds`).

Detailed Comparison

- 1. Strictness of Balancing:** AVL trees guarantee $|h_L - h_R| \leq 1$ at every node. Red-Black trees only guarantee that no root-to-leaf path is more than twice as long as any other. This makes AVL trees shorter (better for lookups) but harder to maintain.
- 2. Rotation Count on Deletion:** This is the key practical difference. AVL deletion may cascade rotations up the tree ($O(\lg n)$ rotations worst case). Red-Black deletion needs at most 3 rotations plus $O(\lg n)$ recolorings (which are cheaper than rotations).
- 3. Memory:** AVL stores a height or balance factor (int). Red-Black stores a color (1 bit). In practice, both are padded to the same size, so this rarely matters.
- 4. Theoretical Comparison:** For $n = 10^6$: AVL height ≤ 29 , RB height ≤ 40 . The 11-level difference means AVL does $\sim 28\%$ fewer comparisons per lookup on average.

16 Dynamic Programming

16.1 The DP Paradigm

Dynamic Programming

Dynamic Programming solves optimization problems by:

1. Breaking the problem into **overlapping subproblems**.
2. Solving each subproblem **once** and caching the result.
3. Combining solutions using **optimal substructure**.

When to Use DP

A problem is amenable to DP when it has:

1. **Optimal substructure:** An optimal solution contains optimal solutions to subproblems.
2. **Overlapping subproblems:** The same subproblems are solved repeatedly in a naive recursion.

If subproblems don't overlap, use divide-and-conquer instead.

16.2 Rod Cutting

Rod Cutting Problem

Given a rod of length n and a price table $p[1..n]$ where $p[i]$ is the price of a rod of length i , find the maximum revenue obtainable by cutting the rod into pieces.

Recurrence:

$$r(n) = \max_{1 \leq i \leq n} (p[i] + r(n - i)), \quad r(0) = 0$$

```

1 vector<int> memo;
2 int cutRod(vector<int>& price, int n) {
3     if (n == 0) return 0;
4     if (memo[n] != -1) return memo[n];
5     int best = INT_MIN;
6     for (int i = 1; i <= n; i++)
7         best = max(best, price[i] + cutRod(price, n - i));
8     return memo[n] = best;
9 }
```

Listing 38: Rod cutting — top-down with memoization

```

1 int cutRodBU(vector<int>& price, int n) {
2     vector<int> dp(n + 1, 0);
3     for (int j = 1; j <= n; j++)
4         for (int i = 1; i <= j; i++)
5             dp[j] = max(dp[j], price[i] + dp[j - i]);
6     return dp[n];
7 }
8 // Time: O(n^2), Space: O(n)
```

Listing 39: Rod cutting — bottom-up tabulation

16.3 Matrix Chain Multiplication

Matrix Chain Multiplication

Given matrices $A_1, A_2, \dots, A_n$ with dimensions $p_0 \times p_1, p_1 \times p_2, \dots, p_{n-1} \times p_n$, find the parenthesization that minimizes the total number of scalar multiplications.

State: $m[i][j]$ = minimum cost to multiply $A_i \cdots A_j$.

Recurrence:

$$m[i][j] = \min_{i \leq k < j} (m[i][k] + m[k+1][j] + p_{i-1} \cdot p_k \cdot p_j)$$

Base: $m[i][i] = 0$.

```

1 int matrixChain(vector<int>& p, int n) {
2     // p has n+1 elements: dimensions p[0] x p[1], ..., p[n-1] x p[n]
3     vector<vector<int>> m(n, vector<int>(n, 0));
4     // l = chain length
5     for (int l = 2; l <= n; l++) {
6         for (int i = 0; i < n - l + 1; i++) {
7             int j = i + l - 1;
8             m[i][j] = INT_MAX;
9             for (int k = i; k < j; k++) {
10                int cost = m[i][k] + m[k+1][j] + p[i]*p[k+1]*p[j+1];
11                m[i][j] = min(m[i][j], cost);
12            }
13        }
14    }
15    return m[0][n-1];
16 }
```

Listing 40: Matrix chain multiplication — $O(n^3)$

16.4 Longest Common Subsequence (LCS)

LCS Problem

Given two sequences $X = \langle x_1, \dots, x_m \rangle$ and $Y = \langle y_1, \dots, y_n \rangle$, find the longest subsequence common to both (not necessarily contiguous).

Recurrence:

$$c[i][j] = \begin{cases} 0 & \text{if } i = 0 \text{ or } j = 0 \\ c[i-1][j-1] + 1 & \text{if } x_i = y_j \\ \max(c[i-1][j], c[i][j-1]) & \text{if } x_i \neq y_j \end{cases}$$

```

1 int lcs(string& X, string& Y) {
2     int m = X.size(), n = Y.size();
3     vector<vector<int>> dp(m + 1, vector<int>(n + 1, 0));
4     for (int i = 1; i <= m; i++)
5         for (int j = 1; j <= n; j++) {
6             if (X[i-1] == Y[j-1])
7                 dp[i][j] = dp[i-1][j-1] + 1;
8             else
9                 dp[i][j] = max(dp[i-1][j], dp[i][j-1]);
10        }
11    return dp[m][n];
12 }

```

Listing 41: LCS — $O(mn)$ time and space

```

1 int lcsOpt(string& X, string& Y) {
2     int m = X.size(), n = Y.size();
3     vector<int> prev(n + 1, 0), curr(n + 1, 0);
4     for (int i = 1; i <= m; i++) {
5         for (int j = 1; j <= n; j++) {
6             if (X[i-1] == Y[j-1])
7                 curr[j] = prev[j-1] + 1;
8             else
9                 curr[j] = max(prev[j], curr[j-1]);
10        }
11        swap(prev, curr);
12        fill(curr.begin(), curr.end(), 0);
13    }
14    return prev[n];
15 }

```

Listing 42: LCS — space-optimized to $O(n)$

16.5 0/1 Knapsack

State: $dp[i][w]$ = max value using items 1.. i with capacity w .

$$dp[i][w] = \max(dp[i-1][w], dp[i-1][w - w_i] + v_i)$$

```

1 int knapsack(int n, int W, vector<int>& wt, vector<int>& val) {
2     vector<int> dp(W + 1, 0);
3     for (int i = 0; i < n; i++)
4         for (int w = W; w >= wt[i]; w--) // reverse order!
5             dp[w] = max(dp[w], dp[w - wt[i]] + val[i]);
6     return dp[W];
7 }

```

Listing 43: 0/1 Knapsack — $O(nW)$

Why Reverse Order?

Processing w from high to low ensures each item is used **at most once**. If we go low to high, an item can be picked multiple times (unbounded knapsack).

16.6 Top-Down vs Bottom-Up

	Top-Down (Memo)	Bottom-Up (Tab)
Style	Recursive	Iterative
Subproblems solved	Only needed ones	All of them
Stack overhead	Yes ($O(n)$ depth)	No
Ease of writing	Often easier	Need correct order
Space optimization	Harder	Often straightforward

17 Greedy Algorithms**17.1 The Greedy Paradigm****Greedy Algorithm**

A **greedy algorithm** makes the locally optimal choice at each step, hoping it leads to a globally optimal solution. Unlike DP, it never reconsiders past choices.

It works when the problem has:

1. **Greedy-choice property:** A globally optimal solution can be assembled from locally optimal choices.
2. **Optimal substructure:** Same as DP.

17.2 Activity Selection**Activity Selection Problem**

Given n activities with start/finish times $[s_i, f_i)$, find the maximum number of mutually compatible (non-overlapping) activities.

Greedy strategy: Always pick the activity that *finishes earliest* among remaining compatible activities.

```

1 int activitySelection(vector<pair<int,int>>& acts) {
2     // acts[i] = {finish, start}
3     sort(acts.begin(), acts.end()); // sort by finish time
4     int count = 1, lastFinish = acts[0].first;
5     for (int i = 1; i < (int)acts.size(); i++) {
6         if (acts[i].second >= lastFinish) { // compatible
7             count++;
8             lastFinish = acts[i].first;
9         }
10    }

```

```

11     return count;
12 }

```

Listing 44: Activity selection — $O(n \lg n)$

Proof of Correctness

Let A be the greedy solution and O be any optimal solution. If O doesn't include the activity a_1 with earliest finish, we can replace the first activity in O with a_1 without reducing the count (since a_1 finishes no later). Repeating this exchange argument shows $|A| = |O|$. $\square$

17.3 Huffman Codes

Huffman Coding

Given characters with frequencies, build a **prefix-free** binary code that minimizes the total encoded length. Frequent characters get shorter codes.

Algorithm: Repeatedly merge the two lowest-frequency nodes:

Algorithm 22 Huffman(C)

```

1:  $Q$  = min-priority queue of characters by frequency
2: for  $i = 1$  to  $|C| - 1$  do
3:    $z$  = new node
4:    $z$ .left =  $x$  = Extract-Min( $Q$ )
5:    $z$ .right =  $y$  = Extract-Min( $Q$ )
6:    $z$ .freq =  $x$ .freq +  $y$ .freq
7:   Insert( $Q, z$ )
8: end for
9: return Extract-Min( $Q$ )

```

▷ root of Huffman tree

```

1 struct HuffNode {
2     int freq;
3     char ch;
4     HuffNode *left, *right;
5     HuffNode(int f, char c) : freq(f), ch(c), left(nullptr), right(nullptr) {}
6 };
7
8 struct Cmp {
9     bool operator()(HuffNode* a, HuffNode* b) { return a->freq > b->freq; }
10 };
11
12 HuffNode* buildHuffman(vector<pair<char, int>>& chars) {
13     priority_queue<HuffNode*, vector<HuffNode*>, Cmp> pq;
14     for (auto& [c, f] : chars)
15         pq.push(new HuffNode(f, c));
16
17     while (pq.size() > 1) {

```

```

18     HuffNode* left = pq.top(); pq.pop();
19     HuffNode* right = pq.top(); pq.pop();
20     HuffNode* parent = new HuffNode(left->freq + right->freq, '\0');
21     parent->left = left;
22     parent->right = right;
23     pq.push(parent);
24 }
25 return pq.top();
26 }
27
28 void printCodes(HuffNode* root, string code) {
29     if (!root) return;
30     if (root->ch != '\0')
31         cout << root->ch << ": " << code << "\n";
32     printCodes(root->left, code + "0");
33     printCodes(root->right, code + "1");
34 }

```

Listing 45: Huffman coding — $O(n \lg n)$

17.4 Fractional Knapsack

Unlike 0/1 knapsack (DP), the **fractional** knapsack allows taking fractions of items. Greedy works: sort by value-per-weight, take greedily.

```

1 double fractionalKnapsack(int W, vector<pair<int,int>>& items) {
2     // items[i] = {value, weight}
3     // Sort by value/weight ratio (descending)
4     sort(items.begin(), items.end(), [](auto& a, auto& b) {
5         return (double)a.first / a.second > (double)b.first / b.second;
6     });
7     double totalValue = 0;
8     int remaining = W;
9     for (auto& [v, w] : items) {
10         if (w <= remaining) {
11             totalValue += v;
12             remaining -= w;
13         } else {
14             totalValue += v * ((double)remaining / w);
15             break;
16         }
17     }
18     return totalValue;
19 }

```

Listing 46: Fractional knapsack — $O(n \lg n)$

17.5 Greedy vs DP

	Greedy	DP
Choices	One best choice per step	All choices explored
Backtracking	Never	Implicit via subproblems
Proof required	Exchange argument	Induction on recurrence
Fractional Knapsack	Works	Overkill
0/1 Knapsack	Fails	Works
Activity Selection	Works	Also works (but slower)

Greedy Can Fail

Greedy does **not** always give optimal solutions. You must *prove* the greedy-choice property for each specific problem. Common proof techniques:

- **Exchange argument:** Show any optimal solution can be transformed to match the greedy solution without loss.
- **Staying ahead:** Show the greedy solution is “at least as good” at every step.

17.6 Matroid Theory (Optional)

Matroid

A **matroid** $M = (S, \mathcal{I})$ consists of a finite set S and a collection $\mathcal{I}$ of “independent” subsets satisfying:

1. $\emptyset \in \mathcal{I}$
2. If $A \in \mathcal{I}$ and $B \subseteq A$, then $B \in \mathcal{I}$ (hereditary)
3. If $A, B \in \mathcal{I}$ and $|A| < |B|$, then $\exists x \in B \setminus A$ such that $A \cup \{x\} \in \mathcal{I}$ (exchange property)

A greedy algorithm finds a maximum-weight independent set in any weighted matroid.

This generalizes why Kruskal’s algorithm works for MSTs: the set of forests forms a graphic matroid.

17.7 Correctness of Huffman’s Algorithm

Theorem 17.1 (Greedy-Choice Property of Huffman). Let x and y be the two characters with the *lowest* frequencies. Then there exists an optimal prefix code in which x and y have the same code length and differ only in the last bit (i.e., they are siblings in the coding tree).

Proof (Exchange Argument)

Let T be any optimal tree. Let a and b be two sibling leaves at the *deepest* level of T (such siblings must exist in a full binary tree).

Since a and b are at the deepest level, $d_T(a) = d_T(b) \geq d_T(c)$ for all other characters c .

Since x and y have the lowest frequencies: $f(x) \leq f(a)$ and $f(y) \leq f(b)$.

Exchange x with a : The change in cost is:

$$\Delta_1 = f(x) \cdot d_T(a) + f(a) \cdot d_T(x) - f(x) \cdot d_T(x) - f(a) \cdot d_T(a) = (f(a) - f(x))(d_T(x) - d_T(a)) \leq 0$$

because $f(a) - f(x) \geq 0$ and $d_T(x) - d_T(a) \leq 0$ (since a is at the deepest level). So cost does not increase.

Similarly exchange y with b : cost does not increase.

After both exchanges, x and y are siblings at the deepest level, and the tree is still optimal (cost didn't increase). $\square$

Theorem 17.2 (Optimal Substructure of Huffman). Let T be an optimal tree for alphabet C . Let x, y be sibling leaves. Let $C' = C \setminus \{x, y\} \cup \{z\}$ where z has frequency $f(z) = f(x) + f(y)$. Let T' be T with x, y replaced by z . Then T' is optimal for C' .

Proof

The cost of T relates to T' :

$$\begin{aligned} B(T) &= \sum_{c \in C} f(c) \cdot d_T(c) \\ &= \sum_{c \in C \setminus \{x, y\}} f(c) \cdot d_T(c) + f(x) \cdot d_T(x) + f(y) \cdot d_T(y) \\ &= \sum_{c \in C \setminus \{x, y\}} f(c) \cdot d_{T'}(c) + f(x)(d_{T'}(z) + 1) + f(y)(d_{T'}(z) + 1) \\ &= B(T') + f(x) + f(y) \end{aligned}$$

since $d_T(x) = d_T(y) = d_{T'}(z) + 1$ (one level deeper than z).

So $B(T) = B(T') + f(x) + f(y)$.

If T' were not optimal for C' , there would exist T'' with $B(T'') < B(T')$. Expanding z back into x, y gives a tree for C with cost $B(T'') + f(x) + f(y) < B(T') + f(x) + f(y) = B(T)$, contradicting the optimality of T . $\square$

Huffman Optimality — Putting It Together

By **induction on $|C|$** :

Base case ($|C| = 2$): Two characters, one gets code 0, the other 1. Clearly optimal.

Inductive step: Assume Huffman is optimal for alphabets of size $< |C|$.

1. By the greedy-choice property, an optimal tree exists where the two lowest-frequency characters x, y are siblings.
2. Replace x, y with z (frequency $f(x) + f(y)$). This gives an alphabet C' with $|C'| = |C| - 1$.
3. By the inductive hypothesis, Huffman produces an optimal tree T' for C' .
4. By optimal substructure, expanding z back to x, y gives an optimal tree for C .

This is exactly what the Huffman algorithm does! $\square$

18 Amortized Analysis

Amortized analysis determines the *average* cost per operation over a worst-case sequence of operations. Unlike average-case analysis, it makes **no probabilistic assumptions**.

18.1 Three Methods

Amortized Analysis Methods

1. **Aggregate method:** Compute total cost of n operations, divide by n .
2. **Accounting method:** Assign “amortized costs” to operations; overcharges build credit for future expensive operations.
3. **Potential method:** Define a potential function Φ on the data structure state. Amortized cost = actual cost + $\Delta\Phi$.

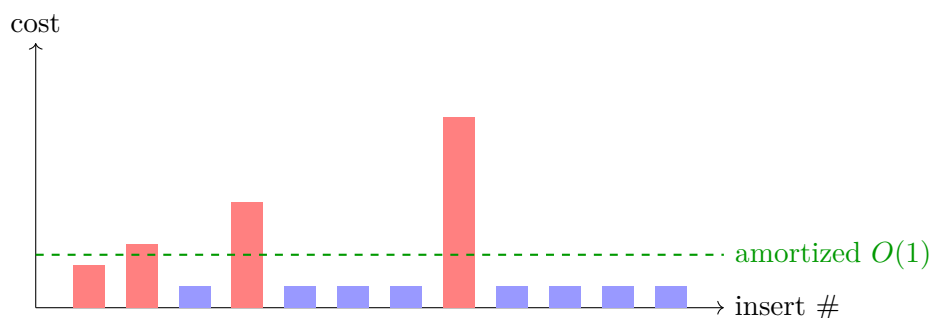
18.2 Example: Dynamic Table (Aggregate Method)

A dynamic array doubles its capacity when full. Inserting n elements:

- Most inserts cost $O(1)$.
- Occasionally, a resize copies all elements: cost $O(n)$.
- Resizes happen at inserts $1, 2, 4, 8, \dots, 2^k$.

Total cost: $n + (1 + 2 + 4 + \dots + 2^{\lfloor \lg n \rfloor}) \leq n + 2n = 3n$.

Amortized cost per insert: $3n/n = O(1)$.



18.3 Accounting Method

Charge each insert \$3 (amortized cost):

- \$1 pays for the actual insertion.
- \$2 is “saved” as credit on the element.
- When a resize doubles from m to $2m$, the $m/2$ new elements since the last resize each have \$2 credit = \$ m total, which pays for copying all m elements.

Credit never goes negative $\Rightarrow$ amortized cost of $O(1)$ per insert is valid.

18.4 Potential Method

Potential Method

Define a **potential function** $\Phi(D_i)$ on the state of the data structure after the i -th operation. Then:

$$\hat{c}_i = c_i + \Phi(D_i) - \Phi(D_{i-1})$$

where $\hat{c}_i$ is the amortized cost and c_i is the actual cost. Total amortized cost:

$$\sum_{i=1}^n \hat{c}_i = \sum_{i=1}^n c_i + \Phi(D_n) - \Phi(D_0)$$

If $\Phi(D_n) \geq \Phi(D_0)$, total amortized cost is an upper bound on total actual cost.

Dynamic table: Let $\Phi = 2 \cdot \text{num} - \text{size}$ where num = number of elements, size = table capacity.

- **Normal insert (no resize):** $c_i = 1$, $\Delta\Phi = 2$. $\hat{c}_i = 1 + 2 = 3$.
- **Insert with resize:** $c_i = \text{num} + 1$ (copy + insert), $\Delta\Phi = 2 - \text{num}$ (size doubles).
 $\hat{c}_i = (\text{num} + 1) + (2 - \text{num}) = 3$.

Both cases give amortized cost $3 = O(1)$.

```

1 struct DynArray {
2     int* data;
3     int size, cap;
4
5     DynArray() : data(new int[1]), size(0), cap(1) {}
6
7     void push_back(int x) {
8         if (size == cap) {
9             cap *= 2;                // double capacity
10            int* newData = new int[cap];
11            for (int i = 0; i < size; i++) // copy O(size)
12                newData[i] = data[i];
13            delete[] data;
14            data = newData;
15        }
16        data[size++] = x;              // O(1)
17    }
18    // Amortized O(1) per push_back
19 };

```

Listing 47: Dynamic array (std::vector behavior)

Key Takeaway

`std::vector::push_back` is amortized $O(1)$ because of the doubling strategy. This is why vectors are so efficient despite occasional reallocations. The same analysis applies to hash table resizing.

19 B-Trees

B-Trees are balanced search trees designed for **disk-based** storage systems where the cost of accessing a node (disk I/O) far exceeds in-memory computation. They minimize disk accesses by packing many keys into each node.

19.1 Motivation: Why Not Binary Trees?

Disk I/O is Expensive

- A disk read takes $\sim 5\text{--}10\text{ms}$. CPU operation takes $\sim 1\text{ns}$. Ratio: $\sim 10^7$.
- Each disk read fetches a *page* (typically 4KB–16KB).
- Binary trees have height $O(\lg n)$ — each level is one disk access.
- A B-Tree with branching factor ~ 1000 has height $\log_{1000} n$. For $n = 10^9$: binary tree needs ~ 30 reads; B-Tree needs ~ 3 reads.

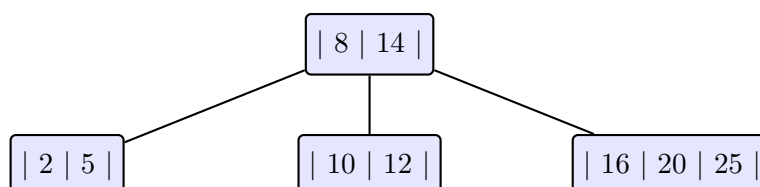
19.2 B-Tree Properties

B-Tree of Minimum Degree t

Every node x satisfies:

1. $x.n$ = number of keys stored in x .
2. Keys are stored in non-decreasing order: $x.\text{key}_1 \leq x.\text{key}_2 \leq \dots$
3. Each internal node has $x.n + 1$ children.
4. Keys separate the ranges of children subtrees.
5. All leaves are at the **same depth**.
6. **Bounds on keys:** every non-root node has $t-1 \leq x.n \leq 2t-1$ keys. The root has $1 \leq x.n \leq 2t-1$ keys.

Property	Value
Min keys per node (non-root)	$t - 1$
Max keys per node	$2t - 1$
Min children (internal, non-root)	t
Max children	$2t$
Height for n keys	$\leq \log_t \frac{n+1}{2}$



19.3 B-Tree Search

At each node, perform a linear (or binary) search among the keys, then recurse into the appropriate child. Each node access is one disk read.

```

1 struct BTreeNode {
2     int n;           // number of keys
3     int keys[2*T - 1]; // keys
4     BTreeNode* children[2*T];
5     bool leaf;
6 };
7
8 BTreeNode* search(BTreeNode* x, int k) {
9     int i = 0;
10    while (i < x->n && k > x->keys[i])
11        i++;
12    if (i < x->n && k == x->keys[i])
13        return x; // found
14    if (x->leaf)
15        return nullptr; // not found
16    // DISK-READ(x->children[i]) <-- in real implementations
17    return search(x->children[i], k);
18 }

```

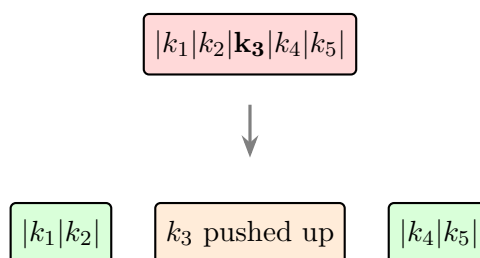
Listing 48: B-Tree search — $O(t \log_t n)$ CPU; $O(\log_t n)$ disk reads

19.4 B-Tree Insertion

Keys are always inserted into **leaf** nodes. To prevent overflow (a node with $2t$ keys), we **split full nodes proactively** on the way down.

Splitting a full node: When a node has $2t-1$ keys, split it into two nodes of $t-1$ keys each, pushing the median key up to the parent.

Full node ($2t-1 = 5$ keys)



```

1 void splitChild(BTreeNode* x, int i) {
2     BTreeNode* y = x->children[i]; // full child
3     BTreeNode* z = new BTreeNode();
4     z->leaf = y->leaf;
5     z->n = T - 1;
6     // Copy upper half of y's keys to z
7     for (int j = 0; j < T - 1; j++)
8         z->keys[j] = y->keys[j + T];
9     if (!y->leaf)
10        for (int j = 0; j < T; j++)
11            z->children[j] = y->children[j + T];
12    y->n = T - 1;

```

```

13 // Shift x's children and keys to make room
14 for (int j = x->n; j > i; j--)
15     x->children[j + 1] = x->children[j];
16 x->children[i + 1] = z;
17 for (int j = x->n - 1; j >= i; j--)
18     x->keys[j + 1] = x->keys[j];
19 x->keys[i] = y->keys[T - 1]; // median goes up
20 x->n++;
21 }

```

Listing 49: B-Tree split child

19.5 B-Tree Deletion

Deletion is more complex. The key idea: ensure that any node we descend into has at least t keys (so we can delete without underflow). Three cases:

1. **Key in leaf:** Delete directly.
2. **Key in internal node:** Replace with predecessor/successor from a child with $\geq t$ keys, or merge children.
3. **Key not in node:** Ensure the child to descend into has $\geq t$ keys by borrowing from a sibling or merging.

B-Tree Complexity Summary

Operation	Disk Reads	CPU Time
Search	$O(\log_t n)$	$O(t \log_t n)$
Insert	$O(\log_t n)$	$O(t \log_t n)$
Delete	$O(\log_t n)$	$O(t \log_t n)$

With $t = 1000$ and $n = 10^9$: height ≤ 3 . Only 3 disk reads per operation!

B-Trees in Practice

- **Databases:** MySQL (InnoDB), PostgreSQL, SQLite all use B+ Trees (variant where all data is in leaves and leaves are linked).
- **Filesystems:** NTFS, ext4, Btrfs, HFS+ use B-Trees.
- **B+ Tree** differs: internal nodes store only keys (no data), leaves are linked for range queries.

20 Fibonacci Heaps

Fibonacci heaps achieve excellent *amortized* bounds for priority queue operations, making them theoretically optimal for algorithms like Dijkstra and Prim.

20.1 Motivation

Operation	Binary Heap	Binomial Heap	Fibonacci Heap
Insert	$O(\lg n)$	$O(\lg n)$	$O(1)$ amortized
Find-Min	$O(1)$	$O(1)$	$O(1)$
Extract-Min	$O(\lg n)$	$O(\lg n)$	$O(\lg n)$ amortized
Decrease-Key	$O(\lg n)$	$O(\lg n)$	$O(1)$ amortized
Delete	$O(\lg n)$	$O(\lg n)$	$O(\lg n)$ amortized
Merge/Union	$O(n)$	$O(\lg n)$	$O(1)$

The key advantage: $O(1)$ amortized **Decrease-Key**. This makes Dijkstra's algorithm run in $O(V \lg V + E)$ instead of $O((V + E) \lg V)$.

20.2 Structure

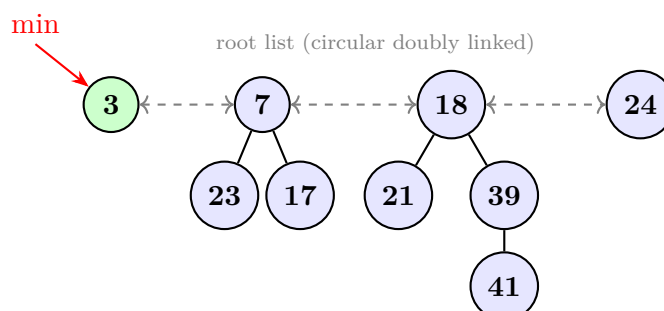
Fibonacci Heap

A Fibonacci heap is a collection of **min-heap-ordered trees**. Unlike binomial heaps, the trees have no prescribed shape.

Each node stores:

- **key**: the priority value.
- **degree**: number of children.
- **mark**: whether the node has lost a child since becoming a child itself.
- **parent, child**: pointers.
- **left, right**: circular doubly linked list of siblings.

The heap maintains a pointer **min** to the node with the smallest key.



20.3 Key Operations

20.3.1 Insert — $O(1)$

Create a new single-node tree and add it to the root list. Update **min** if necessary. No restructuring.

20.3.2 Find-Min — $O(1)$

Simply return **min**.

20.3.3 Union/Merge — $O(1)$

Concatenate the two root lists (circular linked list merge). Update `min`.

20.3.4 Extract-Min — $O(\lg n)$ amortized

1. Remove `min` from the root list.
2. Add all children of `min` to the root list.
3. **Consolidate:** merge trees with the same degree until all root trees have distinct degrees (like counting in binary).
4. Update `min` by scanning the root list.

The consolidation step is where the “lazy” restructuring happens. After consolidation, the number of roots is at most $O(\lg n)$.

20.3.5 Decrease-Key — $O(1)$ amortized

1. Decrease the key of node x .
2. If heap order is violated ($x.\text{key} < x.\text{parent}.\text{key}$):
 - (a) **Cut** x from its parent, add to root list.
 - (b) **Cascading cut:** if parent is marked, cut it too (recursively). If parent is unmarked, mark it.
3. Update `min` if necessary.

Why Cascading Cuts?

The cascading cut mechanism ensures that no node loses more than one child without itself being cut to the root list. This maintains the property that a node of degree k has $\geq F_{k+2} \geq \phi^k$ descendants (where $\phi = (1 + \sqrt{5})/2$), which bounds the maximum degree to $O(\lg n)$.

20.4 Why “Fibonacci”?

The name comes from the key lemma: a node of degree k in a Fibonacci heap has at least F_{k+2} descendants, where F_k is the k -th Fibonacci number. Since $F_k = \Theta(\phi^k)$, the maximum degree is $O(\log_\phi n) = O(\lg n)$.

Practical Considerations

Despite their theoretical superiority, Fibonacci heaps are **rarely used in practice** because:

- Complex implementation with high constant factors.
- Poor cache performance (pointer-heavy structure).
- Binary heaps with decrease-key in $O(\lg n)$ are fast enough for most applications.
- Pairing heaps offer similar amortized bounds with simpler implementation.

For competitive programming, use `priority_queue` (binary heap) or `set` (RB-tree with $O(\lg n)$ decrease-key via erase+insert).

21 van Emde Boas Trees

van Emde Boas (vEB) trees achieve $O(\lg \lg u)$ time for all operations on integers from a bounded universe $\{0, 1, \dots, u-1\}$.

21.1 Motivation

Operation	BST / RB-Tree	Hash Table	vEB Tree
Search	$O(\lg n)$	$O(1)$ avg	$O(\lg \lg u)$
Insert	$O(\lg n)$	$O(1)$ avg	$O(\lg \lg u)$
Delete	$O(\lg n)$	$O(1)$ avg	$O(\lg \lg u)$
Successor	$O(\lg n)$	$O(n)$	$O(\lg \lg u)$
Predecessor	$O(\lg n)$	$O(n)$	$O(\lg \lg u)$
Min / Max	$O(\lg n)$	$O(n)$	$O(1)$

The key advantage: **successor/predecessor** in $O(\lg \lg u)$, which is impossible with hash tables and slower with BSTs when n is large.

For $u = 2^{32}$: $\lg \lg u = \lg 32 = 5$. Near-constant time!

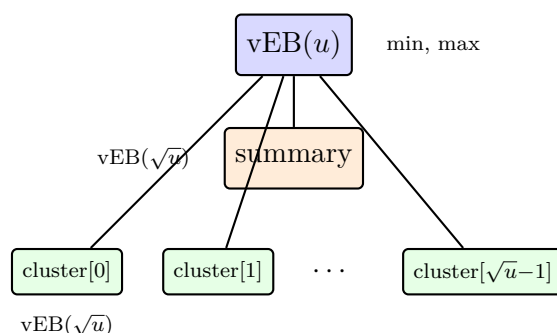
21.2 Key Idea: Recursive Universe Splitting

vEB Tree Structure

A vEB tree for universe $\{0, \dots, u-1\}$ (where $u = 2^k$) contains:

- $\sqrt{u}$ **clusters**, each a vEB tree of size $\sqrt{u}$.
- A **summary** structure: a vEB tree of size $\sqrt{u}$ tracking which clusters are non-empty.
- **min** and **max**: stored separately (not in any cluster).

Element x maps to: cluster $\lfloor x/\sqrt{u} \rfloor$, position $x \bmod \sqrt{u}$.



21.2.1 Index Decomposition

$$\text{high}(x) = \lfloor x/\sqrt{u} \rfloor \quad (\text{cluster number})$$

$$\text{low}(x) = x \bmod \sqrt{u} \quad (\text{position within cluster})$$

$$\text{index}(h, l) = h \cdot \sqrt{u} + l \quad (\text{reconstruct } x)$$

21.3 Operations

21.3.1 Successor — $O(\lg \lg u)$

Algorithm 23 vEB-Successor(V, x)

```

1: if  $V.\text{min} \neq \text{NIL}$  and  $x < V.\text{min}$  then
2:   return  $V.\text{min}$ 
3: end if
4:  $h = \text{high}(x)$ ,  $l = \text{low}(x)$ 
5: if  $V.\text{cluster}[h].\text{max} \neq \text{NIL}$  and  $l < V.\text{cluster}[h].\text{max}$  then
6:    $l' = \text{vEB-Successor}(V.\text{cluster}[h], l)$  ▷ same cluster
7:   return  $\text{index}(h, l')$ 
8: else
9:    $h' = \text{vEB-Successor}(V.\text{summary}, h)$  ▷ next non-empty cluster
10:  return  $\text{index}(h', V.\text{cluster}[h'].\text{min})$ 
11: end if

```

Recurrence: $T(u) = T(\sqrt{u}) + O(1)$. Let $u = 2^k$, then $T(2^k) = T(2^{k/2}) + O(1)$. Substituting $S(k) = T(2^k)$: $S(k) = S(k/2) + O(1) = O(\lg k) = O(\lg \lg u)$.

21.3.2 Insert — $O(\lg \lg u)$

1. If tree is empty, set $\text{min} = \text{max} = x$. Done.
2. If $x < \text{min}$, swap x and min (new min is stored separately).
3. Insert x into $\text{cluster}[\text{high}(x)]$. If that cluster was empty, also insert $\text{high}(x)$ into summary .
4. Update max if needed.

Key insight: storing min separately means we insert into at most *one* recursive structure (either cluster or summary, not both), giving $T(\sqrt{u}) + O(1)$.

Space and Practicality

- **Space:** $O(u)$ — proportional to the universe, not the number of elements. This limits practical use to $u \leq 2^{20}$ or so.
- For competitive programming with $u \leq 10^6$, a vEB tree can give $O(\lg \lg u)$ successor queries, but a balanced BST or `std::set` is usually sufficient.
- X-fast tries and Y-fast tries achieve similar bounds with $O(n)$ space.

22 Data Structures for Disjoint Sets

The **Union-Find** (disjoint set) data structure is one of the most practically important structures in competitive programming and graph algorithms.

22.1 The Disjoint-Set Problem

Disjoint-Set Operations

Maintain a collection of disjoint sets $\{S_1, S_2, \dots\}$. Each set has a **representative** element. Support:

- **Make-Set**(x): create a new set $\{x\}$ with representative x .
- **Find-Set**(x): return the representative of the set containing x .
- **Union**(x, y): merge the sets containing x and y .

Applications:

- **Kruskal's MST**: check if an edge creates a cycle.
- **Connected components**: dynamic connectivity.
- **Equivalence classes**: merging equivalent items.
- **Percolation, image processing**: region merging.

22.2 Linked-List Representation

Each set is a linked list. The first element is the representative.

- **Make-Set**: $O(1)$.
- **Find-Set**: $O(1)$ (each node points to head).
- **Union**: $O(n)$ worst case (append one list to another, update all pointers).

With the **weighted-union heuristic** (always append the shorter list to the longer one), a sequence of m operations on n elements takes $O(m + n \lg n)$.

22.3 Forest Representation (Optimal)

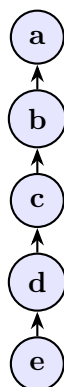
Each set is a **rooted tree**. The root is the representative. Each node stores a **parent** pointer (root points to itself).

Two Key Optimizations

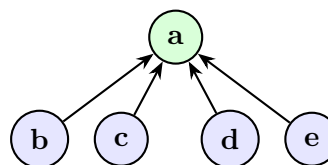
1. **Union by Rank**: Always attach the shorter tree under the root of the taller tree. Keeps trees balanced.
2. **Path Compression**: During **Find**, make every node on the path point directly to the root. Flattens the tree.

Together, these give nearly $O(1)$ amortized time per operation.

Before Find(e):



After Find(e):



```

1 struct DSU {
2     vector<int> parent, rank_;
3
4     DSU(int n) : parent(n), rank_(n, 0) {
5         iota(parent.begin(), parent.end(), 0); // parent[i] = i
6     }
7
8     int find(int x) {
9         if (parent[x] != x)
10             parent[x] = find(parent[x]); // path compression
11         return parent[x];
12     }
13
14     bool unite(int x, int y) {
15         x = find(x); y = find(y);
16         if (x == y) return false; // already same set
17
18         // Union by rank
19         if (rank_[x] < rank_[y]) swap(x, y);
20         parent[y] = x;
21         if (rank_[x] == rank_[y]) rank_[x]++;
22         return true;
23     }
24
25     bool connected(int x, int y) {
26         return find(x) == find(y);
27     }
28 };

```

Listing 50: Union-Find with path compression and union by rank

22.4 Complexity Analysis

Theorem 22.1 (Union-Find Complexity). A sequence of m Make-Set, Union, and Find-Set operations on n elements, with both union by rank and path compression, runs in $O(m \cdot \alpha(n))$ time, where α is the **inverse Ackermann function**.

Inverse Ackermann Function $\alpha(n)$

$\alpha(n)$ grows so slowly that it is ≤ 4 for all practical values of n (up to $n = 2^{2^{65536}}$).
For all intents and purposes:

$$\alpha(n) \leq 4 \quad \text{for any } n \text{ that fits in the observable universe}$$

So Union-Find is effectively **amortized** $O(1)$ per operation.

The proof uses a clever potential function based on the rank structure. The Ackermann function grows so fast that its inverse is nearly constant.

22.5 Union by Size (Alternative)

Instead of rank, track the **size** of each set. Always attach smaller to larger. Same asymptotic bounds, but also lets you query set sizes:

```

1 struct DSUSize {
2     vector<int> parent, size_;
3
4     DSUSize(int n) : parent(n), size_(n, 1) {
5         iota(parent.begin(), parent.end(), 0);
6     }
7
8     int find(int x) {
9         if (parent[x] != x)
10             parent[x] = find(parent[x]);
11         return parent[x];
12     }
13
14     bool unite(int x, int y) {
15         x = find(x); y = find(y);
16         if (x == y) return false;
17         if (size_[x] < size_[y]) swap(x, y);
18         parent[y] = x;
19         size_[x] += size_[y];
20         return true;
21     }
22
23     int getSize(int x) { return size_[find(x)]; }
24 };

```

Listing 51: Union-Find with union by size

22.6 Application: Kruskal's MST

```

1 struct Edge { int u, v, w; };
2
3 long long kruskal(int n, vector<Edge>& edges) {
4     sort(edges.begin(), edges.end(),

```

```

5      [](auto& a, auto& b) { return a.w < b.w; });
6
7      DSU dsu(n);
8      long long mstWeight = 0;
9      int edgesUsed = 0;
10
11     for (auto& [u, v, w] : edges) {
12         if (dsu.unite(u, v)) {
13             mstWeight += w;
14             edgesUsed++;
15             if (edgesUsed == n - 1) break;
16         }
17     }
18     return mstWeight; // -1 if edgesUsed < n-1 (not connected)
19 }

```

Listing 52: Kruskal's algorithm using Union-Find — $O(E \lg E)$

22.7 Common Competitive Programming Patterns

```

1 int countComponents(int n, vector<pair<int,int>>& edges) {
2     DSU dsu(n);
3     for (auto& [u, v] : edges)
4         dsu.unite(u, v);
5     set<int> roots;
6     for (int i = 0; i < n; i++)
7         roots.insert(dsu.find(i));
8     return roots.size();
9 }

```

Listing 53: Counting connected components

```

1 bool hasCycle(int n, vector<pair<int,int>>& edges) {
2     DSU dsu(n);
3     for (auto& [u, v] : edges)
4         if (!dsu.unite(u, v))
5             return true; // u and v already connected -> cycle!
6     return false;
7 }

```

Listing 54: Detecting cycle in undirected graph

Summary

Implementation	Find	Union	Total (m ops)
Naive (no optimizations)	$O(n)$	$O(n)$	$O(mn)$
Union by rank only	$O(\lg n)$	$O(\lg n)$	$O(m \lg n)$
Path compression only	amortized $O(\lg n)$	—	$O(m \lg n)$
Both combined	amortized $O(\alpha(n))$	—	$O(m \cdot \alpha(n))$

Always use both optimizations together. The implementation is simple and the

performance is practically $O(1)$.

Theorem 22.2 (Correctness of Union-Find). $\text{Find}(x)$ returns the representative of x 's set, and $\text{Union}(x, y)$ correctly merges two sets, even with path compression and union by rank.

Proof

Find correctness (path compression): By induction on tree depth. A root r has $\text{parent}[r] = r$, so $\text{Find}(r) = r$. ✓

For non-root x : Find recurses on $\text{parent}[x]$, eventually reaching the root r . Path compression sets $\text{parent}[x] = r$. This doesn't change which set x belongs to — it only flattens the tree. All nodes still reach the same root. ✓

Union correctness (union by rank): $\text{Union}(x, y)$ first finds representatives r_x, r_y . If $r_x = r_y$, they're already in the same set. Otherwise, one root becomes the child of the other. All elements previously pointing to r_y now reach r_x via the new edge. The merged tree correctly represents the union. ✓

Union by rank preserves balance: A tree of rank k has $\geq 2^k$ nodes (provable by induction: merging two rank- k trees gives rank- $(k+1)$ with $\geq 2^k + 2^k = 2^{k+1}$ nodes). So $\text{rank} \leq \lg n$, bounding tree height without path compression. □

23 Elementary Graph Algorithms

23.1 Graph Representations

Graph

A **graph** consists of vertices V and edges $E \subseteq V \times V$.

- **Directed:** edges (u, v) have direction.
- **Undirected:** edges $\{u, v\}$ are symmetric.
- **Weighted:** each edge has a weight $w(u, v)$.

23.1.1 Adjacency List vs Adjacency Matrix

	Adjacency List	Adjacency Matrix
Space	$O(V + E)$	$O(V^2)$
Check edge (u, v)	$O(\deg(u))$	$O(1)$
Iterate neighbors	$O(\deg(u))$	$O(V)$
Add edge	$O(1)$	$O(1)$
Best for	Sparse graphs	Dense graphs

```

1 // Adjacency list (most common)
2 int n, m;
3 vector<vector<int>> adj(n); // unweighted
4 vector<vector<pair<int,int>>> adj(n); // weighted: {neighbor, weight}
5
6 void addEdge(int u, int v, int w = 1) {
7     adj[u].push_back({v, w});
8     adj[v].push_back({u, w}); // for undirected
9 }
10
11 // Adjacency matrix
12 vector<vector<int>> mat(n, vector<int>(n, 0));
13 mat[u][v] = 1; // or weight
14
15 // Edge list (useful for Kruskal)
16 struct Edge { int u, v, w; };
17 vector<Edge> edges;

```

Listing 55: Graph representations

23.2 Breadth-First Search (BFS)

BFS

BFS explores a graph level by level from a source s . It discovers all vertices at distance k before any at distance $k + 1$.

Computes: shortest distances in *unweighted* graphs.

Algorithm 24 BFS(G, s)

```

1: for each vertex  $u \in V \setminus \{s\}$  do
2:      $u.color = WHITE, u.d = \infty, u.\pi = NIL$ 
3: end for
4:  $s.color = GRAY, s.d = 0, s.\pi = NIL$ 
5:  $Q = \text{empty queue; Enqueue}(Q, s)$ 
6: while  $Q$  is not empty do
7:      $u = \text{Dequeue}(Q)$ 
8:     for each  $v \in \text{Adj}[u]$  do
9:         if  $v.color = WHITE$  then
10:             $v.color = GRAY, v.d = u.d + 1, v.\pi = u$ 
11:             $\text{Enqueue}(Q, v)$ 
12:        end if
13:    end for
14:     $u.color = BLACK$ 
15: end while

```

```

1 vector<int> bfs(int src, vector<vector<int>>& adj, int n) {
2     vector<int> dist(n, -1);

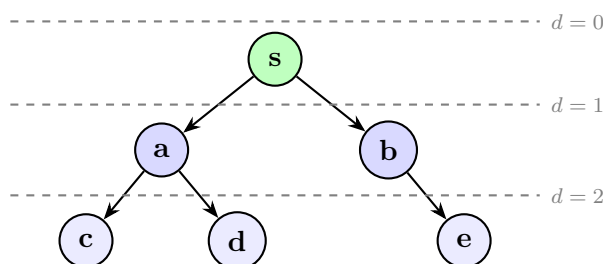
```

```

3  queue<int> q;
4  dist[src] = 0;
5  q.push(src);
6  while (!q.empty()) {
7      int u = q.front(); q.pop();
8      for (int v : adj[u]) {
9          if (dist[v] == -1) {
10             dist[v] = dist[u] + 1;
11             q.push(v);
12         }
13     }
14 }
15 return dist;
16 }

17
18 // Print shortest path from src to target
19 void printPath(vector<int>& parent, int target) {
20     if (target == -1) return;
21     printPath(parent, parent[target]);
22     cout << target << " ";
23 }

```

Listing 56: BFS — $O(V + E)$ 

BFS Properties

- **Time:** $O(V + E)$.
- Computes **shortest path distances** in unweighted graphs.
- The BFS tree gives a shortest-path tree from s .
- Works for both directed and undirected graphs.

23.3 Depth-First Search (DFS)

DFS

DFS explores as deep as possible before backtracking. It timestamps each vertex with *discovery* ($u.d$) and *finish* ($u.f$) times.

```

1  vector<bool> visited;
2  vector<int> disc, fin;
3  int timer_ = 0;

```



```

4
5 void dfs(int u, vector<vector<int>>& adj) {
6     visited[u] = true;
7     disc[u] = timer_++;
8     for (int v : adj[u]) {
9         if (!visited[v])
10             dfs(v, adj);
11     }
12     fin[u] = timer_++;
13 }
14
15 // Launch DFS from all unvisited vertices
16 void dfsAll(vector<vector<int>>& adj, int n) {
17     visited.assign(n, false);
18     disc.resize(n); fin.resize(n);
19     timer_ = 0;
20     for (int u = 0; u < n; u++)
21         if (!visited[u])
22             dfs(u, adj);
23 }

```

Listing 57: DFS — $O(V + E)$

23.3.1 Edge Classification

DFS classifies edges based on discovery/finish times:

Type	Condition	Meaning
Tree edge	v is WHITE when explored	Part of DFS tree
Back edge	v is GRAY (ancestor)	Indicates a cycle
Forward edge	v is BLACK, $u.d < v.d$	Shortcut to descendant
Cross edge	v is BLACK, $u.d > v.d$	Between unrelated branches

Cycle Detection

A directed graph has a **cycle** if and only if DFS finds a **back edge**. An undirected graph has a cycle if DFS visits an already-visited non-parent vertex.

```

1 enum Color { WHITE, GRAY, BLACK };
2 vector<Color> color;
3 bool hasCycleDFS(int u, vector<vector<int>>& adj) {
4     color[u] = GRAY;
5     for (int v : adj[u]) {
6         if (color[v] == GRAY) return true; // back edge = cycle
7         if (color[v] == WHITE && hasCycleDFS(v, adj))
8             return true;
9     }
10    color[u] = BLACK;
11    return false;

```

12 }

Listing 58: Cycle detection in directed graph

23.4 Topological Sort

Topological Sort

A **topological ordering** of a DAG (directed acyclic graph) is a linear ordering of vertices such that for every edge (u, v) , u appears before v . Only DAGs have topological orderings.

Method 1: DFS-based — output vertices in reverse finish-time order.

```

1 vector<int> topoOrder;
2 vector<bool> visited;
3
4 void topoSort(int u, vector<vector<int>>& adj) {
5     visited[u] = true;
6     for (int v : adj[u])
7         if (!visited[v])
8             topoSort(v, adj);
9     topoOrder.push_back(u); // add after all descendants
10 }
11
12 vector<int> topologicalSort(vector<vector<int>>& adj, int n) {
13     visited.assign(n, false);
14     topoOrder.clear();
15     for (int i = 0; i < n; i++)
16         if (!visited[i])
17             topoSort(i, adj);
18     reverse(topoOrder.begin(), topoOrder.end());
19     return topoOrder;
20 }
```

Listing 59: Topological sort via DFS — $O(V + E)$

Method 2: Kahn's algorithm (BFS-based, using in-degrees):

```

1 vector<int> kahnpTopoSort(vector<vector<int>>& adj, int n) {
2     vector<int> indeg(n, 0);
3     for (int u = 0; u < n; u++)
4         for (int v : adj[u]) indeg[v]++;
5
6     queue<int> q;
7     for (int u = 0; u < n; u++)
8         if (indeg[u] == 0) q.push(u);
9
10    vector<int> order;
11    while (!q.empty()) {
12        int u = q.front(); q.pop();
13        order.push_back(u);
14        for (int v : adj[u])
```

```

15         if (--indeg[v] == 0) q.push(v);
16     }
17     // If order.size() < n, graph has a cycle (not a DAG)
18     return order;
19 }

```

Listing 60: Kahn's algorithm (BFS topological sort)

23.5 Strongly Connected Components (SCCs)

Strongly Connected Components

A **SCC** of a directed graph is a maximal set of vertices such that every vertex is reachable from every other. The **component graph** (DAG of SCCs) is always a DAG.

Kosaraju's algorithm finds all SCCs in $O(V + E)$:

```

1 void dfs1(int u, vector<vector<int>>& adj, vector<bool>& vis,
2         vector<int>& order) {
3     vis[u] = true;
4     for (int v : adj[u])
5         if (!vis[v]) dfs1(v, adj, vis, order);
6     order.push_back(u); // finish order
7 }
8
9 void dfs2(int u, vector<vector<int>>& radj, vector<bool>& vis,
10         vector<int>& comp) {
11     vis[u] = true;
12     comp.push_back(u);
13     for (int v : radj[u])
14         if (!vis[v]) dfs2(v, radj, vis, comp);
15 }
16
17 vector<vector<int>> kosaraju(vector<vector<int>>& adj, int n) {
18     // Step 1: DFS on original graph, record finish order
19     vector<bool> vis(n, false);
20     vector<int> order;
21     for (int i = 0; i < n; i++)
22         if (!vis[i]) dfs1(i, adj, vis, order);
23
24     // Step 2: Build reverse graph
25     vector<vector<int>> radj(n);
26     for (int u = 0; u < n; u++)
27         for (int v : adj[u]) radj[v].push_back(u);
28
29     // Step 3: DFS on reverse graph in reverse finish order
30     fill(vis.begin(), vis.end(), false);
31     vector<vector<int>> sccs;
32     for (int i = n - 1; i >= 0; i--) {
33         int u = order[i];

```

```

34     if (!vis[u]) {
35         vector<int> comp;
36         dfs2(u, radj, vis, comp);
37         sccs.push_back(comp);
38     }
39 }
40 return sccs;
41 }

```

Listing 61: Kosaraju's SCC algorithm — $O(V + E)$

Graph Algorithm Complexities

Algorithm	Time	Key Use
BFS	$O(V + E)$	Shortest path (unweighted)
DFS	$O(V + E)$	Cycle detection, connectivity
Topological Sort	$O(V + E)$	Task scheduling on DAGs
Kosaraju's SCC	$O(V + E)$	2-SAT, condensation

23.6 Correctness of BFS

Theorem 23.1 (BFS Shortest-Path Correctness). After running BFS from source s in an unweighted graph G , for every vertex v reachable from s : $v.d = \delta(s, v)$ (the shortest-path distance).

Proof by Induction on Distance

We prove by induction on $\delta(s, v)$ that $v.d = \delta(s, v)$ for all reachable vertices.

Lemma 1 (Upper bound): $v.d \geq \delta(s, v)$ for all v .

Proof: By induction on the number of Enqueue operations. Initially $s.d = 0 = \delta(s, s)$ and $v.d = \infty \geq \delta(s, v)$ for $v \neq s$. When we set $v.d = u.d + 1$ during relaxation from u : $v.d = u.d + 1 \geq \delta(s, u) + 1 \geq \delta(s, v)$ (since $\delta(s, v) \leq \delta(s, u) + 1$ by the edge (u, v)). $\square$

Lemma 2 (Queue monotonicity): If vertices are enqueued in order $v_1, v_2, \dots$, then $v_1.d \leq v_2.d \leq \dots$ and at any moment, all elements in the queue have distances differing by at most 1.

Proof: BFS processes vertices in FIFO order. When u is dequeued and its neighbor v is enqueued, $v.d = u.d + 1$. Since u comes before later queue elements (which have $d \geq u.d$), the queue has distances in $\{u.d, u.d + 1\}$. Monotonicity follows. $\square$

Main theorem: Induction on $k = \delta(s, v)$.

Base ($k = 0$): $v = s$, $s.d = 0 = \delta(s, s)$. $\checkmark$

Inductive step: Assume the claim holds for all vertices at distance $< k$. Let $\delta(s, v) = k$. Then there exists u with $\delta(s, u) = k - 1$ and $(u, v) \in E$.

By inductive hypothesis, $u.d = k - 1$. When u is dequeued, BFS examines edge (u, v) .

Case 1: v is already discovered (WHITE $\rightarrow$ GRAY/BLACK). Then v was discovered via some vertex w with $w.d \leq u.d$, so $v.d \leq w.d + 1 \leq u.d + 1 = k$. Combined with Lemma 1: $v.d = k$. $\checkmark$

Case 2: v is undiscovered (WHITE). BFS sets $v.d = u.d + 1 = k$. ✓
 In both cases $v.d = k = \delta(s, v)$. □

23.7 Correctness of DFS

Theorem 23.2 (Parenthesis Theorem). For any two vertices u and v in a DFS, exactly one of the following holds:

1. $[u.d, u.f]$ and $[v.d, v.f]$ are entirely disjoint (neither is ancestor of the other).
2. $[u.d, u.f] \subseteq [v.d, v.f]$ (u is a descendant of v in the DFS tree).
3. $[v.d, v.f] \subseteq [u.d, u.f]$ (v is a descendant of u in the DFS tree).

The intervals nest like well-matched parentheses: $(u \dots (v \dots v) \dots u)$.

Proof

Consider the case $u.d < v.d$ (the other case is symmetric).

Case A: $v.d < u.f$ (i.e., v is discovered while u is still gray). Then v is discovered during the exploration of u 's subtree, making v a descendant of u . DFS will finish v before returning to u , so $v.f < u.f$. Thus $[v.d, v.f] \subseteq [u.d, u.f]$. ✓

Case B: $v.d > u.f$ (i.e., v is discovered after u is finished). Then v is not in u 's subtree, and u is not in v 's subtree. The intervals are disjoint: $u.d < u.f < v.d < v.f$. ✓

Note that $u.d < v.d < u.f < v.f$ is impossible: if v is discovered while u is gray, v must finish before u (the recursive call for v returns before the call for u). □

Theorem 23.3 (White-Path Theorem). Vertex v is a descendant of u in the DFS tree if and only if at time $u.d$ (when u is discovered), there is a path from u to v consisting entirely of WHITE vertices.

Proof

($\Rightarrow$) If v is a descendant of u in the DFS tree, then $u.d < v.d$ (by the parenthesis theorem). At time $u.d$, every vertex on the tree path from u to v has not yet been discovered (all are WHITE). ✓

($\Leftarrow$) If at time $u.d$ there is a white path $u = w_0, w_1, \dots, w_k = v$, we prove by induction on k that v is a descendant of u .

Base ($k = 0$): $v = u$, trivially a descendant. ✓

Inductive step: w_1 is white at time $u.d$ and is adjacent to u . Before u finishes, DFS will explore all neighbors of u , discovering w_1 and making it a descendant. At time $w_1.d$, vertices $w_2, \dots, w_k$ are still white (they weren't discovered during $[u.d, w_1.d]$). By induction, v is a descendant of w_1 , hence a descendant of u . □

Theorem 23.4 (DFS Cycle Detection Correctness). A directed graph G has a cycle if and only if a DFS of G produces a back edge.

Proof

($\Rightarrow$) If G has a cycle $v_0 \rightarrow v_1 \rightarrow \dots \rightarrow v_k \rightarrow v_0$, let v_i be the first vertex in the cycle discovered by DFS. At time $v_i.d$, all other vertices in the cycle are white. By the white-path theorem, they all become descendants of v_i . In particular, v_{i-1} (the predecessor of v_i in the cycle) is a descendant. The edge (v_{i-1}, v_i) goes from descendant to ancestor, which is a back edge. $\checkmark$

($\Leftarrow$) If DFS finds a back edge (u, v) , then v is an ancestor of u in the DFS tree. The tree path from v to u plus the back edge (u, v) forms a cycle. $\square$

Theorem 23.5 (Topological Sort Correctness). For a DAG, outputting vertices in reverse DFS finish time gives a valid topological ordering.

Proof

We need to show: for every edge $(u, v) \in E$, $u.f > v.f$.

When DFS explores edge (u, v) :

- v cannot be GRAY: that would be a back edge, implying a cycle (contradicting DAG). $\checkmark$
- If v is WHITE: v becomes a descendant of u , so $v.f < u.f$ (parenthesis theorem). $\checkmark$
- If v is BLACK: v is already finished, so $v.f < u.f$ (since u is still active). $\checkmark$

In all cases $u.f > v.f$, so reverse finish order puts u before v . $\square$

24 Minimum Spanning Trees

Minimum Spanning Tree

Given a connected, undirected, weighted graph $G = (V, E, w)$, a **minimum spanning tree** (MST) is a subset $T \subseteq E$ such that:

- T connects all vertices (spans V).
- T is acyclic (a tree: $|T| = |V| - 1$ edges).
- $\sum_{e \in T} w(e)$ is minimized.

24.1 The Cut Property

Theorem 24.1 (Cut Property — Foundation of MST Algorithms). For any cut $(S, V \setminus S)$ of the graph, the minimum-weight edge crossing the cut is in *some* MST. (If unique, it's in *every* MST.)

Both Kruskal's and Prim's algorithms are consequences of this property.

24.2 Kruskal's Algorithm

Strategy: Sort edges by weight. Greedily add each edge if it doesn't create a cycle (checked via Union-Find).

```

1 struct Edge { int u, v, w; };
2
3 pair<long long, vector<Edge>> kruskal(int n, vector<Edge>& edges) {
4     sort(edges.begin(), edges.end(),
5         [](auto& a, auto& b) { return a.w < b.w; });
6
7     DSU dsu(n); // Union-Find (from Chapter 21)
8     long long mstWeight = 0;
9     vector<Edge> mstEdges;
10
11     for (auto& e : edges) {
12         if (dsu.unite(e.u, e.v)) {
13             mstWeight += e.w;
14             mstEdges.push_back(e);
15             if ((int)mstEdges.size() == n - 1) break;
16         }
17     }
18     return {mstWeight, mstEdges};
19 }

```

Listing 62: Kruskal's MST — $O(E \lg E)$

24.3 Prim's Algorithm

Strategy: Grow the MST from a source vertex. Always add the cheapest edge connecting the tree to a non-tree vertex (like Dijkstra but simpler).

```

1 long long prim(int n, vector<vector<pair<int,int>>>& adj) {
2     vector<int> key(n, INT_MAX);
3     vector<bool> inMST(n, false);
4     priority_queue<pair<int,int>, vector<pair<int,int>>,
5         greater<pair<int,int>>> pq;
6
7     key[0] = 0;
8     pq.push({0, 0}); // {weight, vertex}
9     long long mstWeight = 0;
10
11     while (!pq.empty()) {
12         auto [w, u] = pq.top(); pq.pop();
13         if (inMST[u]) continue;
14         inMST[u] = true;
15         mstWeight += w;
16
17         for (auto& [v, wt] : adj[u]) {
18             if (!inMST[v] && wt < key[v]) {
19                 key[v] = wt;
20                 pq.push({wt, v});
21             }
22         }
23     }
24     return mstWeight;

```

25 }

Listing 63: Prim's MST — $O(E \lg V)$ with binary heap

Algorithm	Time	Data Structure	Best For
Kruskal	$O(E \lg E)$	Union-Find	Sparse graphs
Prim (binary heap)	$O(E \lg V)$	Min-heap	Dense graphs
Prim (Fib. heap)	$O(E + V \lg V)$	Fibonacci heap	Theory

Theorem 24.2 (Cut Property — Proof). For any cut $(S, V \setminus S)$ that no MST edge crosses, the minimum-weight crossing edge e belongs to some MST.

Proof by Exchange Argument

Let T be an MST. If $e \in T$, done. Otherwise, adding e to T creates a cycle. This cycle must contain another edge f crossing the same cut (to close the cycle). Since $w(e) \leq w(f)$ (e is the minimum crossing edge), the tree $T' = T \setminus \{f\} \cup \{e\}$ is a spanning tree with $w(T') \leq w(T)$. Since T is an MST, $w(T') = w(T)$, so T' is also an MST containing e . $\square$

Theorem 24.3 (Correctness of Kruskal's Algorithm). Kruskal's algorithm produces a minimum spanning tree.

Proof

We show each edge added by Kruskal's is safe (belongs to some MST). When Kruskal's adds edge $e = (u, v)$, u and v are in different components. Let S = the component containing u . The cut $(S, V \setminus S)$ has no previously selected edges crossing it (since $v \notin S$). Edge e is the minimum-weight edge crossing this cut (all lighter edges were processed earlier and either added or rejected because both endpoints were in the same component). By the cut property, e is in some MST. Since every added edge is safe and $|V| - 1$ edges are added, the result is an MST. $\square$

Theorem 24.4 (Correctness of Prim's Algorithm). Prim's algorithm produces a minimum spanning tree.

Proof

Let S = set of vertices already in the growing tree. At each step, Prim's adds the minimum-weight edge (u, v) with $u \in S$ and $v \notin S$. This edge is the lightest edge crossing the cut $(S, V \setminus S)$. By the cut property, it belongs to some MST. By induction, all edges added are safe, producing an MST after $|V| - 1$ steps. $\square$

25 Single-Source Shortest Paths

Given a weighted directed graph $G = (V, E, w)$ and a source vertex s , find the shortest-path weights $\delta(s, v)$ for all $v \in V$.

25.1 Properties and Relaxation

Relaxation

The fundamental operation. If we find a shorter path to v through u , update:

$$\text{If } d[v] > d[u] + w(u, v) : \quad d[v] = d[u] + w(u, v), \pi[v] = u$$

All shortest-path algorithms are sequences of relaxation steps.

```

1 void relax(int u, int v, int w, vector<int>& dist, vector<int>& par) {
2     if (dist[u] != INT_MAX && dist[u] + w < dist[v]) {
3         dist[v] = dist[u] + w;
4         par[v] = u;
5     }
6 }
```

Listing 64: Relaxation

25.2 Bellman-Ford Algorithm

Bellman-Ford

Handles **negative weights**. Relaxes all edges $|V| - 1$ times. Can detect **negative-weight cycles**.

```

1 struct Edge { int u, v, w; };
2
3 bool bellmanFord(int n, int src, vector<Edge>& edges,
4                 vector<long long>& dist) {
5     dist.assign(n, LLONG_MAX);
6     dist[src] = 0;
7
8     // Relax all edges V-1 times
9     for (int i = 0; i < n - 1; i++)
10         for (auto& [u, v, w] : edges)
11             if (dist[u] != LLONG_MAX && dist[u] + w < dist[v])
12                 dist[v] = dist[u] + w;
13
14     // Check for negative-weight cycles
15     for (auto& [u, v, w] : edges)
16         if (dist[u] != LLONG_MAX && dist[u] + w < dist[v])
17             return false; // negative cycle exists!
18
19     return true;
```

20 }

Listing 65: Bellman-Ford — $O(VE)$ **Why $|V| - 1$ Iterations?**

A shortest path has at most $|V| - 1$ edges (no repeated vertices in a shortest path without negative cycles). After k iterations, all shortest paths of $\leq k$ edges are correctly computed. After $|V| - 1$ iterations, all shortest paths are found.

If the $(|V|)$ -th iteration still reduces some distance, a negative-weight cycle exists (reachable from the source).

25.3 Shortest Paths in DAGs

On DAGs, relax edges in **topological order**. Runs in $O(V + E)$:

```

1 vector<long long> dagShortestPath(int n, int src,
2                                 vector<vector<pair<int,int>>>& adj) {
3     vector<int> topo = topologicalSort(adj, n); // from Ch 22
4     vector<long long> dist(n, LLONG_MAX);
5     dist[src] = 0;
6
7     for (int u : topo)
8         if (dist[u] != LLONG_MAX)
9             for (auto& [v, w] : adj[u])
10                 dist[v] = min(dist[v], dist[u] + w);
11
12     return dist;
13 }
```

Listing 66: DAG shortest paths — $O(V + E)$ **25.4 Dijkstra's Algorithm****Dijkstra's Algorithm**

Finds shortest paths from source s when all edge weights are **non-negative**. Greedily processes the closest unprocessed vertex.

```

1 vector<long long> dijkstra(int n, int src,
2                           vector<vector<pair<int,int>>>& adj) {
3     vector<long long> dist(n, LLONG_MAX);
4     priority_queue<pair<long long,int>,
5                  vector<pair<long long,int>>,
6                  greater<pair<long long,int>>> pq;
7     dist[src] = 0;
8     pq.push({0, src});
9
10    while (!pq.empty()) {
11        auto [d, u] = pq.top(); pq.pop();
```

```

12     if (d > dist[u]) continue; // stale entry
13
14     for (auto& [v, w] : adj[u]) {
15         if (dist[u] + w < dist[v]) {
16             dist[v] = dist[u] + w;
17             pq.push({dist[v], v});
18         }
19     }
20 }
21 return dist;
22 }

```

Listing 67: Dijkstra's — $O(E \lg V)$ with binary heap

Dijkstra Fails with Negative Weights

Dijkstra assumes once a vertex is processed, its distance is final. Negative edges can violate this. Use Bellman-Ford or Johnson's for negative weights.

Algorithm	Time	Negative Wts?	Neg. Cycles?	Graph Type
Bellman-Ford	$O(VE)$	Yes	Detects	Any
DAG shortest paths	$O(V + E)$	Yes	N/A (DAG)	DAG only
Dijkstra (binary heap)	$O(E \lg V)$	No	No	Non-negative
Dijkstra (Fib. heap)	$O(V \lg V + E)$	No	No	Non-negative

Theorem 25.1 (Correctness of Bellman-Ford). After $|V| - 1$ iterations, $d[v] = \delta(s, v)$ for all reachable v (assuming no negative-weight cycles).

Proof by Induction on Path Length

Claim: After iteration i , $d[v] = \delta(s, v)$ for all v whose shortest path from s has $\leq i$ edges.

Base ($i = 0$): $d[s] = 0 = \delta(s, s)$. ✓

Inductive step: Let v have a shortest path $s \rightarrow \dots \rightarrow u \rightarrow v$ with i edges. By IH, after iteration $i-1$, $d[u] = \delta(s, u)$. In iteration i , when edge (u, v) is relaxed: $d[v] \leq d[u] + w(u, v) = \delta(s, u) + w(u, v) = \delta(s, v)$. Since $d[v] \geq \delta(s, v)$ always (upper-bound property), $d[v] = \delta(s, v)$. ✓

Since any shortest path has $\leq |V| - 1$ edges, after $|V| - 1$ iterations all distances are correct. □

Theorem 25.2 (Correctness of Dijkstra's Algorithm). When a vertex u is extracted from the priority queue, $d[u] = \delta(s, u)$ (assuming non-negative edge weights).

Proof by Contradiction

Suppose u is the first vertex extracted with $d[u] > \delta(s, u)$. Then $u \neq s$ (since $d[s] = 0$). Let p be a true shortest path $s \rightarrow \dots \rightarrow u$.

Let (x, y) be the first edge on p where x is already extracted and y is not. Since x was extracted before u , by assumption $d[x] = \delta(s, x)$. When x was extracted, edge (x, y) was relaxed, so: $d[y] \leq d[x] + w(x, y) = \delta(s, y)$.

Since y appears before u on the shortest path and all weights are non-negative: $\delta(s, y) \leq \delta(s, u) < d[u]$.

So $d[y] \leq \delta(s, y) \leq \delta(s, u) < d[u]$, meaning y should have been extracted before u (smaller key in the min-priority queue). Contradiction. $\square$

26 All-Pairs Shortest Paths

Compute shortest-path distances $\delta(u, v)$ for *all* pairs $u, v \in V$.

26.1 Floyd-Warshall Algorithm

Floyd-Warshall

DP over intermediate vertices: $d^{(k)}[i][j]$ = shortest path from i to j using only vertices $\{1, \dots, k\}$ as intermediates.

$$d^{(k)}[i][j] = \min(d^{(k-1)}[i][j], d^{(k-1)}[i][k] + d^{(k-1)}[k][j])$$

```

1 const long long INF = 1e18;
2
3 void floydWarshall(vector<vector<long long>>& dist, int n) {
4     // dist[i][j] initialized to edge weights; dist[i][i] = 0; INF if no edge
5     for (int k = 0; k < n; k++)
6         for (int i = 0; i < n; i++)
7             for (int j = 0; j < n; j++)
8                 if (dist[i][k] < INF && dist[k][j] < INF)
9                     dist[i][j] = min(dist[i][j], dist[i][k] + dist[k][j]);
10
11     // Negative cycle detection: dist[i][i] < 0 for some i
12 }
```

Listing 68: Floyd-Warshall — $O(V^3)$ time, $O(V^2)$ space

Floyd-Warshall Applications

- **All-pairs shortest paths:** works with negative weights (but not negative cycles).
- **Transitive closure:** replace min with OR, + with AND.
- **Minimax/Maximin paths:** replace min and + accordingly.
- **Detecting negative cycles:** check if any $d[i][i] < 0$.

- Best for dense graphs ($E \approx V^2$) where $n \leq 500$.

```

1 void transitiveClosure(vector<vector<bool>>& reach, int n) {
2     for (int k = 0; k < n; k++)
3         for (int i = 0; i < n; i++)
4             for (int j = 0; j < n; j++)
5                 reach[i][j] = reach[i][j] || (reach[i][k] && reach[k][j]);
6 }

```

Listing 69: Transitive closure via Floyd-Warshall

26.2 Johnson's Algorithm

Johnson's Algorithm

Computes all-pairs shortest paths in **sparse** graphs efficiently using:

1. **Bellman-Ford** once to compute a reweighting function $h(v)$.
2. **Dijkstra** from each vertex on the reweighted (non-negative) graph.

Reweighting: Add a new vertex s with zero-weight edges to all vertices. Run Bellman-Ford from s to get $h(v) = \delta(s, v)$. Define new weights: $\hat{w}(u, v) = w(u, v) + h(u) - h(v) \geq 0$.

Time: $O(V^2 \lg V + VE)$ — better than Floyd-Warshall for sparse graphs.

```

1 vector<vector<long long>> johnson(int n, vector<Edge>& edges,
2                                 vector<vector<pair<int,int>>>& adj) {
3     // Step 1: Add super-source s with 0-weight edges to all
4     int s = n;
5     for (int v = 0; v < n; v++)
6         edges.push_back({s, v, 0});
7
8     // Step 2: Bellman-Ford from s to get h[]
9     vector<long long> h;
10    if (!bellmanFord(n + 1, s, edges, h))
11        throw runtime_error("negative cycle");
12
13    // Step 3: Reweight edges
14    for (int u = 0; u < n; u++)
15        for (auto& [v, w] : adj[u])
16            w = w + h[u] - h[v]; // now non-negative
17
18    // Step 4: Dijkstra from each vertex
19    vector<vector<long long>> dist(n);
20    for (int u = 0; u < n; u++) {
21        dist[u] = dijkstra(n, u, adj);
22        for (int v = 0; v < n; v++)
23            if (dist[u][v] < LLONG_MAX)
24                dist[u][v] += h[v] - h[u]; // undo reweighting
25    }
26    return dist;

```

27 }

Listing 70: Johnson's algorithm sketch

Algorithm	Time	Space	Best For
Floyd-Warshall	$O(V^3)$	$O(V^2)$	Dense, $V \leq 500$
Johnson's	$O(V^2 \lg V + VE)$	$O(V^2)$	Sparse, $V \leq 10^4$
$V \times$ Dijkstra	$O(VE \lg V)$	$O(V^2)$	Non-negative weights
$V \times$ Bellman-Ford	$O(V^2 E)$	$O(V^2)$	Negative weights (slow)

Theorem 26.1 (Correctness of Floyd-Warshall). After the algorithm, $d[i][j] = \delta(i, j)$ for all i, j .

Proof by Induction on k

Invariant: After iteration k , $d[i][j] = d^{(k)}[i][j]$ = shortest path from i to j using only intermediate vertices $\{1, \dots, k\}$.

Base ($k = 0$): $d^{(0)}[i][j] = w(i, j)$ (direct edge weight), which is the shortest path with no intermediate vertices. ✓

Inductive step: Consider the shortest $i \rightarrow j$ path using intermediates $\{1, \dots, k\}$. Either it uses vertex k or it doesn't:

- **Doesn't use k :** Path uses only $\{1, \dots, k-1\}$, so optimal value is $d^{(k-1)}[i][j]$.
- **Uses k :** Path goes $i \rightarrow \dots \rightarrow k \rightarrow \dots \rightarrow j$, where both subpaths use only $\{1, \dots, k-1\}$. Cost is $d^{(k-1)}[i][k] + d^{(k-1)}[k][j]$.

Taking the minimum gives the recurrence. After $k = n$: all vertices are allowed as intermediates, so $d^{(n)}[i][j] = \delta(i, j)$. □

Theorem 26.2 (Correctness of Johnson's Reweighting). The reweighted distances $\hat{w}(u, v) = w(u, v) + h(u) - h(v) \geq 0$, and shortest paths are preserved (same path, different weight).

Proof

Non-negativity: By Bellman-Ford, $h(v) \leq h(u) + w(u, v)$ for every edge (triangle inequality). Rearranging: $w(u, v) + h(u) - h(v) \geq 0$. ✓

Path preservation: For any path $p = v_0 \rightarrow v_1 \rightarrow \dots \rightarrow v_k$:

$$\hat{w}(p) = \sum_{i=0}^{k-1} \hat{w}(v_i, v_{i+1}) = \sum_{i=0}^{k-1} [w(v_i, v_{i+1}) + h(v_i) - h(v_{i+1})] = w(p) + h(v_0) - h(v_k)$$

The h terms telescope! So $\hat{w}(p) = w(p) + h(v_0) - h(v_k)$. Since $h(v_0) - h(v_k)$ is the same for all paths from v_0 to v_k , a shortest path under w is also shortest under $\hat{w}$. □

27 Maximum Flow

27.1 Flow Networks

Flow Network

A **flow network** $G = (V, E)$ is a directed graph where each edge $(u, v) \in E$ has a non-negative **capacity** $c(u, v) \geq 0$. Two distinguished vertices: **source** s and **sink** t .

A **flow** $f : V \times V \rightarrow \mathbb{R}$ satisfies:

1. **Capacity constraint:** $0 \leq f(u, v) \leq c(u, v)$ for all (u, v) .
2. **Flow conservation:** For all $u \neq s, t$: $\sum_v f(v, u) = \sum_v f(u, v)$ (flow in = flow out).

The **value** $|f| = \sum_v f(s, v) - \sum_v f(v, s)$.

27.2 Max-Flow Min-Cut Theorem

Theorem 27.1 (Max-Flow Min-Cut). The maximum flow equals the minimum cut capacity:

$$\max_f |f| = \min_{(S,T) \text{ is a cut}} c(S, T)$$

where $c(S, T) = \sum_{u \in S, v \in T} c(u, v)$.

This is one of the most important theorems in combinatorial optimization.

27.3 Ford-Fulkerson Method

Residual Graph

For a flow f , the **residual graph** G_f has edges:

- **Forward:** if $f(u, v) < c(u, v)$, residual capacity $c_f(u, v) = c(u, v) - f(u, v)$.
- **Backward:** if $f(u, v) > 0$, residual capacity $c_f(v, u) = f(u, v)$.

An **augmenting path** is an s - t path in G_f .

Algorithm 25 Ford-Fulkerson(G, s, t)

- 1: Initialize flow $f = 0$ on all edges
 - 2: **while** there exists an augmenting path p in G_f **do**
 - 3: $c_f(p) = \min_{(u,v) \in p} c_f(u, v)$ ▷ bottleneck
 - 4: Augment flow along p by $c_f(p)$
 - 5: **end while**
 - 6: **return** $|f|$
-

27.4 Edmonds-Karp Algorithm

Use **BFS** to find augmenting paths (shortest in terms of number of edges). This guarantees $O(VE)$ augmentations, giving $O(VE^2)$ total time.

```

1 struct MaxFlow {
2     struct Edge { int to, rev; long long cap; };
3     int n;
4     vector<vector<Edge>> graph;
5
6     MaxFlow(int n) : n(n), graph(n) {}
7
8     void addEdge(int u, int v, long long cap) {
9         graph[u].push_back({v, (int)graph[v].size(), cap});
10        graph[v].push_back({u, (int)graph[u].size() - 1, 0}); // reverse
11    }
12
13    long long bfs(int s, int t, vector<int>& parent, vector<int>& parentEdge) {
14        parent.assign(n, -1);
15        parentEdge.assign(n, -1);
16        parent[s] = s;
17        queue<pair<int, long long>> q;
18        q.push({s, LLONG_MAX});
19
20        while (!q.empty()) {
21            auto [u, flow] = q.front(); q.pop();
22            for (int i = 0; i < (int)graph[u].size(); i++) {
23                auto& e = graph[u][i];
24                if (parent[e.to] == -1 && e.cap > 0) {
25                    parent[e.to] = u;
26                    parentEdge[e.to] = i;
27                    long long newFlow = min(flow, e.cap);
28                    if (e.to == t) return newFlow;
29                    q.push({e.to, newFlow});
30                }
31            }
32        }
33        return 0;
34    }
35
36    long long maxflow(int s, int t) {
37        long long flow = 0;
38        vector<int> parent, parentEdge;
39        long long augment;
40
41        while ((augment = bfs(s, t, parent, parentEdge)) > 0) {
42            flow += augment;
43            int cur = t;
44            while (cur != s) {
45                int prev = parent[cur];
46                int idx = parentEdge[cur];
47                graph[prev][idx].cap -= augment;
48                graph[cur][graph[prev][idx].rev].cap += augment;
49                cur = prev;
50            }

```



```

51     }
52     return flow;
53 }
54 };

```

Listing 71: Edmonds-Karp (BFS-based max flow) — $O(VE^2)$

27.5 Dinic's Algorithm

Dinic's uses BFS to build **level graphs** and DFS to find **blocking flows**. Much faster in practice.

```

1 struct Dinic {
2     struct Edge { int to, rev; long long cap; };
3     int n;
4     vector<vector<Edge>> graph;
5     vector<int> level, iter;
6
7     Dinic(int n) : n(n), graph(n), level(n), iter(n) {}
8
9     void addEdge(int u, int v, long long cap) {
10         graph[u].push_back({v, (int)graph[v].size(), cap});
11         graph[v].push_back({u, (int)graph[u].size() - 1, 0});
12     }
13
14     bool bfs(int s, int t) {
15         level.assign(n, -1);
16         queue<int> q;
17         level[s] = 0;
18         q.push(s);
19         while (!q.empty()) {
20             int u = q.front(); q.pop();
21             for (auto& e : graph[u])
22                 if (e.cap > 0 && level[e.to] < 0) {
23                     level[e.to] = level[u] + 1;
24                     q.push(e.to);
25                 }
26         }
27         return level[t] >= 0;
28     }
29
30     long long dfs(int u, int t, long long pushed) {
31         if (u == t) return pushed;
32         for (int& i = iter[u]; i < (int)graph[u].size(); i++) {
33             Edge& e = graph[u][i];
34             if (e.cap > 0 && level[e.to] == level[u] + 1) {
35                 long long d = dfs(e.to, t, min(pushed, e.cap));
36                 if (d > 0) {
37                     e.cap -= d;
38                     graph[e.to][e.rev].cap += d;
39                     return d;

```

```

40     }
41   }
42 }
43 return 0;
44 }
45
46 long long maxflow(int s, int t) {
47   long long flow = 0;
48   while (bfs(s, t)) {
49     iter.assign(n, 0);
50     long long d;
51     while ((d = dfs(s, t, LLONG_MAX)) > 0)
52       flow += d;
53   }
54   return flow;
55 }
56 };

```

Listing 72: Dinic's algorithm — $O(V^2E)$

Flow Algorithm Comparison

Algorithm	Time	Notes
Ford-Fulkerson	$O(E \cdot f^*)$	May not terminate (irrational caps)
Edmonds-Karp	$O(VE^2)$	BFS augmenting paths
Dinic's	$O(V^2E)$	Blocking flows; $O(E\sqrt{V})$ for unit caps
Push-Relabel	$O(V^2E)$	Best for dense graphs

Dinic's is the go-to for competitive programming.

Max-Flow Applications

- **Bipartite matching:** maximum matching = max flow in a bipartite network.
- **Edge-disjoint paths:** max number of edge-disjoint s - t paths = max flow (unit capacities).
- **Minimum vertex cut:** split each vertex into two; max flow gives min cut.
- **Project selection:** model dependencies as flow constraints.

28 Multithreaded Algorithms

This chapter introduces the theory of parallel algorithms using **fork-join** parallelism (spawn/sync model).

28.1 Key Concepts

Parallel Performance Metrics

- $T_1 = \text{work}$: total time on 1 processor (serial time).
- $T_\infty = \text{span}$ (critical path length): time on ∞ processors.
- $T_P =$ time on P processors.
- **Speedup**: T_1/T_P .
- **Parallelism**: T_1/T_∞ (max useful processors).

Theorem 28.1 (Work and Span Laws).

$$T_P \geq T_1/P \quad (\text{work law: can't do better than dividing work})$$

$$T_P \geq T_\infty \quad (\text{span law: can't beat the critical path})$$

Theorem 28.2 (Greedy Scheduler). A greedy scheduler achieves:

$$T_P \leq T_1/P + T_\infty$$

Near-optimal when $P \ll T_1/T_\infty$ (parallelism is high).

28.2 Parallel Merge Sort

```

1 // Work:  $O(n \lg n)$ , Span:  $O(\lg^3 n)$ , Parallelism:  $O(n / \lg^2 n)$ 
2 void parallelMergeSort(int* A, int lo, int hi) {
3     if (hi - lo <= 1) return;
4     int mid = (lo + hi) / 2;
5     // spawn: run left half in parallel
6     spawn parallelMergeSort(A, lo, mid);
7     parallelMergeSort(A, mid, hi);
8     sync; // wait for spawned task
9     parallelMerge(A, lo, mid, hi); // parallel merge
10 }
```

Listing 73: Parallel merge sort (conceptual)

The bottleneck is the merge step. A **parallel merge** using binary search achieves $O(\lg^2 n)$ span, giving overall span $O(\lg^3 n)$.

28.3 Parallel Matrix Multiplication

Divide and conquer: Split matrices into quadrants, multiply recursively.

Work: $T_1 = O(n^3)$. Span: $T_\infty = O(\lg^2 n)$ (using parallel additions).

Parallelism: $\Theta(n^3/\lg^2 n)$ — excellent for large matrices.

Practical Parallel Computing

The CLRS model is theoretical. In practice:

- C++ uses `std::thread`, `std::async`, or OpenMP.
- GPU computing (CUDA) offers massive parallelism.
- Cache effects and synchronization overhead dominate real performance.

- Competitive programming is almost always single-threaded.

29 Matrix Operations

29.1 Solving Systems of Linear Equations

LUP Decomposition

Decompose matrix A into $PA = LU$ where:

- L = lower triangular (1s on diagonal).
- U = upper triangular.
- P = permutation matrix (for numerical stability).

Solves $Ax = b$ in $O(n^2)$ after $O(n^3)$ decomposition.

```

1 // Solves  $Ax = b$ , returns  $x$ 
2 vector<double> gaussianElim(vector<vector<double>>& A, vector<double>& b) {
3     int n = A.size();
4     // Augmented matrix
5     for (int i = 0; i < n; i++) A[i].push_back(b[i]);
6
7     for (int col = 0; col < n; col++) {
8         // Partial pivoting: find row with max  $|A[row][col]|$ 
9         int pivot = col;
10        for (int row = col + 1; row < n; row++)
11            if (abs(A[row][col]) > abs(A[pivot][col]))
12                pivot = row;
13        swap(A[col], A[pivot]);
14
15        if (abs(A[col][col]) < 1e-9) continue; // singular
16
17        // Eliminate below
18        for (int row = col + 1; row < n; row++) {
19            double factor = A[row][col] / A[col][col];
20            for (int j = col; j <= n; j++)
21                A[row][j] -= factor * A[col][j];
22        }
23    }
24
25    // Back substitution
26    vector<double> x(n);
27    for (int i = n - 1; i >= 0; i--) {
28        x[i] = A[i][n];
29        for (int j = i + 1; j < n; j++)
30            x[i] -= A[i][j] * x[j];
31        x[i] /= A[i][i];
32    }
33    return x;
34 }
```

Listing 74: Gaussian elimination with partial pivoting — $O(n^3)$

29.2 Matrix Inversion

Compute A^{-1} by solving $AX = I$ column by column, or via LUP decomposition. Time: $O(n^3)$.

Can also use **Strassen's algorithm** recursively: matrix inversion has the same asymptotic complexity as matrix multiplication ($O(n^{\lg 7}) \approx O(n^{2.807})$).

29.3 Least Squares Approximation

Given overdetermined system $Ax \approx b$ (more equations than unknowns), find x minimizing $\|Ax - b\|^2$:

$$x = (A^T A)^{-1} A^T b$$

This is the **normal equation**. Numerically, use QR decomposition or SVD for better stability.

Matrix Operation Complexities

Operation	Naive	Best Known
Multiplication	$O(n^3)$	$O(n^{2.3728\dots})$
Inversion	$O(n^3)$	$O(n^\omega)$ (same as mult)
Determinant	$O(n^3)$	$O(n^\omega)$
LU Decomposition	$O(n^3)$	$O(n^3)$
Solving $Ax = b$	$O(n^3)$	$O(n^3)$

For competitive programming, $O(n^3)$ Gaussian elimination is standard.

30 Linear Programming

30.1 Overview

Linear Program

Optimize a linear objective subject to linear constraints:

$$\begin{aligned} &\text{Maximize} && \mathbf{c}^T \mathbf{x} \\ &\text{subject to} && A\mathbf{x} \leq \mathbf{b} \\ &&& \mathbf{x} \geq \mathbf{0} \end{aligned}$$

where $\mathbf{x} \in \mathbb{R}^n$ are decision variables, $A \in \mathbb{R}^{m \times n}$, $\mathbf{b} \in \mathbb{R}^m$, $\mathbf{c} \in \mathbb{R}^n$.

Standard form: maximize, $\leq$ constraints, non-negative variables.

Slack form: convert inequalities to equalities using slack variables: $\sum a_{ij}x_j \leq b_i \iff \sum a_{ij}x_j + s_i = b_i, s_i \geq 0$.

30.2 The Simplex Algorithm

Simplex Method

1. Start at a vertex (basic feasible solution) of the feasible polytope.
2. Move to an adjacent vertex that improves the objective.
3. Repeat until no improving neighbor exists (optimum found).

Worst case: exponential. **Average/practical:** very fast, typically polynomial.

Each iteration involves:

1. **Entering variable:** choose a non-basic variable with positive coefficient in the objective (any such variable improves the solution).
2. **Leaving variable:** determine which basic variable becomes zero first (minimum ratio test) to maintain feasibility.
3. **Pivot:** exchange the entering and leaving variables.

30.3 Duality

Theorem 30.1 (LP Duality). Every LP (primal) has a **dual** LP:

Primal	Dual
$\max \mathbf{c}^T \mathbf{x}$	$\min \mathbf{b}^T \mathbf{y}$
$A\mathbf{x} \leq \mathbf{b}$	$A^T \mathbf{y} \geq \mathbf{c}$
$\mathbf{x} \geq 0$	$\mathbf{y} \geq 0$

- **Weak duality:** any dual feasible $\mathbf{y}$ gives an upper bound on the primal.
- **Strong duality:** at optimality, $\mathbf{c}^T \mathbf{x}^* = \mathbf{b}^T \mathbf{y}^*$.

30.4 Applications

LP Applications

Many problems reduce to LP:

- **Shortest paths:** $\max d_t$ s.t. $d_v - d_u \leq w(u, v)$, $d_s = 0$.
- **Max flow:** LP with flow conservation and capacity constraints.
- **Min cost flow:** LP with costs on edges.
- **Matching:** LP relaxation (integer solutions for bipartite graphs).
- **Scheduling, resource allocation, portfolio optimization.**

Integer Linear Programming

If variables must be integers (**ILP**), the problem becomes NP-hard in general. The LP relaxation (allow fractional solutions) gives bounds for branch-and-bound.

31 Polynomials and the FFT

31.1 Polynomial Representations

Polynomial Representations

A polynomial $A(x) = \sum_{j=0}^{n-1} a_j x^j$ of degree $n-1$ can be represented as:

1. **Coefficient:** vector $(a_0, a_1, \dots, a_{n-1})$.
2. **Point-value:** n pairs $\{(x_0, A(x_0)), \dots, (x_{n-1}, A(x_{n-1}))\}$.

Key insight: Polynomial multiplication is $O(n^2)$ in coefficient form but $O(n)$ in point-value form (just multiply componentwise). FFT converts between representations in $O(n \lg n)$.

31.2 The Discrete Fourier Transform

DFT

The DFT of vector $a = (a_0, \dots, a_{n-1})$ evaluates the polynomial $A(x) = \sum a_j x^j$ at the n -th **roots of unity** $\omega_n^k = e^{2\pi i k/n}$:

$$y_k = A(\omega_n^k) = \sum_{j=0}^{n-1} a_j \omega_n^{jk}$$

The **inverse DFT** recovers a from y : $a_j = \frac{1}{n} \sum_{k=0}^{n-1} y_k \omega_n^{-jk}$.

31.3 Fast Fourier Transform (FFT)

FFT — Cooley-Tukey

Divide and conquer on even/odd coefficients:

$$\begin{aligned} A(x) &= A_{\text{even}}(x^2) + x \cdot A_{\text{odd}}(x^2) \\ A_{\text{even}}(x) &= a_0 + a_2 x + a_4 x^2 + \dots \\ A_{\text{odd}}(x) &= a_1 + a_3 x + a_5 x^2 + \dots \end{aligned}$$

Two half-size DFTs + $O(n)$ combination = $O(n \lg n)$ total.

```

1 using cd = complex<double>;
2 const double PI = acos(-1);
3
4 void fft(vector<cd>& a, bool invert) {
5     int n = a.size();
6     if (n == 1) return;
7
8     // Bit-reversal permutation
9     for (int i = 1, j = 0; i < n; i++) {
10         int bit = n >> 1;

```

```

11     for (; j & bit; bit >>= 1) j ^= bit;
12     j ^= bit;
13     if (i < j) swap(a[i], a[j]);
14 }
15
16 // Iterative FFT (bottom-up)
17 for (int len = 2; len <= n; len <<= 1) {
18     double ang = 2 * PI / len * (invert ? -1 : 1);
19     cd wlen(cos(ang), sin(ang));
20     for (int i = 0; i < n; i += len) {
21         cd w(1);
22         for (int j = 0; j < len / 2; j++) {
23             cd u = a[i + j];
24             cd v = a[i + j + len/2] * w;
25             a[i + j] = u + v; // butterfly
26             a[i + j + len/2] = u - v;
27             w *= wlen;
28         }
29     }
30 }
31 if (invert)
32     for (auto& x : a) x /= n;
33 }

```

Listing 75: FFT — $O(n \lg n)$

```

1 vector<long long> multiply(vector<int>& a, vector<int>& b) {
2     vector<cd> fa(a.begin(), a.end()), fb(b.begin(), b.end());
3     int n = 1;
4     while (n < (int)(a.size() + b.size() - 1)) n <<= 1;
5     fa.resize(n); fb.resize(n);
6
7     fft(fa, false); // DFT
8     fft(fb, false);
9     for (int i = 0; i < n; i++)
10         fa[i] *= fb[i]; // pointwise multiply
11     fft(fa, true); // inverse DFT
12
13     vector<long long> result(n);
14     for (int i = 0; i < n; i++)
15         result[i] = llround(fa[i].real());
16     return result;
17 }

```

Listing 76: Polynomial multiplication using FFT — $O(n \lg n)$

Number Theoretic Transform (NTT)

For exact integer arithmetic (avoiding floating-point errors), use the **NTT**: FFT over $\mathbb{Z}/p\mathbb{Z}$ where p is a prime with $p - 1$ divisible by a large power of 2 (e.g., $p = 998244353$). The n -th root of unity is a *primitive root* modulo p .

NTT is essential in competitive programming for problems involving polynomial

multiplication modulo a prime.

```

1  const int MOD = 998244353;
2  const int g = 3; // primitive root of MOD
3
4  long long power(long long a, long long b, long long mod) {
5      long long res = 1; a %= mod;
6      while (b > 0) {
7          if (b & 1) res = res * a % mod;
8          a = a * a % mod;
9          b >>= 1;
10     }
11     return res;
12 }
13
14 void ntt(vector<long long>& a, bool invert) {
15     int n = a.size();
16     for (int i = 1, j = 0; i < n; i++) {
17         int bit = n >> 1;
18         for (; j & bit; bit >>= 1) j ^= bit;
19         j ^= bit;
20         if (i < j) swap(a[i], a[j]);
21     }
22     for (int len = 2; len <= n; len <<= 1) {
23         long long w = invert ? power(g, MOD - 1 - (MOD - 1) / len, MOD)
24                           : power(g, (MOD - 1) / len, MOD);
25         for (int i = 0; i < n; i += len) {
26             long long wn = 1;
27             for (int j = 0; j < len / 2; j++) {
28                 long long u = a[i+j], v = a[i+j+len/2] * wn % MOD;
29                 a[i+j] = (u + v) % MOD;
30                 a[i+j+len/2] = (u - v + MOD) % MOD;
31                 wn = wn * w % MOD;
32             }
33         }
34     }
35     if (invert) {
36         long long inv_n = power(n, MOD - 2, MOD);
37         for (auto& x : a) x = x * inv_n % MOD;
38     }
39 }

```

Listing 77: NTT (mod 998244353) — exact integer FFT

32 Number-Theoretic Algorithms

32.1 GCD and Extended Euclidean

Euclidean Algorithm

$\gcd(a, b) = \gcd(b, a \bmod b)$, with base case $\gcd(a, 0) = a$.

Time: $O(\lg(\min(a, b)))$ — number of steps $\leq 5 \cdot \text{digits}(b)$.

```

1 long long gcd(long long a, long long b) {
2     return b == 0 ? a : gcd(b, a % b);
3 }
4
5 // Returns gcd(a,b), and sets x,y such that ax + by = gcd(a,b)
6 long long extgcd(long long a, long long b, long long &x, long long &y) {
7     if (b == 0) { x = 1; y = 0; return a; }
8     long long x1, y1;
9     long long g = extgcd(b, a % b, x1, y1);
10    x = y1;
11    y = x1 - (a / b) * y1;
12    return g;
13 }
```

Listing 78: GCD and Extended GCD

32.2 Modular Arithmetic

Modular Arithmetic Rules

- $(a + b) \bmod m = ((a \bmod m) + (b \bmod m)) \bmod m$
- $(a \cdot b) \bmod m = ((a \bmod m) \cdot (b \bmod m)) \bmod m$
- $(a - b) \bmod m = ((a \bmod m) - (b \bmod m) + m) \bmod m$
- Division: multiply by modular inverse (not direct division!)

```

1 long long power(long long base, long long exp, long long mod) {
2     long long result = 1;
3     base %= mod;
4     while (exp > 0) {
5         if (exp & 1) result = result * base % mod;
6         base = base * base % mod;
7         exp >>= 1;
8     }
9     return result;
10 }
```

Listing 79: Modular exponentiation (binary exponentiation) — $O(\lg n)$

32.3 Modular Inverse

Modular Inverse

$a^{-1} \pmod{m}$ exists iff $\gcd(a, m) = 1$.

- If m is prime: $a^{-1} \equiv a^{m-2} \pmod{m}$ (Fermat's little theorem).
- General: use extended GCD.

```

1 long long modinv(long long a, long long m) {
2     long long x, y;
3     long long g = extgcd(a, m, x, y);
4     if (g != 1) return -1; // inverse doesn't exist
5     return (x % m + m) % m;
6 }
7
8 // If m is prime:
9 long long modinvPrime(long long a, long long m) {
10     return power(a, m - 2, m);
11 }

```

Listing 80: Modular inverse

32.4 Chinese Remainder Theorem

Theorem 32.1 (CRT). Given pairwise coprime $m_1, \dots, m_k$ and values $a_1, \dots, a_k$, the system

$$x \equiv a_i \pmod{m_i} \quad \text{for } i = 1, \dots, k$$

has a **unique** solution modulo $M = m_1 \cdot m_2 \cdots m_k$.

```

1 // Solve x = a1 (mod m1), x = a2 (mod m2)
2 // Returns {x, lcm(m1,m2)} or {-1,-1} if no solution
3 pair<long long, long long> crt(long long a1, long long m1,
4                               long long a2, long long m2) {
5     long long g = gcd(m1, m2);
6     if ((a2 - a1) % g != 0) return {-1, -1};
7
8     long long lcm = m1 / g * m2;
9     long long x, y;
10    extgcd(m1 / g, m2 / g, x, y);
11    long long diff = (a2 - a1) / g;
12    long long mod = m2 / g;
13    long long t = (diff % mod * (x % mod) % mod + mod) % mod;
14    return {a1 + m1 * t, lcm};
15 }

```

Listing 81: CRT for two equations

32.5 Primality Testing

```

1 bool isPrime(long long n) {
2     if (n < 2) return false;
3     if (n < 4) return true;
4     if (n % 2 == 0 || n % 3 == 0) return false;
5     for (long long i = 5; i * i <= n; i += 6)
6         if (n % i == 0 || n % (i + 2) == 0)
7             return false;
8     return true;
9 }

```

Listing 82: Trial division — $O(\sqrt{n})$

```

1 bool millerRabin(long long n, long long a) {
2     if (n % a == 0) return n == a;
3     long long d = n - 1;
4     int r = 0;
5     while (d % 2 == 0) { d /= 2; r++; }
6
7     long long x = power(a, d, n); // a^d mod n
8     if (x == 1 || x == n - 1) return true;
9     for (int i = 0; i < r - 1; i++) {
10         x = (__int128)x * x % n;
11         if (x == n - 1) return true;
12     }
13     return false;
14 }
15
16 bool isPrimeMR(long long n) {
17     if (n < 2) return false;
18     // Deterministic for n < 3.317*10^24 with these bases
19     for (long long a : {2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37})
20         if (!millerRabin(n, a)) return false;
21     return true;
22 }

```

Listing 83: Miller-Rabin primality test — $O(k \lg^2 n)$

32.6 Sieve of Eratosthenes

```

1 vector<bool> sieve(int n) {
2     vector<bool> is_prime(n + 1, true);
3     is_prime[0] = is_prime[1] = false;
4     for (int i = 2; i * i <= n; i++)
5         if (is_prime[i])
6             for (int j = i * i; j <= n; j += i)
7                 is_prime[j] = false;
8     return is_prime;
9 }
10
11 // Linear sieve + smallest prime factor

```

```

12 vector<int> linearSieve(int n) {
13     vector<int> spf(n + 1, 0); // smallest prime factor
14     vector<int> primes;
15     for (int i = 2; i <= n; i++) {
16         if (spf[i] == 0) { spf[i] = i; primes.push_back(i); }
17         for (int p : primes) {
18             if (p > spf[i] || (long long)i * p > n) break;
19             spf[i * p] = p;
20         }
21     }
22     return spf;
23 }

```

Listing 84: Sieve — $O(n \lg \lg n)$

Euler's Totient Function

$\phi(n)$ = count of integers in $[1, n]$ coprime to n .

$$\phi(n) = n \prod_{p|n} \left(1 - \frac{1}{p}\right)$$

- $\phi(p) = p - 1$ for prime p .
- $\phi(p^k) = p^{k-1}(p - 1)$.
- $a^{\phi(n)} \equiv 1 \pmod{n}$ when $\gcd(a, n) = 1$ (Euler's theorem).

```

1 long long phi(long long n) {
2     long long result = n;
3     for (long long p = 2; p * p <= n; p++) {
4         if (n % p == 0) {
5             while (n % p == 0) n /= p;
6             result -= result / p;
7         }
8     }
9     if (n > 1) result -= result / n;
10    return result;
11 }

```

Listing 85: Euler's totient — $O(\sqrt{n})$

Theorem 32.2 (Correctness of the Euclidean Algorithm). $\gcd(a, b) = \gcd(b, a \bmod b)$ and the algorithm terminates.

Proof

Let $d = \gcd(a, b)$. Write $a = qb + r$ where $r = a \bmod b$. Any common divisor of a and b also divides $r = a - qb$. Conversely, any common divisor of b and r divides $a = qb + r$. So $\gcd(a, b) = \gcd(b, r)$.

Termination: Each step reduces the second argument: $0 \leq r < b$. So the sequence of second arguments is strictly decreasing, reaching 0. Then $\gcd(a, 0) = a$. $\square$

Theorem 32.3 (Correctness of Binary Exponentiation). $\text{power}(b, e, m)$ returns $b^e \bmod m$.

Proof by Loop Invariant

Invariant: $\text{result} \cdot \text{base}^{\text{exp}} \equiv b^e \pmod{m}$.

Init: $\text{result} = 1$, $\text{base} = b$, $\text{exp} = e$. So $1 \cdot b^e = b^e$. ✓

Maintenance: If exp is odd: $\text{result} \leftarrow \text{result} \cdot \text{base}$, $\text{exp} \leftarrow \text{exp} - 1$. If exp is even: $\text{base} \leftarrow \text{base}^2$, $\text{exp} \leftarrow \text{exp}/2$. Both preserve the invariant. ✓

Termination: $\text{exp} = 0$, so $\text{result} \cdot \text{base}^0 = \text{result} = b^e \bmod m$. □

Theorem 32.4 (Correctness of Sieve of Eratosthenes). The sieve correctly identifies all primes up to n .

Proof

Claim: After the sieve, $\text{is_prime}[k] = \text{true}$ iff k is prime.

($\Leftarrow$) If k is prime, no smaller number divides it, so it is never marked as composite. ✓

($\Rightarrow$) If k is composite, then $k = ab$ with $1 < a \leq b < k$. We have $a \leq \sqrt{k} \leq \sqrt{n}$. Let p be the smallest prime factor of k . Then $p \leq a \leq \sqrt{k}$.

When the outer loop reaches $i = p$ (which happens since $p \leq \sqrt{n}$), the inner loop marks k as composite (starting from p^2 and stepping by p , it will reach k since $p \mid k$ and $k \geq p^2$). □

Starting from i^2 : All multiples $i \cdot j$ with $j < i$ were already marked when processing j 's smallest prime factor.

33 String Matching

Given a text $T[1..n]$ and pattern $P[1..m]$, find all occurrences of P in T .

33.1 Naive Algorithm

```

1 vector<int> naiveMatch(string& T, string& P) {
2     int n = T.size(), m = P.size();
3     vector<int> matches;
4     for (int i = 0; i <= n - m; i++) {
5         bool match = true;
6         for (int j = 0; j < m; j++)
7             if (T[i + j] != P[j]) { match = false; break; }
8         if (match) matches.push_back(i);
9     }
10    return matches;
11 }

```

Listing 86: Naive string matching — $O(nm)$

33.2 Rabin-Karp (Rolling Hash)

Rabin-Karp

Use a rolling hash to compare pattern hash with text window hashes. Only verify characters on hash match (to handle collisions).

Expected time: $O(n + m)$. Worst case: $O(nm)$ (many false matches).

```

1 vector<int> rabinKarp(string& T, string& P) {
2     int n = T.size(), m = P.size();
3     if (m > n) return {};
4     const long long MOD = 1e9 + 7, BASE = 31;
5     vector<int> matches;
6
7     // Compute hash of pattern and first window
8     long long pHash = 0, tHash = 0, basePow = 1;
9     for (int i = 0; i < m; i++) {
10         pHash = (pHash * BASE + P[i]) % MOD;
11         tHash = (tHash * BASE + T[i]) % MOD;
12         if (i > 0) basePow = basePow * BASE % MOD;
13     }
14
15     for (int i = 0; i <= n - m; i++) {
16         if (pHash == tHash && T.substr(i, m) == P)
17             matches.push_back(i);
18         if (i < n - m) {
19             tHash = (tHash - T[i] * basePow % MOD + MOD) % MOD;
20             tHash = (tHash * BASE + T[i + m]) % MOD;
21         }
22     }
23     return matches;
24 }
```

Listing 87: Rabin-Karp — expected $O(n + m)$

33.3 KMP (Knuth-Morris-Pratt)

KMP Algorithm

Precompute a **failure function** (prefix function) $\pi[j]$ = length of the longest proper prefix of $P[1..j]$ that is also a suffix. Use it to avoid re-examining characters.

Time: $O(n + m)$ guaranteed.

```

1 vector<int> prefixFunction(string& s) {
2     int n = s.size();
3     vector<int> pi(n, 0);
4     for (int i = 1; i < n; i++) {
5         int j = pi[i - 1];
6         while (j > 0 && s[i] != s[j]) j = pi[j - 1];
7         if (s[i] == s[j]) j++;
8     }
```

```

8     pi[i] = j;
9 }
10 return pi;
11 }
12
13 vector<int> kmpSearch(string& T, string& P) {
14     string combined = P + "#" + T; // # is a separator not in P or T
15     vector<int> pi = prefixFunction(combined);
16     int m = P.size();
17     vector<int> matches;
18
19     for (int i = m + 1; i < (int)combined.size(); i++)
20         if (pi[i] == m)
21             matches.push_back(i - 2 * m); // match at this position
22
23     return matches;
24 }

```

Listing 88: KMP prefix function and search — $O(n + m)$

Pattern:	a	b	a	b	a	b	c	a
π :	0	0	1	2	3	4	0	1

33.4 Z-Algorithm

```

1 vector<int> zFunction(string& s) {
2     int n = s.size();
3     vector<int> z(n, 0);
4     int l = 0, r = 0;
5     for (int i = 1; i < n; i++) {
6         if (i < r)
7             z[i] = min(r - i, z[i - l]);
8         while (i + z[i] < n && s[z[i]] == s[i + z[i]])
9             z[i]++;
10        if (i + z[i] > r) { l = i; r = i + z[i]; }
11    }
12    return z;
13 }
14
15 vector<int> zSearch(string& T, string& P) {
16     string combined = P + "$" + T;
17     vector<int> z = zFunction(combined);
18     int m = P.size();
19     vector<int> matches;
20     for (int i = m + 1; i < (int)combined.size(); i++)
21         if (z[i] == m)
22             matches.push_back(i - m - 1);
23     return matches;
24 }

```


Listing 89: Z-function and matching — $O(n + m)$

Algorithm	Preprocessing	Matching	Notes
Naive	—	$O(nm)$	Simple but slow
Rabin-Karp	$O(m)$	$O(n)$ expected	Hash-based; multiple patterns
KMP	$O(m)$	$O(n)$	Prefix function
Z-Algorithm	$O(m)$	$O(n)$	Alternative to KMP

Theorem 33.1 (Correctness of KMP Prefix Function). $\pi[i]$ = length of the longest proper prefix of $P[0..i]$ that is also a suffix.

Proof

Invariant: At the start of iteration i , j is the length of the longest proper prefix of $P[0..i-1]$ that is also a suffix, i.e., $j = \pi[i-1]$.

When processing position i :

- If $P[i] = P[j]$: the match extends, so $\pi[i] = j + 1$. ✓
- If $P[i] \neq P[j]$ and $j > 0$: fall back to $j = \pi[j-1]$. This is correct because $\pi[j-1]$ gives the next longest prefix-suffix of $P[0..j-1]$ that could still extend. We keep trying shorter prefixes until a match or $j = 0$. ✓
- If $P[i] \neq P[j]$ and $j = 0$: $\pi[i] = 0$ (no prefix-suffix). ✓

$O(m)$ **time:** j increases at most once per iteration (at most m times total) and the while loop decreases j each time. Total decreases $\leq$ total increases $\leq m$. So the while loop runs $O(m)$ times total across all iterations. $\square$

Theorem 33.2 (Correctness of KMP Search). KMP finds all occurrences of P in T in $O(n + m)$ time.

Proof

Concatenate $P + \# + T$ (where $\#$ doesn't appear in P or T). The prefix function of this string at position i gives the longest prefix of the combined string matching a suffix ending at i .

Since $\#$ acts as a barrier, $\pi[i]$ can never exceed m . When $\pi[i] = m$, the entire pattern P matches ending at position i in the combined string, corresponding to position $i - 2m$ in T .

The prefix function computation is $O(n + m)$ by the amortized argument above. $\square$

Theorem 33.3 (Correctness of Rabin-Karp). Rabin-Karp finds all occurrences of P in T .

Proof

Hash function: $h(s) = \sum_{i=0}^{m-1} s[i] \cdot b^{m-1-i} \bmod q$.

Rolling hash: When sliding the window one position right: $h(T[i+1..i+m]) = (h(T[i..i+m-1]) - T[i] \cdot b^{m-1}) \cdot b + T[i+m] \bmod q$. This correctly computes the

hash in $O(1)$ per slide. ✓

No false negatives: If $P = T[i..i+m-1]$, then $h(P) = h(T[i..i+m-1])$, so the hash match triggers a character-by-character verification, which confirms. ✓

No false positives: Hash matches are always verified by direct comparison. False hash matches (collisions) are detected and rejected. ✓

Expected time: With a random prime q , expected collisions = $O(n/q)$. Choosing q large enough makes expected time $O(n + m)$. □

34 Computational Geometry

34.1 Fundamental Primitives

Point and Vector

A **point** $p = (x, y)$ in 2D. A **vector** $\vec{v} = (x, y)$ represents direction and magnitude. Key operations:

- **Dot product:** $\vec{a} \cdot \vec{b} = a_x b_x + a_y b_y = |\vec{a}| |\vec{b}| \cos \theta$
- **Cross product:** $\vec{a} \times \vec{b} = a_x b_y - a_y b_x = |\vec{a}| |\vec{b}| \sin \theta$

```

1 struct Point {
2     double x, y;
3     Point(double x = 0, double y = 0) : x(x), y(y) {}
4     Point operator-(const Point& p) const { return {x-p.x, y-p.y}; }
5     Point operator+(const Point& p) const { return {x+p.x, y+p.y}; }
6     Point operator*(double t) const { return {x*t, y*t}; }
7     double dot(const Point& p) const { return x*p.x + y*p.y; }
8     double cross(const Point& p) const { return x*p.y - y*p.x; }
9     double norm() const { return sqrt(x*x + y*y); }
10 };

```

Listing 90: Point/vector operations

34.2 Cross Product and Orientation

Orientation Test

For three points p_1, p_2, p_3 , compute:

$$d = (p_2 - p_1) \times (p_3 - p_1)$$

- $d > 0$: p_3 is **left** of $\overrightarrow{p_1 p_2}$ (counter-clockwise).
- $d < 0$: p_3 is **right** (clockwise).
- $d = 0$: collinear.

This is the foundation of most computational geometry algorithms.

```

1 int orientation(Point a, Point b, Point c) {
2     double v = (b - a).cross(c - a);

```

```

3   if (v > 0) return 1; // CCW
4   if (v < 0) return -1; // CW
5   return 0;           // collinear
6 }
7
8 // Check if point p is on segment [a, b]
9 bool onSegment(Point a, Point b, Point p) {
10     return min(a.x, b.x) <= p.x && p.x <= max(a.x, b.x)
11         && min(a.y, b.y) <= p.y && p.y <= max(a.y, b.y);
12 }
13
14 // Do segments [a,b] and [c,d] intersect?
15 bool segmentsIntersect(Point a, Point b, Point c, Point d) {
16     int d1 = orientation(c, d, a);
17     int d2 = orientation(c, d, b);
18     int d3 = orientation(a, b, c);
19     int d4 = orientation(a, b, d);
20
21     if (d1 * d2 < 0 && d3 * d4 < 0) return true; // proper crossing
22
23     // Collinear cases
24     if (d1 == 0 && onSegment(c, d, a)) return true;
25     if (d2 == 0 && onSegment(c, d, b)) return true;
26     if (d3 == 0 && onSegment(a, b, c)) return true;
27     if (d4 == 0 && onSegment(a, b, d)) return true;
28     return false;
29 }

```

Listing 91: Orientation and segment intersection

34.3 Convex Hull

Convex Hull

The **convex hull** of a set of points is the smallest convex polygon containing all points. Think of stretching a rubber band around nails.

```

1 vector<Point> convexHull(vector<Point> pts) {
2     int n = pts.size();
3     if (n < 3) return pts;
4     sort(pts.begin(), pts.end(), [](auto& a, auto& b) {
5         return a.x < b.x || (a.x == b.x && a.y < b.y);
6     });
7
8     vector<Point> hull;
9     // Lower hull
10    for (auto& p : pts) {
11        while (hull.size() >= 2) {
12            Point a = hull[hull.size()-2], b = hull[hull.size()-1];
13            if ((b - a).cross(p - a) <= 0) hull.pop_back();
14            else break;

```

```

15     }
16     hull.push_back(p);
17 }
18 // Upper hull
19 int lower_size = hull.size();
20 for (int i = n - 2; i >= 0; i--) {
21     while ((int)hull.size() > lower_size) {
22         Point a = hull[hull.size()-2], b = hull[hull.size()-1];
23         if ((b - a).cross(pts[i] - a) <= 0) hull.pop_back();
24         else break;
25     }
26     hull.push_back(pts[i]);
27 }
28 hull.pop_back(); // remove duplicate of first point
29 return hull;
30 }

```

Listing 92: Andrew's monotone chain convex hull — $O(n \lg n)$

34.4 Closest Pair of Points

Closest Pair

Given n points, find the pair with minimum Euclidean distance.

- **Brute force:** $O(n^2)$.
- **Divide and conquer:** $O(n \lg n)$.

The D&C approach:

1. Sort by x -coordinate. Split into left and right halves.
2. Recursively find closest pair in each half: $\delta = \min(\delta_L, \delta_R)$.
3. Check cross-boundary pairs in a strip of width 2δ around the dividing line.
4. In the strip, sorted by y , each point needs to be compared with at most 7 others.

```

1 double closestPair(vector<Point>& pts) {
2     sort(pts.begin(), pts.end(), [](auto& a, auto& b) {
3         return a.x < b.x;
4     });
5     return closestRec(pts, 0, pts.size() - 1);
6 }
7
8 double closestRec(vector<Point>& pts, int l, int r) {
9     if (r - l < 3) {
10        // Brute force for small sets
11        double best = 1e18;
12        for (int i = l; i <= r; i++)
13            for (int j = i + 1; j <= r; j++)
14                best = min(best, (pts[i] - pts[j]).norm());
15        return best;
16    }
17    int mid = (l + r) / 2;
18    double midX = pts[mid].x;

```

```

19 double d = min(closestRec(pts, l, mid), closestRec(pts, mid+1, r));
20
21 // Merge: check strip
22 vector<Point> strip;
23 for (int i = l; i <= r; i++)
24     if (abs(pts[i].x - midX) < d)
25         strip.push_back(pts[i]);
26 sort(strip.begin(), strip.end(), [](auto& a, auto& b) {
27     return a.y < b.y;
28 });
29 for (int i = 0; i < (int)strip.size(); i++)
30     for (int j = i+1; j < (int)strip.size()
31         && strip[j].y - strip[i].y < d; j++)
32         d = min(d, (strip[i] - strip[j]).norm());
33 return d;
34 }

```

Listing 93: Closest pair — $O(n \lg n)$

Computational Geometry Summary

Problem	Time	Key Technique
Orientation test	$O(1)$	Cross product
Segment intersection	$O(1)$	Orientation tests
Convex hull	$O(n \lg n)$	Monotone chain / Graham scan
Closest pair	$O(n \lg n)$	Divide and conquer
Line sweep (general)	$O(n \lg n)$	Sweep line + BST

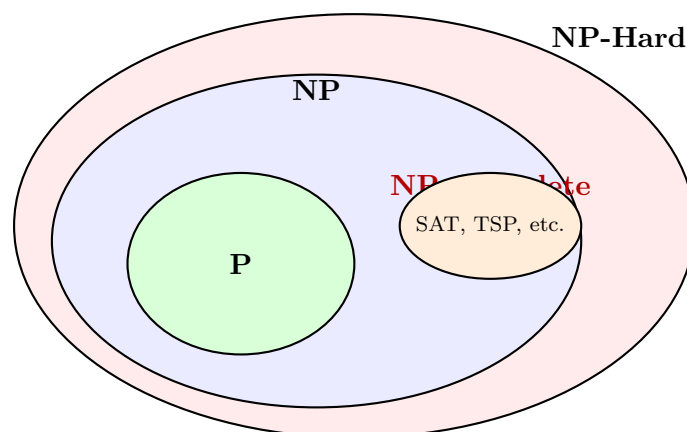
35 NP-Completeness

This chapter formalizes the notion of computational hardness and the class of problems for which no polynomial-time algorithm is known.

35.1 Complexity Classes

Complexity Classes

- **P**: problems solvable in **polynomial time** by a deterministic Turing machine. (e.g., sorting, shortest paths)
- **NP**: problems whose solutions can be **verified** in polynomial time. (e.g., given a Hamiltonian cycle, we can check it in $O(n)$)
- **NP-Hard**: at least as hard as every problem in NP.
- **NP-Complete (NPC)**: in NP **and** NP-Hard. The “hardest” problems in NP.



Theorem 35.1 (The P vs NP Question). Is $P = NP$? This is the most important open problem in computer science. Most experts believe $P \neq NP$, meaning NP-Complete problems have no polynomial-time algorithms.

35.2 Polynomial-Time Reductions

Polynomial-Time Reduction

Problem A **reduces to** problem B (written $A \leq_P B$) if there exists a polynomial-time function that transforms any instance of A into an instance of B with the same answer.

If $A \leq_P B$ and $B \in P$, then $A \in P$.

To prove B is NP-Complete:

1. Show $B \in NP$ (solutions can be verified in polynomial time).
2. Show $A \leq_P B$ for some known NP-Complete problem A .

35.3 The First NP-Complete Problem: SAT

Theorem 35.2 (Cook-Levin Theorem (1971)). **SAT** (Boolean Satisfiability) is NP-Complete.

Given a Boolean formula ϕ in CNF (conjunction of clauses), is there an assignment of variables that makes ϕ true?

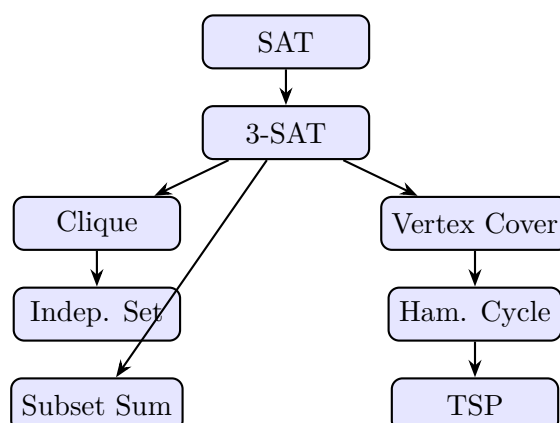
Example: $(x_1 \vee \bar{x}_2) \wedge (\bar{x}_1 \vee x_3) \wedge (x_2 \vee x_3)$

Every problem in NP can be reduced to SAT (by encoding the verification Turing machine as a Boolean formula).

35.4 Classic NP-Complete Problems

Problem	Description
SAT	Is a Boolean formula satisfiable?
3-SAT	SAT where each clause has exactly 3 literals.
Clique	Does graph G have a clique of size k ?
Vertex Cover	Can you cover all edges with $\leq k$ vertices?
Independent Set	Is there an independent set of size k ?
Hamiltonian Cycle	Does G have a cycle visiting every vertex exactly once?
TSP (decision)	Is there a tour of total weight $\leq k$?
Subset Sum	Does a subset sum to exactly W ?
Graph Coloring	Can G be k -colored (for $k \geq 3$)?
Partition	Can a set be divided into two equal-sum halves?

35.5 Reduction Chain



35.6 Dealing with NP-Complete Problems

Strategies for NP-Hard Problems

When you encounter an NP-Complete problem:

1. **Small input:** Exact algorithms with exponential time. Bitmask DP $O(2^n \cdot n)$, backtracking with pruning.
2. **Approximation:** Find a solution within a guaranteed factor of optimal (Chapter 35).
3. **Heuristics:** Simulated annealing, genetic algorithms, local search. No guarantees, but often work well in practice.
4. **Special structure:** Many NP-hard problems become polynomial on trees, planar graphs, bounded-treewidth graphs, etc.

5. **Parameterized complexity:** FPT algorithms, e.g., vertex cover in $O(2^k \cdot n)$ where k is the cover size.

```

1 int tsp(vector<vector<int>>& dist, int n) {
2     // dp[mask][i] = min cost to visit vertices in mask, ending at i
3     vector<vector<int>> dp(1 << n, vector<int>(n, INT_MAX));
4     dp[1][0] = 0; // start at vertex 0
5
6     for (int mask = 1; mask < (1 << n); mask++) {
7         for (int u = 0; u < n; u++) {
8             if (dp[mask][u] == INT_MAX) continue;
9             if (!(mask & (1 << u))) continue;
10            for (int v = 0; v < n; v++) {
11                if (mask & (1 << v)) continue; // already visited
12                int newMask = mask | (1 << v);
13                dp[newMask][v] = min(dp[newMask][v],
14                                    dp[mask][u] + dist[u][v]);
15            }
16        }
17    }
18    // Return to start
19    int full = (1 << n) - 1;
20    int ans = INT_MAX;
21    for (int u = 1; u < n; u++)
22        if (dp[full][u] != INT_MAX)
23            ans = min(ans, dp[full][u] + dist[u][0]);
24    return ans;
25 }

```

Listing 94: TSP via bitmask DP — $O(2^n \cdot n^2)$

36 Approximation Algorithms

When an optimization problem is NP-Hard, we settle for solutions that are provably *close* to optimal.

36.1 Approximation Ratio

Approximation Ratio

An algorithm has **approximation ratio** $\rho(n)$ if for every input of size n :

$$\max\left(\frac{C}{C^*}, \frac{C^*}{C}\right) \leq \rho(n)$$

where C is the algorithm's cost and C^* is the optimal cost.

A ρ -**approximation algorithm** guarantees a solution within factor ρ of optimal.

36.2 Vertex Cover: 2-Approximation

Vertex Cover Problem

Find the smallest set $S \subseteq V$ such that every edge has at least one endpoint in S . This is NP-Hard.

Greedy: Repeatedly pick an arbitrary uncovered edge (u, v) , add *both* endpoints to the cover.

```

1 set<int> approxVertexCover(int n, vector<pair<int,int>>& edges) {
2     set<int> cover;
3     vector<bool> edgeCovered(edges.size(), false);
4
5     for (int i = 0; i < (int)edges.size(); i++) {
6         if (edgeCovered[i]) continue;
7         auto [u, v] = edges[i];
8         if (cover.count(u) || cover.count(v)) continue;
9         cover.insert(u);
10        cover.insert(v);
11    }
12    return cover;
13    // |cover| <= 2 * OPT
14 }
```

Listing 95: Vertex cover 2-approximation — $O(V + E)$

Why Factor 2?

Each chosen edge (u, v) contributes 2 vertices. But the optimal cover must include *at least one* of u, v for that edge. The edges we chose are a **matching** (no shared vertices), so $\text{OPT} \geq |\text{matching}|$. Our cover has $2 \cdot |\text{matching}|$ vertices. Thus ratio ≤ 2 . $\square$

36.3 TSP: 2-Approximation (Metric TSP)

When distances satisfy the **triangle inequality** ($d(u, w) \leq d(u, v) + d(v, w)$):

1. Compute MST T of the complete graph.
2. Take a **full walk** (DFS traversal) of T , listing vertices as visited.
3. **Shortcut** repeated vertices (go directly to the next unvisited).

Analysis: MST cost $\leq \text{OPT}$ (removing one edge from optimal tour gives a spanning tree). The full walk costs $2 \cdot \text{MST}$. Shortcuts only reduce cost (triangle inequality). So: tour cost $\leq 2 \cdot \text{MST} \leq 2 \cdot \text{OPT}$.

Christofides' algorithm improves this to a $3/2$ -approximation using minimum-weight perfect matching.

36.4 Set Cover: $O(\lg n)$ -Approximation

Set Cover Problem

Given a universe U and a collection $\mathcal{S}$ of subsets, find the smallest sub-collection that covers U . NP-Hard.

Greedy: Repeatedly pick the set that covers the most uncovered elements.

```

1 vector<int> greedySetCover(int n, vector<vector<int>>& sets) {
2     vector<bool> covered(n, false);
3     int remaining = n;
4     vector<int> chosen;
5
6     while (remaining > 0) {
7         // Pick set covering the most uncovered elements
8         int bestSet = -1, bestCount = 0;
9         for (int s = 0; s < (int)sets.size(); s++) {
10             int cnt = 0;
11             for (int e : sets[s])
12                 if (!covered[e]) cnt++;
13             if (cnt > bestCount) { bestCount = cnt; bestSet = s; }
14         }
15         // Add it
16         chosen.push_back(bestSet);
17         for (int e : sets[bestSet])
18             if (!covered[e]) { covered[e] = true; remaining--; }
19     }
20     return chosen;
21     // |chosen| <= H_n * OPT = O(lg n) * OPT
22 }
```

Listing 96: Greedy set cover — $O(\lg n)$ -approximation

Theorem 36.1. The greedy set cover achieves approximation ratio $H_n = \sum_{k=1}^n 1/k = O(\lg n)$, where $n = |U|$. This is **tight**: no polynomial-time algorithm can do better than $c \cdot \ln n$ unless $P = NP$.

36.5 Subset Sum: FPTAS

FPTAS

A **Fully Polynomial-Time Approximation Scheme** (FPTAS) achieves $(1 + \epsilon)$ -approximation in time polynomial in both n and $1/\epsilon$.

For subset sum: round values to reduce precision, then run exact DP on the rounded instance. Time: $O(n^2/\epsilon)$.

Approximation Ratio Summary

Problem	Ratio	Technique
Vertex Cover	2	Maximal matching
Metric TSP	2	MST + shortcuts
Metric TSP	$3/2$	Christofides (MST + matching)
Set Cover	$O(\lg n)$	Greedy
Max-Cut	2	Random assignment
Subset Sum	$1 + \epsilon$	FPTAS (rounding + DP)
Knapsack	$1 + \epsilon$	FPTAS
Bin Packing	~ 1.5	First Fit Decreasing

Inapproximability

Some problems *cannot* be approximated well (unless $P = NP$):

- **General TSP** (no triangle inequality): no constant-factor approximation.
- **Graph Coloring**: hard to approximate within $n^{1-\epsilon}$.
- **Clique**: hard to approximate within $n^{1-\epsilon}$.
- **Set Cover**: cannot beat $(1 - \epsilon) \ln n$.

The PCP Theorem and Unique Games Conjecture give tight inapproximability bounds.

Appendix: LeetCode Practice Problems

A curated set of LeetCode problems (1 Medium + 1 Hard) mapped to each CLRS chapter topic, with complete C++ solutions.

Sorting

LC 56 — Merge Intervals (Medium)

Given an array of intervals, merge all overlapping intervals.

```

1 vector<vector<int>> merge(vector<vector<int>>& intervals) {
2     sort(intervals.begin(), intervals.end());
3     vector<vector<int>> res;
4     for (auto& iv : intervals) {
5         if (!res.empty() && iv[0] <= res.back()[1])
6             res.back()[1] = max(res.back()[1], iv[1]);
7         else
8             res.push_back(iv);
9     }
10    return res;
11 }
```

Listing 97: LC 56: Merge Intervals — $O(n \lg n)$

LC 315 — Count of Smaller Numbers After Self (Hard)

For each $A[i]$, count how many $A[j] < A[i]$ with $j > i$. Uses merge sort to count inversions.

```

1 vector<int> counts;
2 vector<pair<int,int>> arr; // {value, original index}
3
4 void mergeCount(int lo, int hi) {
5     if (hi - lo <= 1) return;
6     int mid = (lo + hi) / 2;
7     mergeCount(lo, mid);
8     mergeCount(mid, hi);
9
10    vector<pair<int,int>> tmp;
11    int i = lo, j = mid, smaller = 0;
12    while (i < mid && j < hi) {
13        if (arr[j].first < arr[i].first) {
14            smaller++;
15            tmp.push_back(arr[j++]);
16        } else {
17            counts[arr[i].second] += smaller;
18            tmp.push_back(arr[i++]);
19        }
20    }
21    while (i < mid) {
22        counts[arr[i].second] += smaller;
23        tmp.push_back(arr[i++]);
```

```

24     }
25     while (j < hi) tmp.push_back(arr[j++]);
26     copy(tmp.begin(), tmp.end(), arr.begin() + lo);
27 }
28
29 vector<int> countSmaller(vector<int>& nums) {
30     int n = nums.size();
31     counts.assign(n, 0);
32     arr.resize(n);
33     for (int i = 0; i < n; i++) arr[i] = {nums[i], i};
34     mergeCount(0, n);
35     return counts;
36 }

```

Listing 98: LC 315: Count Smaller — modified merge sort $O(n \lg n)$ **LC 493 — Reverse Pairs (Hard)**

Count pairs (i, j) where $i < j$ and $A[i] > 2 \cdot A[j]$. Modified merge sort.

```

1 int reversePairs(vector<int>& nums) {
2     return mergeSort(nums, 0, nums.size() - 1);
3 }
4 int mergeSort(vector<int>& a, int lo, int hi) {
5     if (lo >= hi) return 0;
6     int mid = lo + (hi - lo) / 2;
7     int cnt = mergeSort(a, lo, mid) + mergeSort(a, mid+1, hi);
8     // Count cross pairs before merging
9     int j = mid + 1;
10    for (int i = lo; i <= mid; i++) {
11        while (j <= hi && (long long)a[i] > 2LL * a[j]) j++;
12        cnt += j - (mid + 1);
13    }
14    inplace_merge(a.begin()+lo, a.begin()+mid+1, a.begin()+hi+1);
15    return cnt;
16 }

```

Listing 99: LC 493: Reverse Pairs — merge sort $O(n \lg n)$ **Heaps / Priority Queues****LC 215 — Kth Largest Element in an Array (Medium)**

```

1 int findKthLargest(vector<int>& nums, int k) {
2     priority_queue<int, vector<int>, greater<int>> pq; // min-heap
3     for (int x : nums) {
4         pq.push(x);
5         if ((int)pq.size() > k) pq.pop();
6     }
7     return pq.top();
8 }

```

Listing 100: LC 215: Kth Largest — min-heap of size k , $O(n \lg k)$

LC 295 — Find Median from Data Stream (Hard)

Maintain two heaps: max-heap for lower half, min-heap for upper half.

```

1 class MedianFinder {
2     priority_queue<int> lo; // max-heap
3     priority_queue<int, vector<int>, greater<int>> hi; // min-heap
4 public:
5     void addNum(int num) {
6         lo.push(num);
7         hi.push(lo.top()); lo.pop();
8         if (hi.size() > lo.size()) {
9             lo.push(hi.top()); hi.pop();
10        }
11    }
12    double findMedian() {
13        if (lo.size() > hi.size()) return lo.top();
14        return (lo.top() + hi.top()) / 2.0;
15    }
16 };

```

Listing 101: LC 295: Median Finder — two heaps, $O(\lg n)$ per add

LC 23 — Merge K Sorted Lists (Hard)

Merge k sorted linked lists into one using a min-heap.

```

1 ListNode* mergeKLists(vector<ListNode*>& lists) {
2     auto cmp = [](ListNode* a, ListNode* b) {
3         return a->val > b->val;
4     };
5     priority_queue<ListNode*, vector<ListNode*>, decltype(cmp)> pq(cmp);
6     for (auto* l : lists)
7         if (l) pq.push(l);
8
9     ListNode dummy(0), *tail = &dummy;
10    while (!pq.empty()) {
11        auto* node = pq.top(); pq.pop();
12        tail->next = node;
13        tail = tail->next;
14        if (node->next) pq.push(node->next);
15    }
16    return dummy.next;
17 }

```

Listing 102: LC 23: Merge K Sorted Lists — heap $O(n \lg k)$

Stacks & Queues

LC 155 — Min Stack (Medium)

```

1 class MinStack {
2     stack<pair<int,int>> st; // {value, current_min}
3 public:
4     void push(int val) {
5         int mn = st.empty() ? val : min(val, st.top().second);
6         st.push({val, mn});
7     }
8     void pop() { st.pop(); }
9     int top() { return st.top().first; }
10    int getMin() { return st.top().second; }
11 };

```

Listing 103: LC 155: Min Stack — $O(1)$ all operations

LC 84 — Largest Rectangle in Histogram (Hard)

```

1 int largestRectangleArea(vector<int>& heights) {
2     stack<int> st;
3     int maxArea = 0, n = heights.size();
4     for (int i = 0; i <= n; i++) {
5         int h = (i == n) ? 0 : heights[i];
6         while (!st.empty() && h < heights[st.top()]) {
7             int height = heights[st.top()]; st.pop();
8             int width = st.empty() ? i : (i - st.top() - 1);
9             maxArea = max(maxArea, height * width);
10        }
11        st.push(i);
12    }
13    return maxArea;
14 }

```

Listing 104: LC 84: Largest Rectangle — monotonic stack $O(n)$

LC 42 — Trapping Rain Water (Hard)

```

1 int trap(vector<int>& h) {
2     int l = 0, r = h.size() - 1;
3     int lmax = 0, rmax = 0, water = 0;
4     while (l < r) {
5         if (h[l] < h[r]) {
6             lmax = max(lmax, h[l]);
7             water += lmax - h[l];
8             l++;
9         } else {
10            rmax = max(rmax, h[r]);
11            water += rmax - h[r];
12            r--;

```

```

13     }
14 }
15 return water;
16 }

```

Listing 105: LC 42: Trapping Rain Water — two pointers $O(n)$

Hash Tables

LC 1 — Two Sum (Medium)

```

1 vector<int> twoSum(vector<int>& nums, int target) {
2     unordered_map<int,int> seen;
3     for (int i = 0; i < (int)nums.size(); i++) {
4         int complement = target - nums[i];
5         if (seen.count(complement))
6             return {seen[complement], i};
7         seen[nums[i]] = i;
8     }
9     return {};
10 }

```

Listing 106: LC 1: Two Sum — hash map $O(n)$

LC 76 — Minimum Window Substring (Hard)

Find shortest substring of s containing all characters of t .

```

1 string minWindow(string s, string t) {
2     unordered_map<char,int> need, have;
3     for (char c : t) need[c]++;
4     int required = need.size(), formed = 0;
5     int l = 0, minLen = INT_MAX, minStart = 0;
6
7     for (int r = 0; r < (int)s.size(); r++) {
8         have[s[r]]++;
9         if (need.count(s[r]) && have[s[r]] == need[s[r]])
10             formed++;
11         while (formed == required) {
12             if (r - l + 1 < minLen) {
13                 minLen = r - l + 1;
14                 minStart = l;
15             }
16             have[s[l]]--;
17             if (need.count(s[l]) && have[s[l]] < need[s[l]])
18                 formed--;
19             l++;
20         }
21     }
22     return minLen == INT_MAX ? "" : s.substr(minStart, minLen);
23 }

```

Listing 107: LC 76: Min Window Substring — sliding window + hash $O(n)$

LC 41 — First Missing Positive (Hard)

Find the smallest missing positive integer in $O(n)$ time, $O(1)$ space. Use the array itself as a hash table: place each value v at index $v-1$.

```

1 int firstMissingPositive(vector<int>& nums) {
2     int n = nums.size();
3     // Place each number in its correct position
4     for (int i = 0; i < n; i++)
5         while (nums[i] > 0 && nums[i] <= n
6             && nums[nums[i]-1] != nums[i])
7             swap(nums[i], nums[nums[i]-1]);
8     // Find first mismatch
9     for (int i = 0; i < n; i++)
10        if (nums[i] != i + 1) return i + 1;
11    return n + 1;
12 }
```

Listing 108: LC 41: First Missing Positive — cyclic sort $O(n)$

Binary Search Trees

LC 98 — Validate Binary Search Tree (Medium)

```

1 bool isValidBST(TreeNode* root, long lo = LONG_MIN, long hi = LONG_MAX) {
2     if (!root) return true;
3     if (root->val <= lo || root->val >= hi) return false;
4     return isValidBST(root->left, lo, root->val)
5         && isValidBST(root->right, root->val, hi);
6 }
```

Listing 109: LC 98: Validate BST — recursive bounds $O(n)$

LC 124 — Binary Tree Maximum Path Sum (Hard)

A path can start and end at any node. Find the maximum sum.

```

1 int ans;
2 int maxGain(TreeNode* node) {
3     if (!node) return 0;
4     int left = max(0, maxGain(node->left)); // ignore negative
5     int right = max(0, maxGain(node->right));
6     ans = max(ans, left + right + node->val); // path through node
7     return max(left, right) + node->val; // best single branch
8 }
9 int maxPathSum(TreeNode* root) {
10    ans = INT_MIN;
11    maxGain(root);
```

```

12     return ans;
13 }

```

Listing 110: LC 124: Max Path Sum — DFS $O(n)$ **LC 297 — Serialize and Deserialize Binary Tree (Hard)**

```

1  class Codec {
2  public:
3      string serialize(TreeNode* root) {
4          if (!root) return "#";
5          return to_string(root->val) + "," +
6                  serialize(root->left) + "," +
7                  serialize(root->right);
8      }
9
10     TreeNode* deserialize(string data) {
11         queue<string> tokens;
12         stringstream ss(data);
13         string tok;
14         while (getline(ss, tok, ',')) tokens.push(tok);
15         return build(tokens);
16     }
17
18     TreeNode* build(queue<string>& q) {
19         string val = q.front(); q.pop();
20         if (val == "#") return nullptr;
21         TreeNode* node = new TreeNode(stoi(val));
22         node->left = build(q);
23         node->right = build(q);
24         return node;
25     }
26 };

```

Listing 111: LC 297: Serialize/Deserialize BST — preorder $O(n)$ **Dynamic Programming****LC 322 — Coin Change (Medium)**

Find minimum coins to make amount. Unbounded knapsack variant.

```

1  int coinChange(vector<int>& coins, int amount) {
2      vector<int> dp(amount + 1, amount + 1);
3      dp[0] = 0;
4      for (int i = 1; i <= amount; i++)
5          for (int c : coins)
6              if (c <= i)
7                  dp[i] = min(dp[i], dp[i - c] + 1);
8      return dp[amount] > amount ? -1 : dp[amount];
9  }

```

Listing 112: LC 322: Coin Change — DP $O(n \cdot \text{amount})$ **LC 72 — Edit Distance (Hard)**

Minimum insertions, deletions, and substitutions to transform $s \rightarrow t$.

```

1 int minDistance(string word1, string word2) {
2     int m = word1.size(), n = word2.size();
3     vector<vector<int>> dp(m+1, vector<int>(n+1));
4     for (int i = 0; i <= m; i++) dp[i][0] = i;
5     for (int j = 0; j <= n; j++) dp[0][j] = j;
6
7     for (int i = 1; i <= m; i++)
8         for (int j = 1; j <= n; j++) {
9             if (word1[i-1] == word2[j-1])
10                dp[i][j] = dp[i-1][j-1];
11             else
12                dp[i][j] = 1 + min({dp[i-1][j],    // delete
13                                   dp[i][j-1],      // insert
14                                   dp[i-1][j-1]});    // replace
15         }
16     return dp[m][n];
17 }
```

Listing 113: LC 72: Edit Distance — DP $O(mn)$ **LC 312 — Burst Balloons (Hard)**

Burst balloons to maximize coins. Interval DP: $dp[l][r]$ = max coins from bursting all balloons in (l, r) , where l and r are boundaries.

```

1 int maxCoins(vector<int>& nums) {
2     int n = nums.size();
3     nums.insert(nums.begin(), 1);
4     nums.push_back(1); // sentinel boundaries
5     // dp[l][r] = max coins from bursting (l, r) exclusive
6     vector<vector<int>> dp(n+2, vector<int>(n+2, 0));
7     for (int len = 1; len <= n; len++)
8         for (int l = 0; l + len + 1 <= n + 1; l++) {
9             int r = l + len + 1;
10            for (int k = l+1; k < r; k++) // k = last balloon to burst
11                dp[l][r] = max(dp[l][r],
12                               dp[l][k] + dp[k][r] + nums[l]*nums[k]*nums[r]);
13        }
14     return dp[0][n+1];
15 }
```

Listing 114: LC 312: Burst Balloons — interval DP $O(n^3)$

Greedy Algorithms

LC 55 — Jump Game (Medium)

Can you reach the last index? Track the farthest reachable position.

```

1 bool canJump(vector<int>& nums) {
2     int farthest = 0;
3     for (int i = 0; i < (int)nums.size(); i++) {
4         if (i > farthest) return false;
5         farthest = max(farthest, i + nums[i]);
6     }
7     return true;
8 }

```

Listing 115: LC 55: Jump Game — greedy $O(n)$

LC 135 — Candy (Hard)

Each child gets at least 1 candy. Higher-rated children get more than neighbors.

```

1 int candy(vector<int>& ratings) {
2     int n = ratings.size();
3     vector<int> candies(n, 1);
4     // Left to right: ensure right neighbor > left if higher rated
5     for (int i = 1; i < n; i++)
6         if (ratings[i] > ratings[i-1])
7             candies[i] = candies[i-1] + 1;
8     // Right to left: ensure left neighbor > right if higher rated
9     for (int i = n-2; i >= 0; i--)
10        if (ratings[i] > ratings[i+1])
11            candies[i] = max(candies[i], candies[i+1] + 1);
12    return accumulate(candies.begin(), candies.end(), 0);
13 }

```

Listing 116: LC 135: Candy — two-pass greedy $O(n)$

LC 45 — Jump Game II (Hard)

Minimum number of jumps to reach the end. BFS-style greedy.

```

1 int jump(vector<int>& nums) {
2     int jumps = 0, curEnd = 0, farthest = 0;
3     for (int i = 0; i < (int)nums.size() - 1; i++) {
4         farthest = max(farthest, i + nums[i]);
5         if (i == curEnd) {
6             jumps++;
7             curEnd = farthest;
8         }
9     }
10    return jumps;
11 }

```

Listing 117: LC 45: Jump Game II — greedy BFS $O(n)$

Union-Find

LC 200 — Number of Islands (Medium)

```

1 int numIslands(vector<vector<char>>& grid) {
2     int m = grid.size(), n = grid[0].size(), count = 0;
3     for (int i = 0; i < m; i++)
4         for (int j = 0; j < n; j++)
5             if (grid[i][j] == '1') {
6                 count++;
7                 // Flood fill to sink the island
8                 queue<pair<int,int>> q;
9                 q.push({i, j});
10                grid[i][j] = '0';
11                while (!q.empty()) {
12                    auto [r, c] = q.front(); q.pop();
13                    for (auto [dr, dc] : vector<pair<int,int>>
14                        {{0,1},{0,-1},{1,0},{-1,0}}) {
15                        int nr = r+dr, nc = c+dc;
16                        if (nr>=0 && nr<m && nc>=0 && nc<n
17                            && grid[nr][nc]=='1') {
18                            grid[nr][nc] = '0';
19                            q.push({nr, nc});
20                        }
21                    }
22                }
23            }
24     return count;
25 }

```

Listing 118: LC 200: Number of Islands — DFS/Union-Find $O(mn)$

LC 839 — Similar String Groups (Hard)

Two strings are similar if they differ by exactly one swap. Find number of groups.

```

1 bool isSimilar(string& a, string& b) {
2     int diff = 0;
3     for (int i = 0; i < (int)a.size(); i++)
4         if (a[i] != b[i] && ++diff > 2) return false;
5     return true;
6 }
7
8 int numSimilarGroups(vector<string>& strs) {
9     int n = strs.size();
10    DSU dsu(n);
11    for (int i = 0; i < n; i++)
12        for (int j = i+1; j < n; j++)
13            if (isSimilar(strs[i], strs[j]))
14                dsu.unite(i, j);
15
16    set<int> groups;

```

```

17   for (int i = 0; i < n; i++) groups.insert(dsu.find(i));
18   return groups.size();
19 }

```

Listing 119: LC 839: Similar String Groups — Union-Find $O(n^2 \cdot m)$ **LC 685 — Redundant Connection II (Hard)**

In a rooted tree with one extra edge, find the edge to remove. Handles both duplicate-parent and cycle cases using Union-Find.

```

1  vector<int> findRedundantDirectedConnection(
2      vector<vector<int>>& edges) {
3      int n = edges.size();
4      vector<int> parent(n+1, 0);
5      vector<int> cand1, cand2; // two candidate edges
6      // Find node with two parents
7      for (auto& e : edges) {
8          if (parent[e[1]] != 0) {
9              cand1 = {parent[e[1]], e[1]};
10             cand2 = e;
11             e[1] = 0; // invalidate edge
12         } else {
13             parent[e[1]] = e[0];
14         }
15     }
16     // Union-Find to detect cycle
17     iota(parent.begin(), parent.end(), 0);
18     function<int(int)> find = [&](int x) {
19         return parent[x] == x ? x : parent[x] = find(parent[x]);
20     };
21     for (auto& e : edges) {
22         if (e[1] == 0) continue;
23         int pu = find(e[0]), pv = find(e[1]);
24         if (pu == pv)
25             return cand1.empty() ? e : cand1;
26         parent[pv] = pu;
27     }
28     return cand2;
29 }

```

Listing 120: LC 685: Redundant Connection II — Union-Find $O(n)$ **Graph Algorithms****LC 207 — Course Schedule (Medium)**

Detect cycle in a directed graph (topological sort).

```

1  bool canFinish(int n, vector<vector<int>>& prereqs) {
2      vector<vector<int>> adj(n);
3      vector<int> indeg(n, 0);

```

```

4   for (auto& p : prereqs) {
5       adj[p[1]].push_back(p[0]);
6       indeg[p[0]]++;
7   }
8   queue<int> q;
9   for (int i = 0; i < n; i++)
10      if (indeg[i] == 0) q.push(i);
11   int count = 0;
12   while (!q.empty()) {
13       int u = q.front(); q.pop(); count++;
14       for (int v : adj[u])
15          if (--indeg[v] == 0) q.push(v);
16   }
17   return count == n;
18 }

```

Listing 121: LC 207: Course Schedule — Kahn's topo sort $O(V + E)$ **LC 127 — Word Ladder (Hard)**

Find shortest transformation sequence from beginWord to endWord (BFS on implicit graph).

```

1   int ladderLength(string beginWord, string endWord,
2       vector<string>& wordList) {
3       unordered_set<string> dict(wordList.begin(), wordList.end());
4       if (!dict.count(endWord)) return 0;
5
6       queue<pair<string,int>> q;
7       q.push({beginWord, 1});
8       dict.erase(beginWord);
9
10      while (!q.empty()) {
11          auto [word, steps] = q.front(); q.pop();
12          if (word == endWord) return steps;
13          for (int i = 0; i < (int)word.size(); i++) {
14              char orig = word[i];
15              for (char c = 'a'; c <= 'z'; c++) {
16                  word[i] = c;
17                  if (dict.count(word)) {
18                      dict.erase(word);
19                      q.push({word, steps + 1});
20                  }
21              }
22              word[i] = orig;
23          }
24      }
25      return 0;
26  }

```

Listing 122: LC 127: Word Ladder — BFS $O(n \cdot m^2)$

LC 332 — Reconstruct Itinerary (Hard)

Find the lexicographically smallest Eulerian path starting from “JFK”. Uses Hierholzer’s algorithm (DFS + post-order).

```

1 vector<string> findItinerary(vector<vector<string>>& tickets) {
2     unordered_map<string, multiset<string>> graph;
3     for (auto& t : tickets)
4         graph[t[0]].insert(t[1]);
5     vector<string> route;
6     function<void(string)> dfs = [&](string u) {
7         while (!graph[u].empty()) {
8             string next = *graph[u].begin();
9             graph[u].erase(graph[u].begin());
10            dfs(next);
11        }
12        route.push_back(u);
13    };
14    dfs("JFK");
15    reverse(route.begin(), route.end());
16    return route;
17 }
```

Listing 123: LC 332: Reconstruct Itinerary — Hierholzer $O(E \lg E)$

Shortest Paths**LC 743 — Network Delay Time (Medium)**

Dijkstra’s algorithm on a weighted directed graph.

```

1 int networkDelayTime(vector<vector<int>>& times, int n, int k) {
2     vector<vector<pair<int,int>>> adj(n + 1);
3     for (auto& t : times) adj[t[0]].push_back({t[1], t[2]});
4
5     vector<int> dist(n + 1, INT_MAX);
6     priority_queue<pair<int,int>, vector<pair<int,int>>,
7         greater<pair<int,int>>> pq;
8     dist[k] = 0;
9     pq.push({0, k});
10
11    while (!pq.empty()) {
12        auto [d, u] = pq.top(); pq.pop();
13        if (d > dist[u]) continue;
14        for (auto& [v, w] : adj[u])
15            if (dist[u] + w < dist[v]) {
16                dist[v] = dist[u] + w;
17                pq.push({dist[v], v});
18            }
19    }
20    int ans = *max_element(dist.begin()+1, dist.end());
21    return ans == INT_MAX ? -1 : ans;
22 }
```


Listing 124: LC 743: Network Delay — Dijkstra $O(E \lg V)$ **LC 787 — Cheapest Flights Within K Stops (Hard)**

Bellman-Ford variant with at most k relaxation rounds.

```

1 int findCheapestPrice(int n, vector<vector<int>>& flights,
2                       int src, int dst, int k) {
3     vector<int> dist(n, INT_MAX);
4     dist[src] = 0;
5     for (int i = 0; i <= k; i++) {
6         vector<int> tmp(dist); // copy: use only distances from previous round
7         for (auto& f : flights) {
8             int u = f[0], v = f[1], w = f[2];
9             if (dist[u] != INT_MAX)
10                tmp[v] = min(tmp[v], dist[u] + w);
11        }
12        dist = tmp;
13    }
14    return dist[dst] == INT_MAX ? -1 : dist[dst];
15 }
```

Listing 125: LC 787: Cheapest Flights — modified Bellman-Ford $O(k \cdot E)$ **LC 778 — Swim in Rising Water (Hard)**

Find the minimum time t such that you can swim from $(0, 0)$ to $(n-1, n-1)$ where cell (i, j) is underwater at time $\text{grid}[i][j]$. Modified Dijkstra.

```

1 int swimInWater(vector<vector<int>>& grid) {
2     int n = grid.size();
3     vector<vector<int>> dist(n, vector<int>(n, INT_MAX));
4     priority_queue<tuple<int, int, int>, vector<tuple<int, int, int>>,
5                   greater<>> pq;
6     dist[0][0] = grid[0][0];
7     pq.push({grid[0][0], 0, 0});
8     int dx[] = {0, 0, 1, -1}, dy[] = {1, -1, 0, 0};
9     while (!pq.empty()) {
10        auto [t, x, y] = pq.top(); pq.pop();
11        if (x == n-1 && y == n-1) return t;
12        if (t > dist[x][y]) continue;
13        for (int d = 0; d < 4; d++) {
14            int nx = x+dx[d], ny = y+dy[d];
15            if (nx < 0 || nx >= n || ny < 0 || ny >= n) continue;
16            int nt = max(t, grid[nx][ny]);
17            if (nt < dist[nx][ny]) {
18                dist[nx][ny] = nt;
19                pq.push({nt, nx, ny});
20            }
21        }
22    }
```

```

22     }
23     return -1;
24 }

```

Listing 126: LC 778: Swim in Rising Water — Dijkstra $O(n^2 \lg n)$

String Matching

LC 28 — Find the Index of the First Occurrence (Medium)

```

1  int strStr(string haystack, string needle) {
2      string combined = needle + "#" + haystack;
3      int m = needle.size(), n = combined.size();
4      vector<int> pi(n, 0);
5      for (int i = 1; i < n; i++) {
6          int j = pi[i-1];
7          while (j > 0 && combined[i] != combined[j]) j = pi[j-1];
8          if (combined[i] == combined[j]) j++;
9          pi[i] = j;
10         if (j == m) return i - 2 * m; // found
11     }
12     return -1;
13 }

```

Listing 127: LC 28: strStr — KMP $O(n + m)$

LC 214 — Shortest Palindrome (Hard)

Find shortest palindrome by adding characters to the front. Uses KMP to find the longest palindromic prefix.

```

1  string shortestPalindrome(string s) {
2      string rev = s;
3      reverse(rev.begin(), rev.end());
4      string combined = s + "#" + rev;
5      int n = combined.size();
6      vector<int> pi(n, 0);
7      for (int i = 1; i < n; i++) {
8          int j = pi[i-1];
9          while (j > 0 && combined[i] != combined[j]) j = pi[j-1];
10         if (combined[i] == combined[j]) j++;
11         pi[i] = j;
12     }
13     // pi.back() = length of longest palindromic prefix of s
14     string add = rev.substr(0, s.size() - pi.back());
15     return add + s;
16 }

```

Listing 128: LC 214: Shortest Palindrome — KMP trick $O(n)$

LC 336 — Palindrome Pairs (Hard)

Find all pairs (i, j) where $\text{words}[i] + \text{words}[j]$ is a palindrome. Use a hash map for reverse lookups.

```

1 vector<vector<int>> palindromePairs(vector<string>& words) {
2     unordered_map<string, int> mp;
3     for (int i = 0; i < (int)words.size(); i++) {
4         string r = words[i];
5         reverse(r.begin(), r.end());
6         mp[r] = i;
7     }
8     vector<vector<int>> result;
9     for (int i = 0; i < (int)words.size(); i++) {
10        string& w = words[i];
11        for (int j = 0; j <= (int)w.size(); j++) {
12            string left = w.substr(0, j);
13            string right = w.substr(j);
14            // If right is palindrome, check if reverse(left) exists
15            auto isPalin = [](string& s) {
16                int l=0, r=s.size()-1;
17                while (l<r) if (s[l++]!=s[r--]) return false;
18                return true;
19            };
20            if (isPalin(right) && mp.count(left) && mp[left] != i)
21                result.push_back({i, mp[left]});
22            if (j > 0 && isPalin(left) && mp.count(right)
23                && mp[right] != i)
24                result.push_back({mp[right], i});
25        }
26    }
27    return result;
28 }

```

Listing 129: LC 336: Palindrome Pairs — hash map $O(n \cdot m^2)$

Backtracking (NP-Complete in Practice)**LC 78 — Subsets (Medium)**

Generate all 2^n subsets.

```

1 vector<vector<int>> subsets(vector<int>& nums) {
2     vector<vector<int>> result;
3     vector<int> current;
4     function<void(int)> backtrack = [&](int i) {
5         result.push_back(current);
6         for (int j = i; j < (int)nums.size(); j++) {
7             current.push_back(nums[j]);
8             backtrack(j + 1);
9             current.pop_back();
10        }
11    };

```

```

12 backtrack(0);
13 return result;
14 }

```

Listing 130: LC 78: Subsets — backtracking $O(n \cdot 2^n)$ **LC 51 — N-Queens (Hard)**

Place n queens on an $n \times n$ board so no two attack each other.

```

1 vector<vector<string>> solveNQueens(int n) {
2     vector<vector<string>> result;
3     vector<string> board(n, string(n, '.'));
4     vector<bool> cols(n), diag1(2*n), diag2(2*n);
5
6     function<void(int)> solve = [&](int row) {
7         if (row == n) { result.push_back(board); return; }
8         for (int col = 0; col < n; col++) {
9             if (cols[col] || diag1[row+col] || diag2[row-col+n])
10                continue;
11            board[row][col] = 'Q';
12            cols[col] = diag1[row+col] = diag2[row-col+n] = true;
13            solve(row + 1);
14            board[row][col] = '.';
15            cols[col] = diag1[row+col] = diag2[row-col+n] = false;
16        }
17    };
18    solve(0);
19    return result;
20 }

```

Listing 131: LC 51: N-Queens — backtracking with pruning

LC 37 — Sudoku Solver (Hard)

```

1 void solveSudoku(vector<vector<char>>& board) {
2     bool row[9][9]={}, col[9][9]={}, box[9][9]={};
3     // Initialize constraints
4     for (int i = 0; i < 9; i++)
5         for (int j = 0; j < 9; j++)
6             if (board[i][j] != '.') {
7                 int d = board[i][j] - '1';
8                 row[i][d] = col[j][d] = box[i/3*3+j/3][d] = true;
9             }
10
11     function<bool(int, int)> solve = [&](int r, int c) -> bool {
12         if (r == 9) return true;
13         int nr = (c == 8) ? r+1 : r;
14         int nc = (c == 8) ? 0 : c+1;
15         if (board[r][c] != '.') return solve(nr, nc);
16         int b = r/3*3 + c/3;

```

```
17     for (int d = 0; d < 9; d++) {
18         if (row[r][d] || col[c][d] || box[b][d]) continue;
19         board[r][c] = '1' + d;
20         row[r][d] = col[c][d] = box[b][d] = true;
21         if (solve(nr, nc)) return true;
22         board[r][c] = '.';
23         row[r][d] = col[c][d] = box[b][d] = false;
24     }
25     return false;
26 };
27 solve(0, 0);
28 }
```

Listing 132: LC 37: Sudoku Solver — backtracking with constraint sets

Appendix: Suggested Practice Problems

Curated problems from LeetCode, GeeksForGeeks, and Codeforces mapped to CLRS topics. Practice these after studying each chapter.

Arrays and Sorting (Ch 2, 7, 8)

LeetCode:

ID	Problem	Difficulty
75	Sort Colors (Dutch National Flag)	Medium
215	Kth Largest Element in an Array	Medium
493	Reverse Pairs	Hard
315	Count of Smaller Numbers After Self	Hard

GeeksForGeeks:

- Merge Sort — *GFG: Merge Sort* (Medium)
- Count Inversions — *GFG: Count Inversions* (Medium)
- Minimum Swaps to Sort — *GFG: Minimum Swaps to Sort* (Medium)

Codeforces:

Problem ID	Rating
CF 1324D	1400
CF 1791G2	1900
CF 1847C	1600

Divide and Conquer (Ch 4)

LeetCode:

ID	Problem	Difficulty
23	Merge K Sorted Lists	Hard
932	Beautiful Array	Medium
4	Median of Two Sorted Arrays	Hard

Codeforces:

Problem ID	Rating
CF 448C	1600
CF 768B	1800

Heaps and Priority Queues (Ch 6)

LeetCode:

ID	Problem	Difficulty
347	Top K Frequent Elements	Medium
373	Find K Pairs with Smallest Sums	Medium
295	Find Median from Data Stream	Hard
632	Smallest Range Covering Elements from K Lists	Hard

GeeksForGeeks:

- *GFG: Merge K Sorted Arrays* (Medium)
- *GFG: K-th Largest Element in a Stream* (Medium)

Codeforces:

Problem ID	Rating
CF 1462E2	1700
CF 1703G	2000

Stacks and Queues (Ch 10)**LeetCode:**

ID	Problem	Difficulty
739	Daily Temperatures	Medium
503	Next Greater Element II	Medium
84	Largest Rectangle in Histogram	Hard
42	Trapping Rain Water	Hard

GeeksForGeeks:

- *GFG: Stock Span Problem* (Medium)
- *GFG: The Celebrity Problem* (Medium)

Codeforces:

Problem ID	Rating
CF 1921D	1500
CF 1279C	1600

Hash Tables (Ch 11)**LeetCode:**

ID	Problem	Difficulty
49	Group Anagrams	Medium
560	Subarray Sum Equals K	Medium
76	Minimum Window Substring	Hard
41	First Missing Positive	Hard

Codeforces:

Problem ID	Rating
CF 1512D	1400
CF 1676G	1700

BST and Balanced Trees (Ch 12–14)

LeetCode:

ID	Problem	Difficulty
230	Kth Smallest Element in a BST	Medium
450	Delete Node in a BST	Medium
124	Binary Tree Maximum Path Sum	Hard
297	Serialize and Deserialize Binary Tree	Hard

GeeksForGeeks:

- *GFG: Check for BST* (Medium)
- *GFG: Merge Two BSTs* (Hard)
- *GFG: AVL Tree Insertion* (Medium)

Codeforces:

Problem ID	Rating
CF 675D	1900
CF 863D	2000

Dynamic Programming (Ch 15)

LeetCode:

ID	Problem	Difficulty
300	Longest Increasing Subsequence	Medium
1143	Longest Common Subsequence	Medium
312	Burst Balloons	Hard
10	Regular Expression Matching	Hard

GeeksForGeeks:

- *GFG: Matrix Chain Multiplication* (Hard)
- *GFG: Egg Dropping Puzzle* (Medium)
- *GFG: Optimal BST* (Hard)

Codeforces:

Problem ID	Rating
CF 1dp_contest	—
CF 1472G	2100
CF 1613D	1700
CF 1525D	2100

Greedy Algorithms (Ch 16)

LeetCode:

ID	Problem	Difficulty
452	Minimum Number of Arrows to Burst Balloons	Medium
763	Partition Labels	Medium
135	Candy	Hard
45	Jump Game II	Hard

GeeksForGeeks:

- *GFG: Activity Selection* (Medium)
- *GFG: Fractional Knapsack* (Medium)
- *GFG: Huffman Encoding* (Hard)

Codeforces:

Problem ID	Rating
CF 1637D	1800
CF 1700D	2000

Union-Find / Disjoint Sets (Ch 21)**LeetCode:**

ID	Problem	Difficulty
721	Accounts Merge	Medium
547	Number of Provinces	Medium
685	Redundant Connection II	Hard
839	Similar String Groups	Hard

Codeforces:

Problem ID	Rating
CF 1559D2	2000
CF 1851G	1900

Graph Algorithms — BFS/DFS (Ch 22)**LeetCode:**

ID	Problem	Difficulty
207	Course Schedule	Medium
802	Find Eventual Safe States	Medium
127	Word Ladder	Hard
332	Reconstruct Itinerary	Hard

GeeksForGeeks:

- *GFG: Detect Cycle in Directed Graph* (Medium)
- *GFG: Topological Sort* (Medium)
- *GFG: Strongly Connected Components (Kosaraju)* (Hard)

Codeforces:

Problem ID	Rating
CF 1594D	1700
CF 1593D2	1900
CF 1682E	2100

MST (Ch 23)**LeetCode:**

ID	Problem	Difficulty
1584	Min Cost to Connect All Points	Medium
1489	Find Critical and Pseudo-Critical Edges in MST	Hard

Codeforces:

Problem ID	Rating
CF 609E	2100
CF 160D	1800
CF 1245D	1800

Shortest Paths (Ch 24–25)**LeetCode:**

ID	Problem	Difficulty
743	Network Delay Time	Medium
1514	Path with Maximum Probability	Medium
787	Cheapest Flights Within K Stops	Hard
778	Swim in Rising Water	Hard

GeeksForGeeks:

- *GFG: Shortest Path in Weighted Undirected Graph* (Medium)
- *GFG: Negative Weight Cycle Detection* (Medium)
- *GFG: Floyd-Warshall* (Medium)

Codeforces:

Problem ID	Rating
CF 20C	1900
CF 1473E	2100
CF 1725M	2200

Network Flow (Ch 26)**LeetCode:**

ID	Problem	Difficulty
1820	Maximum Number of Accepted Invitations	Medium

Codeforces:

Problem ID	Rating
CF 546E	2100
CF 653D	2400

Number Theory (Ch 31)

LeetCode:

ID	Problem	Difficulty
204	Count Primes	Medium
50	Pow(x, n)	Medium

GeeksForGeeks:

- *GFG: Sieve of Eratosthenes* (Medium)
- *GFG: Euler's Totient Function* (Medium)
- *GFG: Modular Multiplicative Inverse* (Medium)

Codeforces:

Problem ID	Rating
CF 1349A	1600
CF 1114C	1700
CF 1900D	1800
CF 1294D	1900

String Matching (Ch 32)

LeetCode:

ID	Problem	Difficulty
28	Find the Index of the First Occurrence	Medium
686	Repeated String Match	Medium
214	Shortest Palindrome	Hard
336	Palindrome Pairs	Hard

Codeforces:

Problem ID	Rating
CF 126B	1700
CF 432D	2000
CF 1200E	2000

Backtracking / NP-Completeness (Ch 34)

LeetCode:

ID	Problem	Difficulty
46	Permutations	Medium
79	Word Search	Medium
51	N-Queens	Hard
37	Sudoku Solver	Hard

GeeksForGeeks:

- *GFG: Rat in a Maze* (Medium)

- *GFG: M-Coloring Problem* (Medium)
- *GFG: Hamiltonian Path* (Hard)

Codeforces:

Problem ID	Rating
CF 1594C	1500
CF 1863E	2000